

EC-Lab[®] Development Package

User's Guide
Version 6.11 – May 2024



History

Version	Description
V6.11	Support for VMP-3 board L version
V6.08	Support for VMP300-series P board.
V6.06	Support new VMP-300 boards VU version 3
V6.05	Support new VMP-300 boards VU version 2
V6.04	SP-50e / SP-150e support added 1A48VP booster support added MP-MEA (MuxPad) option support removed fixed USB for 64b apps
V6.03	VMP-3e / VSP-3e support open-in support for VMP3 BL_GetOptErr function description
V6.02	Bug fix on 100pA range update for VMP-300 series. Adding of BP300 device
V6.01	Restructured examples code.
V6.00	64-bits compatibility added for LABVIEW to resolve the problem of alignment.
V5.39	Extended techniques memory
V5.38	Optimized the loop technique for mandatory goto use
V5.37	Fixed xilinx version comparison at the end of LoadXilinx step.
V5.36	64-bits compatibility added. New DLLs (EClib64.dll, blfind64.dll) and new Delphi 64-bits examples. New form in Delphi examples showing how to search and select a device in a broadcast list (uses blfind.dll).
V5.35	Support for HCP-1005 and MPG-2xx
V5.34	Updated the kernel4.bin to enable new amplifiers (2A and 10A) for VMP-300 Updated DLL for the new Kernel Updated VMP-300 techniques for new record mode (see section 8) Updated xlx firmware to accommodate new kernel Updated C and C# examples for new kernel Timebase parameter added to some LabVIEW examples

	Updated Techniques description according to the new record mode
V5.33	Clarifications about BL_UpdateParameters usage Fixed references to non-existent section 4.1 Added C/C++ and C# code examples
V5.32	New USB driver for Windows 8 LabVIEW examples are now provided for LabVIEW version 12 and 8.5 User guide corrections: Reference to "section 4.2" replaced by "section 5.3. Constants"; TchannelInfos.NbAmps, TdataInfos.MuxPad, TCurrentValues.OptErr and OptPos fields were missing in the structure declaration
V5.31	Allows connection to SP-240 device. External control bug correction for VMP-300 series.
V5.29	Modular Pulse technique : add records mode and step index ECP v5.26
v5.28	Add BL_UpdateParameters_LV
v5.27	Add Modular Pulse technique Add CASG technique Add CASP technique kernel v5.25 Kernel4 v5.27 Add blfind.dll v1.1 Structure TChannelInfos modified
v5.26	Record and external control options Windows Installer New USB driver for Windows XP/Vista/Seven 32bits/64bits New fields: TdataInfos.MuxPad, TCurrentValues.OptErr and OptPos
v5.23	Add techniques with limits to VMP-300 series : vscanlimit, iscanlimit, calimit, cplimit. Option 4A amplifier for VMP-300 Allows connection to MPG2 device Add Floating / grounded mode to VMP-300 series Change name of constant

v5.22	Update LASV parameters (Record_every_dI, Record_every_dT) Kernel VMP3 v5.22 (Update change mode Potentio-Galvano with linked techniques)
v5.21	LASV (Large Amplitude sinusoidal voltammetry) Technique
V5.20	Control SP-300 and SP-200
v5.19	Description image with all technique Correction of parameter range
v5.18	First version

Overview.....	13
2. Files package.....	14
3. General information.....	16
3.1. Calling conventions.....	16
3.2. Data alignment.....	16
3.3. Multi-thread applications.....	16
3.4. Data types.....	16
3.5. Variables description.....	18
4. Using the library.....	19
5. Structures and Constants.....	20
5.1. Structures.....	20
5.2. Other structures.....	25
5.2.1. Labview Structures.....	25
5.3. Constants.....	26
5.4. Error codes.....	33
6. Functions reference.....	36
6.1. Functions overview.....	36
6.2. General functions.....	39
6.3. Communication functions.....	41
6.4. Firmware functions.....	45
6.5. Channel information functions.....	47
6.6. Technique functions.....	54
6.7. Start/stop functions.....	61
6.8. Data functions.....	65
6.9. Miscellaneous functions.....	71
7. Techniques.....	75
7.1. Notes.....	75
7.1.1 Recording additional values and XCTR changes from v5.34.....	75
7.1.2 Timebase calculation.....	76
7.2. Open Circuit Voltage technique.....	77
7.2.1. Description.....	77
7.2.2. Technique parameters.....	77
7.2.3. Data format.....	77
7.2.4. Data conversion.....	78
7.3. Cyclic Voltammetry technique.....	79
7.3.1. Description.....	79
7.3.2. Technique parameters.....	79

7.3.3. Data format.....	80
7.3.4. Data conversion.....	80
7.4. Cyclic Voltammetry Advanced technique.....	81
7.4.1. Description.....	81
7.4.1. Technique parameters.....	81
7.4.2. Data format.....	82
7.4.3. Data conversion.....	82
7.5. Chrono-Potentiometry technique.....	83
7.5.1. Description.....	83
7.5.1. Technique parameters.....	83
7.5.2. Data format.....	84
7.5.3. Data conversion.....	84
7.6. Chrono-Amperometry technique.....	85
7.6.1. Description.....	85
7.6.2. Technique parameters.....	85
7.6.3. Data format.....	86
7.6.4. Data conversion.....	86
7.7. Voltage Scan technique.....	87
7.7.1. Description.....	87
7.7.2. Technique parameters.....	87
7.7.3. Data format.....	88
7.7.4. Data conversion.....	88
7.8. Current Scan technique.....	89
7.8.1. Description.....	89
7.8.2. Technique parameters.....	89
7.8.3. Data format.....	90
7.8.4. Data conversion.....	90
7.9. Constant Power technique.....	91
7.9.1. Description.....	91
7.9.2. Technique parameters.....	91
7.9.3. Data format.....	92
7.9.4. Data conversion.....	92
7.10. Constant Load technique.....	93
7.10.1. Description.....	93
7.10.2. Technique parameters.....	93
7.10.3. Data format.....	94
7.10.4. Data conversion.....	94

7.11. Potentio Electrochemical Impedance Spectroscopy technique.....	95
7.11.1. Description.....	95
7.11.2. Technique parameters.....	95
7.11.3. Data format.....	96
7.11.4. Data conversion.....	97
7.12. Staircase Potentio Electrochemical Impedance Spectroscopy technique.....	99
7.12.1. Description.....	99
7.12.2. Technique parameters.....	99
7.12.3. Data format.....	100
7.12.4. Data conversion.....	102
7.13. Galvano Electrochemical Impedance Spectroscopy technique.....	103
7.13.1. Description.....	103
7.13.2. Technique parameters.....	103
7.13.3. Data format.....	104
7.13.4. Data conversion.....	104
7.14. Staircase Galvano Electrochemical Impedance Spectroscopy technique.....	105
7.14.1. Description.....	105
7.14.2. Technique parameters.....	105
7.14.3. Data format.....	106
7.14.4. Data conversion.....	106
7.15. Differential Pulse Voltammetry technique.....	107
7.15.1. Description.....	107
7.15.2. Technique parameters.....	107
7.15.3. Data format.....	108
7.15.4. Data conversion.....	108
7.16. Square Wave Voltammetry technique.....	109
7.16.1. Description.....	109
7.16.2. Technique parameters.....	109
7.16.3. Data format.....	110
7.16.4. Data conversion.....	110
7.17. Normal Pulse Voltammetry technique.....	112
7.17.1. Description.....	112
7.17.2. Technique parameters.....	112
7.17.3. Data format.....	113
7.17.4. Data conversion.....	113
7.18. Reverse Normal Pulse Voltammetry technique.....	115
7.18.1. Description.....	115

7.18.2. Technique parameters.....	115
7.18.3. Data format.....	116
7.18.4. Data conversion.....	116
7.19. Differential Normal Pulse Voltammetry technique.....	117
7.19.1. Description.....	117
7.19.2. Technique parameters.....	117
7.19.3. Data format.....	118
7.19.4. Data conversion.....	118
7.20. Differential Pulse Amperometry technique.....	120
7.20.1. Description.....	120
7.20.2. Technique parameters.....	120
7.20.3. Data format.....	121
7.20.4. Data conversion.....	121
7.21. Ecorr. Vs Time technique.....	122
7.21.1. Description.....	122
7.21.2. Technique parameters.....	122
7.21.3. Data format.....	122
7.21.4. Data conversion.....	123
7.22. Linear Polarization technique.....	124
7.22.1. Description.....	124
7.22.2. Technique parameters.....	124
7.22.3. Data format.....	125
7.22.4. Data conversion.....	126
7.23. Generalized Corrosion technique.....	127
7.23.1. Description.....	127
7.23.2. Technique parameters.....	127
7.23.3. Data format.....	128
7.23.4. Data conversion.....	129
7.24. Cyclic PotentioDynamic Polarization technique.....	130
7.24.1. Description.....	130
7.24.2. Technique parameters.....	130
7.24.3. Data format.....	131
7.24.4. Data conversion.....	132
7.25. PotentioDynamic Pitting technique.....	133
7.25.1. Description.....	133
7.25.2. Technique parameters.....	133
7.25.3. Data format.....	134

7.25.4. Data conversion.....	135
7.26. PotentioStatic Pitting technique.....	136
7.26.1. Description.....	136
7.26.2. Technique parameters.....	136
7.26.3. Data format.....	137
7.26.4. Data conversion.....	137
7.27. Zero Resistance Ammeter technique.....	138
7.27.1. Description.....	138
7.27.2. Technique parameters.....	138
7.27.3. Data format.....	139
7.27.4. Data conversion.....	140
7.28. Manual IR technique.....	141
7.28.1. Description.....	141
7.28.2. Technique parameters.....	141
7.28.3. Data format.....	141
7.28.4. Data conversion.....	141
7.29. IR Determination with PotentioStatic Impedance technique.....	142
7.29.1. Description.....	142
7.29.2. Technique parameters.....	142
7.29.3. Data format.....	143
7.29.4. Data conversion.....	144
7.30. IR Determination with GalvanoStatic Impedance technique.....	145
7.30.1. Description.....	145
7.30.2. Technique parameters.....	145
7.30.3. Data format.....	146
7.30.4. Data conversion.....	147
7.31. Loop technique.....	149
7.31.1. Description.....	149
7.31.2. Technique parameters.....	149
7.31.3. Data format.....	149
7.31.4. Data conversion.....	149
7.32. Trigger Out technique.....	150
7.32.1. Description.....	150
7.32.2. Technique parameters.....	150
7.32.3. Data format.....	150
7.32.4. Data conversion.....	150
7.33. Trigger In technique.....	151

7.33.1. Description.....	151
7.33.2. Technique parameters.....	151
7.33.3. Data format.....	151
7.33.4. Data conversion.....	151
7.34. Trigger Out Set technique.....	152
7.34.1. Description.....	152
7.34.2. Technique parameters.....	152
7.34.3. Data format.....	152
7.34.4. Data conversion.....	152
7.35. Large Amplitude Sinusoidal Voltammetry technique.....	153
7.35.1. Description.....	153
7.35.2. Technique parameters.....	153
7.35.3. Data format.....	154
7.35.4. Data conversion.....	154
7.36. Chrono-Potentiometry technique with limits.....	155
7.36.1. Description.....	155
7.36.1. Technique parameters.....	155
7.36.2. Data format.....	157
7.36.3. Data conversion.....	157
7.37. Chrono-Amperometry technique with limits.....	158
7.37.1. Description.....	158
7.37.2. Technique parameters.....	158
7.37.3. Data format.....	160
7.37.4. Data conversion.....	160
7.38. Voltage Scan technique with limits.....	161
7.38.1. Description.....	161
7.38.2. Technique parameters.....	161
7.38.3. Data format.....	163
7.38.4. Data conversion.....	163
7.38.5. Delphi Example.....	163
7.39. Current Scan technique with limits.....	165
7.39.1. Description.....	165
7.39.2. Technique parameters.....	165
7.39.3. Data format.....	167
7.39.4. Data conversion.....	167
7.40. Modular Pulse technique.....	168
7.40.1. Description.....	168

7.40.2. Technique parameters.....	168
7.40.3. Data format.....	169
7.40.4. Data conversion.....	169
7.41. Constant Amplitude Sinusoidal micro Galvano polarization technique.....	170
7.41.1. Description.....	170
7.41.2. Technique parameters.....	170
7.41.3. Data format.....	171
7.41.4. Data conversion.....	171
7.42. Constant Amplitude Sinusoidal micro Potentio polarization technique.....	172
7.42.1. Description.....	172
7.42.2. Technique parameters.....	172
7.42.3. Data format.....	173
7.42.4. Data conversion.....	173
8. Global parameters for hardware configuration.....	174
8.1 Electrode connection.....	174
8.2 Instrument ground.....	174
8.3 Record and external control options.....	174
APPENDIX A. Find instruments.....	177
1. Calling conventions.....	177
2. Multi-thread applications.....	177
3. Data types.....	177
4. Functions reference.....	177
5. Serializations format.....	181
6. Error codes.....	183
APPENDIX B. Version Compatibility.....	184

This document is a part of the BioLogic OEM Package and is protected by the terms of the OEM Package **licence** as well as other intellectual property rights owned by BioLogic SAS. This document may only be used for non-commercial purposes such as for the integration of BioLogic equipment to larger technical solutions manufactured and/or delivered to end-users.

Overview

The **EC-Lab[®] Development Package** is intended for software developers who need to integrate Bio-Logic potentiostats / galvanostats in OEM applications. This package supports the following Biologic instruments:

VMP-3 series	VMP-300 series
• SP-50 • SP-50e • SP-150 • SP-150e • VSP • VSP-3e • VMP2 • VMP3 • VMP-3e • BiStat • HCP-803 • HCP-1005 • MPG2	• SP-200 • SP-240 • SP-300 • BP-300 • VSP-300 • VMP-300

The library accommodates some functionality offered by EC-Lab[®] software:

- Detection of connected instruments,
- Connection / disconnection to the instrument through Ethernet/USB,
- Channels initialization (firmware loading),
- Load techniques on selected channel(s) (OCV, CA, CP...),
- Start/stop selected channel(s),
- Retrieving data...

LabVIEW[®] VIs examples are also provided to help the user in the integration of the library in his application. Other examples are provided, in C, C++, C#, Delphi and Python, nevertheless they may not be updated to the latest features.

Notes:

- Theoretically, any software development tool able to call a DLL is suitable for using the **EC-Lab[®] Development Package** (C++, Pascal, LabVIEW[®]...). However limitations of some compilers may prevent a proper calling of some or all the functions.
- The **EC-Lab[®] Development Package** is provided "AS IS". No specific support is provided to this free package. There is no warranty for damages incurred using it. We strongly recommend you to test your techniques on a dummy cell before using them on a real cell.
- Use of the **EC-Lab[®] Development Package** in a commercial software is forbidden without a written authorization of Bio-Logic SAS.

- The software includes a LEPMI/ENSEEG/INPG license.

2. Files package

The **EC-Lab[®] Development Package** is composed with the following files:

blfind.dll:	library used to find available instruments (Ethernet, USB)
blfind64.dll:	same as blfind.dll provided for 64-bits compatibility
Eclib.dll:	library used to communicate with the instrument
Eclib64.dll:	same as Eclib.dll provided for 64-bits compatibility
kernel.bin:	channel firmware for VMP-3 devices
kernel4.bin:	channel firmware for VMP-300 series
Kernel5.bin:	channel firmware for VMP-300 series P boards.
Vmp_ii_0437_a4.xlx:	channel firmware for VMP-3 L devices
Vmp_ii_0437_a6.xlx:	channel firmware for VMP-3 devices
Vmp_iv_0395_aa.xlx:	channel firmware for VMP-300 series
ocv.ecc:	open circuit voltage technique
cv.ecc:	cyclic voltammetry technique
biovscan.ecc:	cyclic voltammetry advanced technique
ca.ecc:	chrono-amperometry technique
cp.ecc:	chrono-potentiometry technique
iscan.ecc:	current scan technique
vscan.ecc:	voltage scan technique
lasv.ecc:	large amplitude sinusoidal voltammetry
load.ecc:	constant load technique
pow.ecc:	constant power technique
peis.ecc:	potentio electrochemical impedance spectroscopy technique
geis.ecc:	galvano electrochemical impedance spectroscopy technique
seisp.ecc:	staircase potentiostatic impedance technique
seisg.ecc:	staircase galvanostatic Impedance technique
dpv.ecc:	differential pulse voltammetry technique
swv.ecc:	square wave voltammetry technique
npv.ecc:	normal pulse voltammetry technique
rnpv.ecc:	reverse normal pulse voltammetry technique
dnpv.ecc:	differential normal pulse voltammetry technique

dpa.ecc:	differential pulse amperometry technique
evt.ecc:	ecorr. vs time technique
lp.ecc:	linear polarization technique
gc.ecc:	generalized corrosion technique
cpp.ecc:	cyclic potentiodynamic polarization technique
pdp.ecc:	potentiodynamic pitting technique
psp.ecc:	potentiostatic pitting technique
zra.ecc:	zero resistance ammeter technique
ircmp.ecc:	manual IR technique
pzir.ecc:	IR determination with potentiostatic impedance technique
gzir.ecc:	IR determination with galvanostatic impedance technique
loop.ecc:	loop technique
TO.ecc:	trigger out technique
TI.ecc:	trigger in technique
TOS.ecc:	trigger out set technique
vscanlimit.ecc:	voltage scan technique with limits
iscanlimit.ecc:	current scan technique with limits
mp.ecc:	modular pulse technique
casp.ecc:	constant amplitude sinusoidal micro potentio polarization technique
casg.ecc:	constant amplitude sinusoidal micro galvano polarization technique
calimit.ecc:	chrono-amperometry technique with limits
cplimit.ecc:	chrono-potentiometry technique with limits

These techniques are doubled with the name *name4.ecc* only for VMP-300 series and *name5.ecc* for VMP-300 series P boards. The techniques which have not these file must not to be used with VMP-300 series (e.g. do not load a file *name.ecc* on VMP-300 series and do not load a file *name4.ecc* on VMP-3 based devices).

The *biovsan*, *load* and *pow* techniques are only supported for VMP-3 based devices.

All the files of **EC-Lab[®] Development Package** must be stored in the same directory.

3. General information

3.1. Calling conventions

The library uses the **stdcall** calling conventions for all exported functions.

3.2. Data alignment

All the structures used by the library are aligned on double-word to simplify the communication with other programming environments.

3.3. Multi-thread applications

All exported functions are protected by a synchronization object, they can be called in a multi-thread application.

3.4. Data types

The library is written in object Pascal under Delphi. All data type used in this document are Pascal types. The type translation table C/C++ - Object Pascal is :

Type translation table

C/C++ Type	ObjectPascal Type
unsigned short [int]	Word
[signed] short [int]	SmallInt
unsigned [int]	Cardinal { 3.25 fix }
[signed] int	Integer
UINT	LongInt { or Cardinal }
WORD	Word
DWORD	LongInt { or Cardinal }
unsigned long	LongInt { or Cardinal }
unsigned long int	LongInt { or Cardinal }
[signed] long	LongInt
[signed] long int	LongInt
char	Char
signed char	ShortInt

Type translation table

C/C++ Type	ObjectPascal Type
unsigned char	Byte
char*	PChar
LPSTR or PSTR	PChar
LPWSTR or PWSTR	PWideChar { 3.12 fix }
void*	Pointer
BOOL	Bool
float	Single
double	Double
long double	Extended
UserType*	^UserType
NULL	NIL

The following data types are used by the library:

Data types

Data types	Format	Range
int8	signed 8-bit	-128..127
int16	signed 16-bit	-32768..32767
int32	signed 32-bit	-2147483648..2147483647
uint8	unsigned 8-bit	0..255
uint16	unsigned 16-bit	0..65535
uint32	unsigned 32-bit	0..4294967295
boolean	unsigned 8-bit	FALSE=0, TRUE=1
single	Single precision floating point (32 bits, 7–8 significant digits)	[1.5×10^{-45} , 3.4×10^{38}]
double	Double precision floating point (64 bits, 15–16 significant digits)	[5.0×10^{-324} , 1.7×10^{308}]

3.5. Variables description

The following variables are used by the library:

Variable description		
Variable	Description	Unit
t	time	second (s)
I	Current	Ampere (A)
Ic	Current control	Ampere (A)
<I>	Average current	Ampere (A)
Ewe	WE potential versus REF	Volt (V)
<Ewe>	Average of WE potential versus REF	Volt (V)
Ece	CE potential versus REF	Volt (V)
Ewe-Ece	WE versus CE potential	Volt (V)
Ec	Potential control	Volt (V)
R	Resistor	Ohm (Ω)
power	Power	Watt (W)
cycle	Cycle number	-
Q	Electric charge from the beginning of the technique	Ampere.second (A.s)
f	Frequency	Hertz (Hz)
phase	Angle	radian (rad)
Ewe	Module of Ewe (V)	Volt (V)
Ece	Module of Ece (V)	Volt (V)
Ice	Module of Ice (A)	Ampere (A)
I	Module of I	Ampere (A)
I Range	Current range	-
E Range	WE potential range	-
tb	Timebase	Microsecond (μ s)

4. Using the library

First of all, the function `BL_Connect` must be called to establish the connection with the selected instrument through Ethernet or USB. The function will return a device identifier (ID) which will have to be used with all the functions of the library to communicate with this instrument. Note that one can communicate with several instruments thanks to the device identifier.

After establishing the connection, the firmware must be loaded (if not already done) on channels plugged with the function `BL_LoadFirmware` to make them operational. Use the function `BL_IsChannelPlugged` to list channels plugged on the instrument, and the function `BL_GetChannelBoardType` can be used to identify the channel before loading the firmware. Once the firmware is loaded, the function `BL_GetChannelInfos` can return channel information such as firmware version, board version, memory size, ...

Now the instrument is ready to receive techniques with user's parameters on selected channels thanks to the function `BL_LoadTechnique`. Note that the techniques are defined by the *.ecc files delivered with the library (for instance the file `ocv.ecc` defines the *Open Circuit Voltage* technique). See the section 7. Techniques for a complete description of parameters available for each technique. Electrochemical techniques parameters must be carefully programmed according to the instrument hardware specifications. Be aware that wrong parameters can generate faulty operations of the technique.

Once the techniques are loaded, channels can be started (or stopped) with the function `BL_StartChannel` (or `BL_StopChannel`). One can also synchronize channels together with the functions `BL_StartChannels` / `BL_StopChannels`.

The data can be recovered from selected channels with the function `BL_GetData`. The format of the returned data depends on the technique used to generate these data. One can find the identifier of this technique in the structure `TDataInfos` returned by the function `BL_GetData`. See the section 7. Techniques for a complete description of the data format for each technique.

Once the techniques are finished and all data recovered, one must close the connection with the instrument with the function `BL_Disconnect`.

5. Structures and Constants

5.1. Structures

The structure `TDeviceInfos` defines device information and is used by the function `BL_Connect` :

Structure **TDeviceInfos**

Field name	Data type	Description
DeviceCode	int32	Device code (see section 5.3. <i>Constants</i>)
RAMsize	int32	RAM size, in MBytes
CPU	int32	Computer board cpu
NumberOfChannels	int32	Number of channels connected
NumberOfSlots	int32	Number of slots available
FirmwareVersion	int32	Communication firmware version
FirmwareDate_YYYY	int32	Communication firmware date YYYY
FirmwareDate_MM	int32	Communication firmware date MM
FirmwareDate_DD	int32	Communication firmware date DD
HTdisplayOn	int32	Allow hyper-terminal prints (true/false)
NbOfConnectedPC	int32	Number of connected PC

The structure `TChannelInfos` defines channel information and is used by the function `BL_GetChannelInfos` :

Structure **TChannelInfos**

Field name	Data type	Description
Channel	int32	Channel (0..15)
BoardVersion	int32	Board version
BoardSerialNumber	int32	Board serial number
FirmwareCode	int32	Identifier of the firmware loaded on the channel (see section 5.3. <i>Constants</i>)
FirmwareVersion	int32	Firmware version
XilinxVersion	int32	Xilinx version
AmpCode	int32	Amplifier code (see section 5.3. <i>Constants</i>)
NbAmps	int32	Number of amplifiers (0..16)

Structure TChannelInfos

Field name	Data type	Description
Lcboard	int32	Low current board present (= 1)
Zboard	int32	TRUE if channel with impedance capabilities
RESERVED	int32	not used
RESERVED	int32	not used
MemSize	int32	Memory size (in bytes)
MemFilled	int32	Memory filled (in bytes)
State	int32	Channel state : run/stop/pause (see section 5.3. <i>Constants</i>)
MaxIRange	int32	Maximum I range allowed (see section 5.3. <i>Constants</i>)
MinIRange	int32	Minimum I range allowed (see section 5.3. <i>Constants</i>)
MaxBandwidth	int32	Maximum bandwidth allowed (see section 5.3. <i>Constants</i>)
NbOfTechniques	int32	Number of techniques loaded

The structure TCurrentValues defines channel current values (Ewe, Ece, I, ...) and is used by functions BL_GetCurrentValues and BL_GetData :

Structure TCurrentValues

Field name	Data type	Description
State	int32	Channel state : run/stop/pause (see section 5.3. <i>Constants</i>)
MemFilled	int32	Memory filled (in Bytes)
TimeBase	single	Time base (s)
Ewe	single	Working electrode potential (V)
EweRangeMin	single	Ewe min range (V)
EweRangeMax	single	Ewe max range (V)
Ece	single	Counter electrode potential (V)
EceRangeMin	single	Ece min range (V)
EceRangeMax	single	Ece max range (V)
Eoverflow	int32	Potential overflow
I	single	Current value (A)
IRange	int32	Current range (see section 5.3. <i>Constants</i>)
Ioverflow	int32	Current overflow

Structure TCurrentValues

Field name	Data type	Description
ElapsedTime	single	Elapsed time (s)
Freq	single	Frequency (Hz)
Rcomp	single	R compensation (Ohm)
Saturation	int32	E or/and I saturation
OptErr	int32	Hardware Option Error Code (see section 5.4. <i>Error codes</i>)
OptPos	int32	Index of the option generating the OptErr (VMP-300 series only, otherwise 0)

The structure TDataInfos defines data information (i.e. information on the data saved in the data buffer) and is used by the function BL_GetData :

Structure TDataInfos

Field name	Data type	Description
IRQskipped	int32	Number of IRQ skipped
NbRows	int32	Number of rows in the data buffer, i.e. number of points saved in the data buffer
NbCols	int32	Number of columns in the data buffer, i.e. number of variables defining a point in the data buffer
TechniqueIndex	int32	Index (0-based) of the technique which has generated the data. This field is only useful for linked techniques
TechniqueID	int32	Identifier of the technique which has generated the data. Must be used to identify the data format in the data buffer (see section 5.3. <i>Constants</i>)
ProcessIndex	Int32	Index (0-based) of the process of the technique which has generated the data. Must be used to identify the data format in the data buffer
loop	int32	Loop number
StartTime	double	Start time (s)
MuxPad	int32	(no longer used)

The array TDataBuffer is used to retrieve data from the device by the function BL_GetData:

Type TDataBuffer

```
TDataBuffer = array[1..1000] of uint32;
PDataBuffer = ^TDataBuffer
```

The structure TEccParam defines an elementary technique parameter and is used by the function BL_LoadTechnique:

Structure TEccParam

Field name	Data type	Description
ParamStr	array[1..64] of char	string defining the parameter label (see section 7. Techniques for a complete description of parameters available for each technique)
ParamType	int32	Parameter type (0=int32, 1=boolean, 2=single)
ParamVal	int32	Parameter value (WARNING: numerical value)
ParamIndex	int32	Parameter index (0-based). Useful for multi-step parameters only.

Type PEccParam

```
PEccParam = ^TEccParam
```

The structure TEccParams defines an array of elementary technique parameters and is used by the function BL_LoadTechnique:

Structure TEccParams : array of elementary technique parameters

Field name	Data type	Description
len	int32	Length of the array pointed to by pParams
pParams	PEccParam	Pointer on the array of technique parameters (array of structure TEccParam)

Structure THardwareConf : hardware configuration

Field name	Data type	Description
Conn	int32	Electrode connection for constant value of this parameter see 5. Structures and Constants
Ground	int32	Instrument ground

Structure THardwareConf : hardware configuration

Field name	Data type	Description
		for constant value of this parameter see 5. Structures and Constants

Type PHardwareConf

PHardwareConf = ^THardwareConf

5.2. Other structures

5.2.1. Labview Structures

The structures below are defined for LabVIEW compatibility for technique parameter loading and are used by the function BL_LoadTechnique_LV.

Structure **TArrayOfChar_LV** : array of char

Field name	Data type	Description
dimSize	int32	Length of the array
FirstChar	char	First element in the array of char

Type **PArrayOfChar_LV**

```
PArrayOfChar_LV = ^TArrayOfChar_LV;
PPArrayOfChar_LV = ^PArrayOfChar_LV;
```

Structure **TEccParam_LV** : elementary technique parameter

Field name	Data type	Description
ParamStr	PPArrayOfChar_LV	string defining the parameter (see section 7. Techniques for a complete description of parameters available for each technique)
ParamType	int32	Parameter type (0=int32, 1=boolean, 2=single)
ParamVal	int32	Parameter value (WARNING : numerical value)
ParamIndex	int32	Parameter index (0-based). Useful for multi-step parameters only.

Structure **TEccParams_LV** : array of elementary technique parameters

Field name	Data type	Description
dimSize	int32	Length of the array
FirstEccParam_LV	TEccParam_LV	First element in the array

Type **PPEccParams_LV**

```
PEccParams_LV = ^TEccParams_LV
PPEccParams_LV = ^PEccParams_LV
```

5.3. Constants

Device constants (used by the function BL_Connect)

Constant	Value	Description
KBIO_DEV_VMP	0	VMP device
KBIO_DEV_VMP2	1	VMP2 device
KBIO_DEV_MPG	2	MPG device
KBIO_DEV_BISTAT	3	BISTAT device
KBIO_DEV_MCS_200	4	MCS-200 device
KBIO_DEV_VMP3	5	VMP3 device
KBIO_DEV_VSP	6	VSP device
KBIO_DEV_HCP803	7	HCP-803 device
KBIO_DEV_EPP400	8	EPP-400 device
KBIO_DEV_EPP4000	9	EPP-4000 device
KBIO_DEV_BISTAT2	10	BISTAT 2 device
KBIO_DEV_FCT150S	11	FCT-150S device
KBIO_DEV_VMP300	12	VMP-300 device
KBIO_DEV_SP50	13	SP-50 device
KBIO_DEV_SP150	14	SP-150 device
KBIO_DEV_FCT50S	15	FCT-50S device
KBIO_DEV_SP300	16	SP300 device
KBIO_DEV_CLB500	17	CLB-500 device
KBIO_DEV_HCP1005	18	HCP-1005 device
KBIO_DEV_CLB2000	19	CLB-2000 device
KBIO_DEV_VSP300	20	VSP-300 device
KBIO_DEV_SP200	21	SP-200 device
KBIO_DEV_MPG2	22	MPG2 device
KBIO_DEV_ND1	23	RESERVED
KBIO_DEV_ND2	24	RESERVED
KBIO_DEV_ND3	25	RESERVED
KBIO_DEV_ND4	26	RESERVED

Device constants (used by the function BL_Connect)

Constant	Value	Description
KBIO_DEV_SP240	27	SP-240 device
KBIO_DEV_MPG205	28	MPG-205 (VMP3)
KBIO_DEV_MPG210	29	MPG-210 (VMP3)
KBIO_DEV_MPG220	30	MPG-220 (VMP3)
KBIO_DEV_MPG240	31	MPG-240 (VMP3)
KBIO_DEV_BP300	32	BP-300 (VMP300)
KBIO_DEV_VMP3e	33	VMP-3e (VMP3)
KBIO_DEV_VSP3e	34	VSP-3e (VMP3)
KBIO_DEV_SP50E	35	SP-50e (VMP3)
KBIO_DEV_SP150E	36	SP-150e (VMP3)
KBIO_DEV_UNKNOWN	255	Unknown device

Firmware code constants (used by the structure TChannelInfos)

Constant	Value	Description
KBIO_FIRM_NONE	0	No firmware loaded
KBIO_FIRM_INTERPR	1	Firmware for EC-Lab [®] software
KBIO_FIRM_UNKNOWN	4	Unknown firmware loaded
KBIO_FIRM_KERNEL	5	Firmware for the library
KBIO_FIRM_INVALID	8	Invalid firmware loaded
KBIO_FIRM_ECAL	10	Firmware for calibration software

Amplifier constants (used by the structure TChannelInfos)

Constant	Value	Description	Device Family
KBIO_AMPL_NONE	0	No amplifier	VMP3 series
KBIO_AMPL_2A	1	Amplifier 2 A	VMP3 series
KBIO_AMPL_1A	2	Amplifier 1 A	VMP3 series
KBIO_AMPL_5A	3	Amplifier 5 A	VMP3 series
KBIO_AMPL_10A	4	Amplifier 10 A	VMP3 series
KBIO_AMPL_20A	5	Amplifier 20 A	VMP3 series
KBIO_AMPL_HEUS	6	reserved	VMP3 series
KBIO_AMPL_LC	7	Low current amplifier	VMP3 series
KBIO_AMPL_80A	8	Amplifier 80 A	VMP3 series
KBIO_AMPL_4AI	9	Amplifier 4 A	VMP3 series

Amplifier constants (used by the structure TChannelInfos)

Constant	Value	Description	Device Family
KBIO_AMPL_PAC	10	Fuel Cell Tester	VMP3 series
KBIO_AMPL_4AI_VSP	11	Amplifier 4 A (VSP instrument)	VMP3 series
KBIO_AMPL_LC_VSP	12	Low current amplifier (VSP instrument)	VMP3 series
KBIO_AMPL_UNDEF	13	Undefined amplifier	VMP3 series
KBIO_AMPL_MUIC	14	reserved	VMP3 series
KBIO_AMPL_ERROR	15	No amplifier in case of error	VMP3 series
KBIO_AMPL_8AI	16	Amplifier 8 A	VMP3 series
KBIO_AMPL_LB500	17	Amplifier LB500	VMP3 series
KBIO_AMPL_100A5V	18	Amplifier 100 A	VMP3 series
KBIO_AMPL_LB2000	19	Amplifier LB2000	VMP3 series
KBIO_AMPL_1A48V	20	Amplifier 1A 48V	VMP-300 series
KBIO_AMPL_4A14V	21	Amplifier 4A 14V	VMP-300 series
KBIO_AMPL_5A_MPG2B	22	Amplifier 5A	MPG-205
KBIO_AMPL_10A_MPG2B	23	Amplifier 10A	MPG-210
KBIO_AMPL_20A_MPG2B	24	Amplifier 20A	MPG-220
KBIO_AMPL_40A_MPG2B	25	Amplifier 40A	MPG-240
KBIO_AMPL_COIN_CELL_HOLDER	26	Coin cell holder amplifier	
KBIO_AMPL4_10A5V	27	Amplifier 10A 5V	VMP-300 series
KBIO_AMPL4_2A30V	28	Amplifier 2A 30V	VMP-300 series
KBIO_AMPL4_1A48VP	129	Amplifier 1A 48VP	VMP-300 series

I range constants (used by the structures TDataInfos and TEccParam)

Constant	Value	Description	Device Family
KBIO_IRANGE_KEEP	-1	Keep previous I range	VMP-300 series
KBIO_IRANGE_100pA	0	I range 100 pA	VMP-300 series
KBIO_IRANGE_1nA	1	I range 1 nA	VMP3 /VMP-300 series
KBIO_IRANGE_10nA	2	I range 10 nA	VMP3 /VMP-300 series
KBIO_IRANGE_100nA	3	I range 100 nA	VMP3 /VMP-300 series
KBIO_IRANGE_1uA	4	I range 1 μ A	VMP3 /VMP-300 series
KBIO_IRANGE_10uA	5	I range 10 μ A	VMP3 /VMP-300 series
KBIO_IRANGE_100uA	6	I range 100 μ A	VMP3 /VMP-300 series
KBIO_IRANGE_1mA	7	I range 1 mA	VMP3 /VMP-300 series
KBIO_IRANGE_10mA	8	I range 10 mA	VMP3 /VMP-300 series
KBIO_IRANGE_100mA	9	I range 100 mA	VMP3 /VMP-300 series
KBIO_IRANGE_1A	10	I range 1 A	VMP3 /VMP-300 series
KBIO_IRANGE_BOOSTER	11	Booster	VMP3 /VMP-300 series

I range constants (used by the structures TDataInfos and TEccParam)

Constant	Value	Description	Device Family
KBIO_IRANGE_AUTO	12	Auto range	VMP3 /VMP-300 series
KBIO_IRANGE_10pA		IRANGE_100pA + Igain x10	VMP-300 series
KBIO_IRANGE_1pA		IRANGE_100pA + Igain x100	VMP-300 series

E range constants (used by the structures TDataInfos and TEccParam)

Constant	Value	Description
KBIO_ERANGE_2_5	0	±2.5 V
KBIO_ERANGE_5	1	±5 V
KBIO_ERANGE_10	2	±10 V
KBIO_ERANGE_AUTO	3	Auto range

Bandwidth constants (used by the structures TDataInfos and TEccParam)

Constant	Value	Description
KBIO_BW_KEEP	-1	Keep previous bandwidth
KBIO_BW_1	1	Bandwidth #1
KBIO_BW_2	2	Bandwidth #2
KBIO_BW_3	3	Bandwidth #3
KBIO_BW_4	4	Bandwidth #4
KBIO_BW_5	5	Bandwidth #5
KBIO_BW_6	6	Bandwidth #6
KBIO_BW_7	7	Bandwidth #7
KBIO_BW_8	8	Bandwidth #8 (only with VMP-300 series)
KBIO_BW_9	9	Bandwidth #9 (only with VMP-300 series)

Filter Constants (used by the structures TDataInfos and TEccParam)

Constant	Value	Description
KBIO_FILTER_RSRVD	-1	Reserved value, should not be used
KBIO_FILTER_NONE	0	Full band
KBIO_FILTER_50KHZ	1	50 kHz filter
KBIO_FILTER_1KHZ	2	1 kHz filter
KBIO_FILTER_5HZ	3	5 Hz filter

Electrode connection constants (used by the structure THardwareConf)

Constant	Value	Description
KBIO_CONN_STD	0	Standard connection
KBIO_CONN_CETOGRND	1	CE to ground connection
KBIO_CONN_WETOGRND	2	WE to ground connection
KBIO_CONN_HV	3	High Voltage connection (0-50V measure)

Channel mode constants (used only with VMP-300 series, by the structure THardwareConf)

Constant	Value	Description
KBIO_MODE_GROUNDED	0	Grounded mode
KBIO_MODE_FLOATING	1	Floating mode

Technique identifier constants (used by the structure TDataInfos)

Constant	Value	Description
KBIO_TECHID_NONE	0	None
KBIO_TECHID_OCV	100	Open Circuit Voltage (Rest) identifier
KBIO_TECHID_CA	101	Chrono-amperometry identifier
KBIO_TECHID_CP	102	Chrono-potentiometry identifier
KBIO_TECHID_CV	103	Cyclic Voltammetry identifier
KBIO_TECHID_PEIS	104	Potential Electrochemical Impedance Spectroscopy identifier
KBIO_TECHID_POTPULSE	105	(unused)
KBIO_TECHID_GALPULSE	106	(unused)
KBIO_TECHID_GEIS	107	Galvano Electrochemical Impedance Spectroscopy identifier
KBIO_TECHID_STACKPEIS_SLAVE	108	Potential Electrochemical Impedance Spectroscopy on stack identifier
KBIO_TECHID_STACKPEIS	109	Potential Electrochemical Impedance Spectroscopy on stack identifier
KBIO_TECHID_CPOWER	110	Constant Power identifier
KBIO_TECHID_CLOAD	111	Constant Load identifier
KBIO_TECHID_FCT	112	(unused)
KBIO_TECHID_SPEIS	113	Staircase Potential Electrochemical Impedance Spectroscopy identifier
KBIO_TECHID_SGEIS	114	Staircase Galvano Electrochemical Impedance Spectroscopy identifier
KBIO_TECHID_STACKPDYN	115	Potential dynamic on stack identifier
KBIO_TECHID_STACKPDYN_SLAVE	116	Potential dynamic on stack identifier

Technique identifier constants (used by the structure TDataInfos)

Constant	Value	Description
KBIO_TECHID_STACKGDYN	117	Galvano dynamic on stack identifier
KBIO_TECHID_STACKGEIS_SLAVE	118	Galvano Electrochemical Impedance Spectroscopy on stack identifier
KBIO_TECHID_STACKGEIS	119	Galvano Electrochemical Impedance Spectroscopy on stack identifier
KBIO_TECHID_STACKGDYN_SLAVE	120	Galvano dynamic on stack identifier
KBIO_TECHID_CPO	121	(unused)
KBIO_TECHID_CGA	122	(unused)
KBIO_TECHID_COKINE	123	(unused)
KBIO_TECHID_PDYN	124	Potentio dynamic identifier
KBIO_TECHID_GDYN	125	Galvano dynamic identifier
KBIO_TECHID_CVA	126	Cyclic Voltammetry Advanced identifier
KBIO_TECHID_DPV	127	Differential Pulse Voltammetry identifier
KBIO_TECHID_SWV	128	Square Wave Voltammetry identifier
KBIO_TECHID_NPV	129	Normal Pulse Voltammetry identifier
KBIO_TECHID_RNPV	130	Reverse Normal Pulse Voltammetry identifier
KBIO_TECHID_DNPV	131	Differential Normal Pulse Voltammetry identifier
KBIO_TECHID_DPA	132	Differential Pulse Amperometry identifier
KBIO_TECHID_EVT	133	Ecorr vs. time identifier
KBIO_TECHID_LP	134	Linear Polarization identifier
KBIO_TECHID_GC	135	Generalized corrosion identifier
KBIO_TECHID_CPP	136	Cyclic Potentiodynamic Polarization identifier
KBIO_TECHID_PDP	137	Potentiodynamic Pitting identifier
KBIO_TECHID_PSP	138	Potentiostatic Pitting identifier
KBIO_TECHID_ZRA	139	Zero Resistance Ammeter identifier
KBIO_TECHID_MIR	140	Manual IR identifier
KBIO_TECHID_PZIR	141	IR Determination with Potentiostatic Impedance identifier
KBIO_TECHID_GZIR	142	IR Determination with Galvanostatic Impedance identifier
KBIO_TECHID_LOOP	150	Loop (used for linked techniques) identifier
KBIO_TECHID_TO	151	Trigger Out identifier
KBIO_TECHID_TI	152	Trigger In identifier
KBIO_TECHID_TOS	153	Trigger Set identifier
KBIO_TECHID_CPLIMIT	155	Chrono-potentiometry with limits identifier
KBIO_TECHID_GDYNLIMIT	156	Galvano dynamic with limits identifier

Technique identifier constants (used by the structure TDataInfos)

Constant	Value	Description
KBIO_TECHID_CALIMIT	157	Chrono-amperometry with limits identifier
KBIO_TECHID_PDYNLIMIT	158	Potentio dynamic with limits identifier
KBIO_TECHID_LASV	159	Large amplitude sinusoidal voltammetry
KBIO_TECHID_MP	167	Modular Pulse
KBIO_TECHID_CASG	169	Constant amplitude sinusoidal micro galvanic polarization
KBIO_TECHID_CASP	170	Constant amplitude sinusoidal micro potentio polarization

Channel state constants (used by the structure TChannelInfos)

Constant	Value	Description
KBIO_STATE_STOP	0	Channel is stopped
KBIO_STATE_RUN	1	Channel is running
KBIO_STATE_PAUSE	2	Channel is paused

Parameter type constants (used by the structure TEccParam)

Constant	Value	Description
PARAM_INT32	0	Parameter type = int32
PARAM_BOOLEAN	1	Parameter type = boolean
PARAM_SINGLE	2	Parameter type = single

Channel type constant (returned by BL_GetChannelBoardType)

Constant	Value	Description
BOARD_TYPE_CHANNEL_UNKNOWN	0	The channel board could not be identified or is not supported
BOARD_TYPE_CHANNEL_ESSENTIAL	1	The channel is an VMP-3 series board
BOARD_TYPE_CHANNEL_PREMIUM	2	The channel is a VMP-300 series board
BOARD_TYPE_CHANNEL_PREMIUM_P	3	The channel is a VMP-300 series P board

5.4. Error codes

All functions (with a few exceptions) exported by the library return a signed 32-bit as a result. If the function succeeded the returned value is 0, otherwise the returned value is negative, as described below:

General error codes

Constant	Value	Description
ERR_GEN_NOTCONNECTED	-1	no instrument connected
ERR_GEN_CONNECTIONINPROGRESS	-2	connection in progress
ERR_GEN_CHANNELNOTPLUGGED	-3	selected channel(s) unplugged
ERR_GEN_INVALIDPARAMETERS	-4	invalid function parameters
ERR_GEN_FILENOTEXISTS	-5	selected file does not exist
ERR_GEN_FUNCTIONFAILED	-6	function failed
ERR_GEN_NOCHANNELELECTED	-7	no channel selected
ERR_GEN_INVALIDCONF	-8	invalid instrument configuration
ERR_GEN_ECLAB_LOADED	-9	EC-Lab [®] firmware loaded in the instrument
ERR_GEN_LIBNOTCORRECTLYLOADED	-10	library not correctly loaded in memory
ERR_GEN_USBLIBRARYERROR	-11	USB library not correctly loaded in memory
ERR_GEN_FUNCTIONINPROGRESS	-12	function of the library already in progress
ERR_GEN_CHANNEL_RUNNING	-13	selected channel(s) already used
ERR_GEN_DEVICE_NOTALLOWED	-14	device not allowed
ERR_GEN_UPDATEPARAMETERS	-15	Invalid update function parameters

Instrument error codes

Constant	Value	Description
ERR_INSTR_VMEERROR	-101	internal instrument communication failed
ERR_INSTR_TOOMANYDATA	-102	too many data to transfer from the instrument (device error)
ERR_INSTR_RESPNOTPOSSIBLE	-103	selected channel(s) unplugged (device error)
ERR_INSTR_RESPERROR	-104	Instrument response error
ERR_INSTR_MSGSIZEERROR	-105	Invalid message size

Communication error codes

Constant	Value	Description
ERR_COMM_COMMFAILED	-200	communication failed with the instrument

Communication error codes

Constant	Value	Description
ERR_COMM_CONNECTIONFAILED	-201	cannot establish connection with the instrument
ERR_COMM_WAITINGACK	-202	waiting for the instrument response
ERR_COMM_INVALIDIPADDRESS	-203	invalid IP address
ERR_COMM_ALLOCMEMFAILED	-204	cannot allocate memory in the instrument
ERR_COMM_LOADFIRMWAREFAILED	-205	cannot load firmware on the selected channel(s)
ERR_COMM_INCOMPATIBLESERVER	-206	communication firmware not compatible with the library
ERR_COMM_MAXCONNREACHED	-207	maximum number of allowed connections reached

Firmware error codes

Constant	Value	Description
ERR_FIRM_FIRMFILENOTEXISTS	-300	cannot find kernel.bin file
ERR_FIRM_FIRMFILEACCESSFAILED	-301	cannot read kernel.bin file
ERR_FIRM_FIRMINVALIDFILE	-302	invalid kernel.bin file
ERR_FIRM_FIRMLOADINGFAILED	-303	cannot load kernel.bin on the selected channel(s)
ERR_FIRM_XILFILENOTEXISTS	-304	cannot find FPGA firmware file
ERR_FIRM_XILFILEACCESSFAILED	-305	cannot read FPGA firmware file
ERR_FIRM_XILINVALIDFILE	-306	invalid FPGA firmware file
ERR_FIRM_XILOLOADINGFAILED	-307	cannot load FPGA firmware file on the selected channel(s)
ERR_FIRM_FIRMWARENOTLOADED	-308	no firmware loaded on the selected channel(s)
ERR_FIRM_FIRMWAREINCOMPATIBLE	-309	loaded firmware not compatible with the library

Technique error codes

Constant	Value	Description
ERR_TECH_ECCFILENOTEXISTS	-400	cannot find the selected ECC file
ERR_TECH_INCOMPATIBLEECC	-401	ECC file not compatible with the channel firmware
ERR_TECH_ECCFILECORRUPTED	-402	ECC file is corrupted
ERR_TECH_LOADTECHNIQUEFAILED	-403	cannot load the ECC file
ERR_TECH_DATACORRUPTED	-404	data returned by the instrument are corrupted
ERR_TECH_MEMFULL	-405	cannot load techniques: full memory

VMP-300 series hardware options error codes (TCurrentValues.OptErr)

Constant	Value	Description
ERR_NO_ERROR	0	No error found
ERR_OPT_CHANGE	1	Number of options changed
ERR_OPEN	2	Open-in signal was asserted
ERR_IRCMP_OVR	3	R Compensation overflow
ERR_OPT_4A	100	4A amplifier unknown error
ERR_OPT_4A_OVRTEMP	101	4A amplifier temperature overflow
ERR_OPT_4A_BADPOW	102	4A amplifier bad power
ERR_OPT_4A_POWFAIL	103	4A amplifier power fail
ERR_OPT_48V	200	48V amplifier unknown error
ERR_OPT_48V_OVRTEMP	201	48V amplifier temperature overflow
ERR_OPT_48V_BADPOW	202	48V amplifier bad power
ERR_OPT_48V_POWFAIL	203	48V amplifier power fail
KBIO_OPT_10A5V_ERR	300	10A 5V amplifier error
KBIO_OPT_10A5V_OVRTEMP	301	10A 5V amplifier overheat
KBIO_OPT_10A5V_BADPOW	302	10A 5V amplifier bad power
KBIO_OPT_10A5V_POWFAIL	303	10A 5V amplifier power fail
KBIO_OPT_1A48VP_ERR	600	1A 48VP amplifier error
KBIO_OPT_1A48VP_OVRTEMP	601	1A 48VP amplifier overheat
KBIO_OPT_1A48VP_BADPOW	602	1A 48VP amplifier bad power
KBIO_OPT_1A48VP_POWFAIL	603	1A 48VP amplifier power fail

VMP3 series hardware options error codes (TCurrentValues.OptErr)

Constant	Value	Description
ERR_NO_ERROR	0	No error found
ERR_OPEN	2	Open-in signal was asserted

6. Functions reference

6.1. Functions overview

Function overview of the library:

General functions

BL_GetLibVersion	Return the version of the library
BL_GetVolumeSerialNumber	Return the volume serial number
BL_GetErrorMsg	Return the message corresponding to the selected error code

Communication functions

BL_Connect	Open a connection with the selected instrument
BL_Disconnect	Close the connection
BL_TestConnection	Test the communication with the instrument
BL_TestCommSpeed	Test the communication speed
BL_GetUSBdeviceinfos	Get information on a USB device

Firmware functions

BL_LoadFirmware	Load the firmware on selected channels
-----------------	--

Channel information functions

BL_IsChannelPlugged	Test if selected channel is plugged
BL_GetChannelsPlugged	Return the channels which are plugged
BL_GetChannelBoardType	Return the type of the selected channel (see section 5.3. <i>Constants</i>)
BL_GetChannelInfos	Return information on selected channel
BL_GetMessage	Return the messages generated by the firmware of the selected channel
BL_GetHardConf	Return ThardwareConf object with electrode connection mode and instrument ground.
BL_SetHardConf	Set the electrode connection mode and instrument ground with a ThardwareConf object.
BL_GetOptErr	Retrieve the current error status

Technique functions

BL_LoadTechnique	Load a technique and its parameters on selected channel
BL_LoadTechnique_LV	loads a technique and its parameters on selected channel Used with LABVIEW
BL_UpdateParameters	Update a technique with new parameters on selected channel
BL_UpdateParameters_LV	Update a technique with new parameters on selected channel Used with LabView
BL_DefineBoolParameter	Populate TEccParam structure with a boolean
BL_DefineSglParameter	Populate TEccParam structure with a single
BL_DefineIntParameter	Populate TEccParam structure with an integer

Start/stop functions

BL_StartChannel	Start technique(s) loaded on selected channel
BL_StartChannels	Start technique(s) loaded on selected channels
BL_StopChannel	Stops selected channel
BL_StopChannels	Stops selected channels

Data functions

BL_GetCurrentValues	Return current values (Ewe, Ece, I, t, ...) from selected channel
BL_GetData	Return data from selected channel
BL_GetData_LV	Return data from selected channel Used with LABVIEW
BL_ConvertNumericIntoSingle	Deprecated, use BL_ConvertChannelNumericIntoSingle instead.
BL_ConvertChannelNumericIntoSingle	Convert a numerical value coming from the data buffer of a channel from the specified type into a single
BL_ConvertTimeChannelNumericIntoSeconds	Convert the time numerical value coming from the data buffer of a channel from

the specified type into a time value in seconds.

Miscellaneous functions

BL_SetExperimentInfos	Save experiment information on selected channel
BL_GetExperimentInfos	Read experiment information from selected channel
BL_SendMsg	Send a message to the selected channel
BL_LoadFlash	Update the communication firmware of the instrument

6.2. General functions

Function	BL_GetLibVersion
Syntax	function BL_GetLibVersion(pVersion: PChar; psize: uint32): int32;
Parameters	<i>pVersion</i> pointer to the buffer that will receive the text (C-string format) <i>psize</i> pointer to a uint32 specifying the maximum number of characters of the buffer. It also returns the number of characters of the copied string.
Return value	= 0 : the function succeeded < 0 : see section 5. Structures and Constants
Description	This function copies the version of the library into the buffer.
Delphi example	<pre> procedure DisplayVersion; var pVersion: PChar; len: int32; begin len:= 255 pVersion:= StrAlloc(len); zeromemory(pVersion, len); BL_GetLibVersion(pVersion, @len); ShowMessage(pVersion); StrDispose(pVersion); end; </pre>

Function	BL_GetVolumeSerialNumber
Syntax	procedure BL_GetVolumeSerialNumber: uint32;
Description	This function returns the volume serial number. NOTE: the serial number of a (logical) drive is generated every time a drive is formatted. When Windows formats a drive, a drive's serial number gets calculated using the current date and time and is stored in the drive's boot sector. The odds of two disks getting the same number are virtually nil on the same machine.
Delphi	<pre> procedure DisplayVolumeSerialNumber; </pre>

example	<pre> var VolumeSerialNumber: uint32; begin VolumeSerialNumber := BL_GetVolumeSerialNumber; ShowMessage(inttostr(VolumeSerialNumber)); end; </pre>
----------------	--

Function	BL_GetErrorMsg
----------	----------------

Syntax	<pre> function BL_GetErrorMsg(errorcode: int32; pmsg: PChar; psize: pint32): int32; </pre>
Parameters	<i>errorcode</i> error code selected <i>pmsg</i> pointer to the buffer that will receive the text (C-string format) <i>psize</i> pointer to a uint32 specifying the maximum number of characters of the buffer. It also returns the number of characters of the copied string.
Return value	= 0 : the function succeeded < 0 : see section 5. Structures and Constants
Description	This function copies into the buffer the message corresponding to a given error code.
Delphi example	<pre> procedure DisplayErrorMessage; var msg: PChar; len: int32; begin len := 255; msg := StrAlloc(len); zeromemory(msg, len); BL_GetErrorMsg(-10, msg, @len); ShowMessage(msg); StrDispose(msg); end; </pre>

6.3. Communication functions

Function	BL_Connect
Syntax	<pre>function BL_Connect(pstr : PChar; TimeOut: uint8; pID: pint32; pInfos: PDeviceInfos): int32;</pre>
Parameters	<i>pstr</i> pointer to the buffer specifying the instrument to connect to (C-string format). Ex : 192.109.209.200, USB0, USB1, ... (see section APPENDIX A. Find instruments to detect available instruments) <i>TimeOut</i> communication time-out in second (5 s recommended). This timeout will be used for the multithreading mutex that is used to control access to the network. When 2 functions are using the network, the second will be denied access until the first has finished. If the timeout is reached, the second function will return an error ERR_GEN_FUNCTIONINPROGRESS. <i>pID</i> pointer to a int32 that will receive the device identifier of the instrument <i>pInfos</i> pointer to a device information structure (see section 5. Structures and Constants)
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function establishes the connection with the selected instrument and copies general information (device code, RAM size, ...) into the TDeviceInfos structure. The returned identifier (ID) must be used in all other routines to communicate with the instrument.
Delphi example	<pre>var ID: int32; {device identifier} procedure Connect; var IPaddress: array[0..15] of char; {IP address} Infos: TDeviceInfos; {device information} begin IPaddress:= '192.109.209.226' + #0; if BL_Connect(IPaddress, 10, @ID, @Infos) = 0 then ShowMessage('Connection OK !'); end;</pre>

Function	BL_Disconnect
Syntax	function BL_Disconnect(ID: int32): int32;
Parameters	ID : device identifier
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function closes the connection with the instrument.
Delphi example	<pre> var ID: int32; {device identifier} procedure Disconnect; begin BL_Disconnect(ID); end; </pre>

Function	BL_TestConnection
Syntax	function BL_TestConnection(ID: int32): int32;
Parameters	ID: device identifier
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function tests the communication with the selected instrument.
Delphi example	<pre> var ID: int32; {device identifier} procedure TestConnection; begin if BL_TestConnection(ID) = 0 then ShowMessage('Connection OK') else ShowMessage('Connection failed !'); end; </pre>

Function	BL_TestCommSpeed
Syntax	<pre>function BL_TestCommSpeed(ID: int32; channel: uint8; spd_rcvt: pint32; spd_kernel: pint32): int32;</pre>
Parameters	<i>ID</i> device identifier <i>channel</i> selected channel (0 .. 15) <i>spd_rcvt</i> pointer to a int32 that will receive the communication speed (in ms) between the library and the device <i>spd_kernel</i> pointer to a int32 that will receive the communication speed (in ms) between the library and the selected channel
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function tests the communication speed between the library and the selected channel. This function is for advanced users only.
Delphi example	<pre>var ID: int32; {device identifier} procedure TestCommSpeed; var spd_rcvt, spd_kernel: int32; begin if BL_TestCommSpeed(FID, 0, @spd_rcvt, @spd_kernel) = ERR_NOERROR then ShowMessage('Comm speed : ' + inttostr(spd_rcvt) + 'ms/' + inttostr(spd_kernel) + 'ms'); end;</pre>

Function	BL_GetUSBdeviceinfos
Syntax	<pre>function BL_GetUSBdeviceinfos(USBIndex: uint32; pcompany: PChar; pcompanysize: uint32; pdevice: PChar; pdevicesize: uint32; pSN: PChar; pSNsize: uint32): boolean;</pre>
Parameters	<i>USBIndex</i> index of USB device selected (0-based) <i>pcompany</i> pointer to the buffer that will receive the company name (C-string format) <i>pcompanysize</i> pointer to a uint32 specifying the maximum number of characters of the buffer. It also returns the number of characters of the copied string. <i>pdevice</i> pointer to the buffer that will receive the device name (C-string format) <i>pdevicesize</i> pointer to a uint32 specifying the maximum number of characters of the buffer. It also returns the number of characters of the copied string. <i>pSN</i> pointer to the buffer that will receive the device serial number (C-string format) <i>pSNsize</i> pointer to a uint32 specifying the maximum number of characters of the buffer. It also returns the number of characters of the copied string.
Return value	= TRUE: the function succeeded = FALSE: the function failed
Description	This function returns information stored in the selected USB device. This function is for advanced users only.

6.4. Firmware functions

Function	BL_LoadFirmware
Syntax	<pre>function BL_LoadFirmware(ID: int32; pChannels: puint8; pResults: pint32; Length: uint8; ShowGauge: boolean; ForceReload: boolean; BinFile: Pchar; XlxFile: PChar): int32;</pre>
Parameters	<i>ID</i> device identifier <i>pChannels</i> pointer to the array specifying a set of channels of the device. For each element of the array: = 0: channel not selected = 1: channel selected <i>pResults</i> pointer to the array that will receive the result of the function for each channel : = 0: the function succeeded < 0: see section 4.3 for error codes <i>Length</i> length of the arrays pointed to by pChannels and pResults. <i>ShowGauge</i> if TRUE a gauge is shown during the firmware loading. <i>ForceReload</i> if TRUE the firmware is loaded unconditionally, if FALSE the firmware is loaded only if not already done. <i>BinFile</i> Name of bin file (nil for default file). <i>XlxFile</i> Name of xilinx file (nil for default file). VMP300-series P boards don't need a Xlx file
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function loads the firmware on selected channels. Be aware that channels are unusable until the firmware is loaded.

**Delphi
example**

```
var ID: int32; {device identifier}

procedure LoadFirmware;
var
  Channels: array[1..16] of uint8;
  Results: array[1..16] of int32;
begin
  {Initialize array}
  BL_GetChannelsPlugged(ID, @Channels, 16);
  zeromemory(@Results, sizeof(Results));

  {Load firmware}
  BL_LoadFirmware(ID, @Channels, @Results, 16, TRUE, FALSE, nil,
nil);
end;
```


6.5. Channel information functions

Function	BL_IsChannelPlugged
Syntax	function BL_IsChannelPlugged(ID: int32; ch: uint8): boolean;
Parameters	<i>ID</i> device identifier <i>ch</i> selected channel (0 .. 15)
Return value	TRUE: selected channel is plugged FALSE: selected channel is not plugged
Description	This function tests if the selected channel is plugged.
Delphi example	<pre> var ID: int32; {device identifier} procedure TestChannelsPlugged; var i: int32; begin for i:= 0 to 15 do begin if BL_IsChannelPlugged(ID, i) then ShowMessage('Channel #' + inttostr(i) + ' plugged'); end; end; </pre>

Function	BL_GetChannelsPlugged
Syntax	function BL_GetChannelsPlugged (ID: int32; pChPlugged: puint8; Size: uint8): int32;
Parameters	<i>ID</i> device identifier <i>pChPlugged</i> pointer to the array representing channels of the device. For each element of the array : 0 = channel not plugged 1 = channel plugged <i>Size</i> size of the array pointed by pChPlugged

Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function returns the plugged channels.
Delphi example	See example of BL_LoadFirmware function

Function	BL_GetChannelInfos
Syntax	function BL_GetChannelInfos (ID: int32; ch: uint8; pInfos: PChannelInfos): int32;
Parameters	<i>ID</i> device identifier <i>ch</i> channel selected (0 .. 15) <i>pInfos</i> pointer on a channel information structure (see section 5. Structures and Constants)
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function copies information of the selected channel into the TChannelInfos structure.
Delphi example	<pre> var ID: int32; {device identifier} procedure DisplayChannelInfos; var Infos: TChannelInfos; begin if BL_GetChannelInfos(ID, 0, @Infos) = 0 then begin ShowMessage('SerialNumber=' + inttostr(Infos.BoardSerialNumber) + ' MemSize=' + inttostr(Infos.MemSize)); end; end; </pre>

Function	BL_GetMessage
Syntax	<pre>function BL_GetMessage(ID: int32; ch: uint8; msg: PChar; size: uint32): int32;</pre>
Parameters	<i>ID</i> device identifier <i>channel</i> selected channel (0 .. 15) <i>msg</i> pointer to the buffer that will receive the text (C-string format) <i>size</i> pointer to a uint32 specifying the maximum number of characters of the buffer. It also returns the number of characters of the copied string.
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function copies into the buffer the messages generated by the firmware of selected channel. Be aware that messages are retrieved one-by-one, this implies this function must be called several times in order to get all messages available in the message queue.
Delphi example	<pre>var ID: int32; {device identifier} procedure DisplayMessages; var msg: PChar; len: int32; begin msg:= StrAlloc(255); repeat len:= 255; ZeroMemory(msg, len); if BL_GetMessage(ID, 0, msg, @len) <> 0 then Exit; if len > 0 then ShowMessage(msg); until (len = 0); StrDispose(msg); end;</pre>

Function	BL_GetHardConf
Syntax	<pre>function BL_GetHardConf(ID: int32; ch: uint8; pHardConf : PHardwareConf): int32;</pre>
Parameters	<i>ID</i> device identifier <i>ch</i> selected channel (0 .. 15) <i>pHardConf</i> pointer to a ThardwareConf object
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function return in <i>pHardConf</i> the hardware configuration of the channel <i>ch</i>. The hardware configuration is the electrode connection and the instrument ground. This function must be used only with VMP-300 series.
Delphi example	<pre>procedure GetHardConf(aId : int32; aCh : int32; var apHardConf : THardwareConf); var errcode : int32; begin errcode := BL_GetHardConf(aId, aCh, @apHardConf); end;</pre>

Function	BL_SetHardConf
Syntax	<pre>function BL_SetHardConf(ID: int32; ch: uint8; HardConf : THardwareConf): int32;</pre>
Parameters	<i>ID</i> device identifier <i>ch</i> selected channel (0 .. 15) <i>HardConf</i> THardwareConf object. The attribute "Conn" is the electrode connection mode and the attribute "Ground" is the instrument ground. See 5. Structures and Constants to have the value of these attributes.
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function set the hardware configuration of a channel with <i>HardConf</i> object. This function must be used only with VMP-300 series.
Delphi example	<pre>procedure SetHardConf(aId : int32; aCh : int32; var aHardConf : THardwareConf); var errcode : int32; begin errcode := BL_SetHardConf(aId, aCh, aHardConf); end;</pre>

Function	BL_GetOptErr
Syntax	<pre>function BL_GetOptErr(ID: int32; ch: uint8; pOptError: pint32; pOptPos: pint32): int32;</pre>
Parameters	<i>ID</i> device identifier <i>ch</i> selected channel (0 .. 15) <i>pOptError</i> pointer to the current error status, an integer. (see section 5.4. Error codes) <i>pOptPos</i> pointer to the current option address in error, an integer. It is 0 when the current error is not related to an option.
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function returns the current error status. If the error is related to an option, OptPos contains the address of that option. Note : the error status is reset to 0 (No Error) after this function has been called.
Delphi example	<pre>procedure GetOptErr(aId : int32; aCh : int32; var OptError : pint32; var OptPos : pint32); var errcode : int32; begin errcode := BL_GetOptErr(aId, aCh, @OptError, @OptPos); end;</pre>

Function BL_GetChannelBoardType	
Syntax	<pre>function BL_GetChannelBoardType(ID: int32; Channel: uint8; pChannelType: puint32;): int32;</pre>
Parameters	<i>ID</i> device identifier <i>Channel</i> selected channel (0 .. 15) <i>pChannelType</i> Pointer to the returned channel type.
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function returns the board Type (see 5. Structures and Constants) of the target channel into pChannelType. It can be called before the firmware is loaded and can be used to select the firmware file that should be sent.
Delphi example	See BL_GetData.

6.6. Technique functions

Function	BL_LoadTechnique
Syntax	<pre>function BL_LoadTechnique (ID: int32; channel: uint8; pFName: PChar; Params: TEccParams; FirstTechnique: boolean; LastTechnique: boolean; DisplayParams: boolean): int32;</pre>
Parameters	<i>ID</i> device identifier <i>channel</i> selected channel (0 .. 15) <i>pFName</i> pointer to the buffer containing the name of the *.ecc file which defines the technique (C-string format) <i>Params</i> structure of parameters of selected technique (see section 5. Structures and Constants). See section 7. Techniques for a complete description of parameters available for each technique. <i>FirstTechnique</i> TRUE if the technique loaded is the first one <i>LastTechnique</i> TRUE if the technique loaded is the last one <i>DisplayParams</i> Display parameters sent (for debugging purpose)
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function loads a technique and its parameters on the selected channel. Note: to run linked techniques, this function must be called for each selected technique.
Delphi example	<pre>var ID: int32; {device identifier} function LoadTechniques: boolean; var</pre>


```

{OCV}
EccParamArray_OCV: array of TEccParam;
EccParams_OCV: TEccParams;
fname_OCV: PChar;
{VSCAN}
EccParamArray_VSCAN: array of TEccParam;
EccParams_VSCAN: TEccParams;
fname_VSCAN: Pchar;

begin
  Result:= FALSE;
  fname_OCV:= nil;
  fname_VSCAN:= nil;
  SetLength(EccParamArray_OCV, 4);
  SetLength(EccParamArray_VSCAN, 20);
  try
    {Define OCV parameters}
    fname_OCV:= StrNew('ocv.ecc' + #0);
    BL_DefineSglParameter('Rest_time_T', 0.1, 0,
                          @EccParamArray_OCV[0]);
    BL_DefineSglParameter('Record_every_dE', 0.1, 0,
                          @EccParamArray_OCV[1]);
    BL_DefineSglParameter('Record_every_dT', 0.01, 0,
                          @EccParamArray_OCV[2]);
    BL_DefineIntParameter('E_Range', ERANGE_AUTO, 0,
                          @EccParamArray_OCV[3]);

    {Load OCV on selected channel}
    EccParams_OCV.len:= length(EccParamArray_OCV);
    EccParams_OCV.pParams:= @EccParamArray_OCV[0];
    if BL_LoadTechnique(ID, {device identifier}
                       0, {selected channel}
                       fname_OCV, {ECC filename(c-string)}
                       EccParams_OCV, {parameters}
                       TRUE, {first technique}
                       FALSE, {last technique}
                       FALSE) <> 0 then Exit; {display params}

    {Define VSCAN parameters}
    fname_VSCAN:= StrNew('vscan.ecc' + #0);
    {Vertex #0}
    BL_DefineSglParameter ('Voltage_step', 0.0, 0,
                          @EccParamArray_VSCAN[0]);
    BL_DefineBoolParameter('vs_initial', FALSE, 0,
                          @EccParamArray_VSCAN[1]);
    BL_DefineSglParameter ('Scan_Rate', 0.0, 0,
                          @EccParamArray_VSCAN[2]);

    {Vertex #1}
    BL_DefineSglParameter ('Voltage_step', 1.0, 1,
                          @EccParamArray_VSCAN[3]);
    BL_DefineBoolParameter('vs_initial', FALSE, 1,
                          @EccParamArray_VSCAN[4]);
    BL_DefineSglParameter ('Scan_Rate', 10.0, 1,
                          @EccParamArray_VSCAN[5]);

    {Vertex #2}
    BL_DefineSglParameter ('Voltage_step', -2.0, 2,
                          @EccParamArray_VSCAN[6]);
    BL_DefineBoolParameter('vs_initial', FALSE, 2,
                          @EccParamArray_VSCAN[7]);
    BL_DefineSglParameter ('Scan_Rate', 15.0, 2,
                          @EccParamArray_VSCAN[8]);

    {Vertex #3}
    BL_DefineSglParameter ('Voltage_step', 0.0, 3,

```

```

                                @EccParamArray_VSCAN[9]);
BL_DefineBoolParameter('vs_initial', FALSE, 3,
                                @EccParamArray_VSCAN[10]);
BL_DefineSglParameter ('Scan_Rate', 20.0, 3,
                                @EccParamArray_VSCAN[11]);
{Misc.}
BL_DefineIntParameter ('Scan_number', 2, 0,
                                @EccParamArray_VSCAN[12]);
BL_DefineIntParameter ('N_Cycles', 0, 0,
                                @EccParamArray_VSCAN[13]);
BL_DefineSglParameter ('Record_every_dE', 0.01, 0,
                                @EccParamArray_VSCAN[14]);
BL_DefineSglParameter ('Begin_measuring_I', 0.4, 0,
                                @EccParamArray_VSCAN[15]);
BL_DefineSglParameter ('End_measuring_I', 0.8, 0,
                                @EccParamArray_VSCAN[16]);
BL_DefineIntParameter ('I_Range', IRANGE_10MA, 0,
                                @EccParamArray_VSCAN[17]);
BL_DefineIntParameter ('E_Range', ERANGE_AUTO, 0,
                                @EccParamArray_VSCAN[18]);
BL_DefineIntParameter ('Bandwidth', BANDWIDTH_5, 0,
                                @EccParamArray_VSCAN[19]);

{Load VSCAN on selected channel}
EccParams_VSCAN.len:= length(EccParamArray_VSCAN);
EccParams_VSCAN.pParams:= @EccParamArray_VSCAN[0];
if BL_LoadTechnique(ID, {device identifier}
                    0, {selected channel}
                    fname_VSCAN, {*.ecc filename(c-string)}
                    EccParams_VSCAN, {parameters}
                    FALSE, {first technique}
                    TRUE, {last technique}
                    FALSE) <> 0 then Exit; {display params}

    Result:= TRUE;
finally
    if fname_OCV <> nil then StrDispose(fname_OCV);
    if fname_VSCAN <> nil then StrDispose(fname_VSCAN);
    SetLength(EccParamArray_OCV, 0);
    SetLength(EccParamArray_VSCAN, 0);
end;
end;

```

Function	BL_LoadTechnique_LV
Syntax	<pre>function BL_LoadTechnique_LV (ID: int32; channel: uint8; pFName: PChar; HdIParams: PPEccParams_LV; FirstTechnique: boolean; LastTechnique: boolean; DisplayParams: boolean): int32;</pre>
Parameters	<i>ID</i> device identifier <i>channel</i> selected channel (0 .. 15) <i>FName</i> pointer to the buffer containing the name of the *.ecc file which defines the technique (C-string format) <i>HdIParams</i> structure of parameters of selected technique (see section 5. Structures and Constants). See section 7. Techniques for a complete description of parameters available for each technique. <i>FirstTechnique</i> TRUE if the technique loaded is the first one. <i>LastTechnique</i> TRUE if the technique loaded is the last one. <i>DisplayParams</i> Display parameters sent (for debugging purpose).
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function loads a technique and its parameters on the selected channel. It has been developed for LabVIEW compatibility. Note: to run linked techniques, this function must be called for each selected technique.
Delphi example	See example of BL_LoadTechnique function

Function	BL_DefineBoolParameter
Syntax	function BL_DefineBoolParameter(lbl: PChar; value: boolean; index: int32; pParam: PEccParam): int32;
Parameters	<i>lbl</i> parameter label (C-string format) <i>value</i> parameter value (boolean) <i>index</i> parameter index (useful only for multi-step parameters) <i>pParam</i> pointer on a elementary technique parameter structure (see section 5. Structures and Constants)
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function must be used to populate TEccParam structure with a boolean.
Delphi example	See example of BL_LoadTechnique function

Function	BL_DefineSglParameter
Syntax	function BL_DefineSglParameter(lbl: PChar; value: single; index: int32; pParam: PEccParam): int32;
Parameters	<i>lbl</i> parameter label (C-string format) <i>value</i> parameter value (single) <i>index</i> parameter index (useful only for multi-step parameters) <i>pParam</i> pointer on a elementary technique parameter structure (see section 5. Structures and Constants)
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function must be used to populate TEccParam structure with a

	single.
Delphi example	See example of BL_LoadTechnique function

Function BL_DefineIntParameter

Syntax	function BL_DefineIntParameter(lbl: PChar; value: int32; index: int32; pParam: PEccParam): int32;
Parameters	<i>lbl</i> parameter label (C-string format) <i>value</i> parameter value (int32) <i>index</i> parameter index (useful only for multi-step parameters) <i>pParam</i> pointer on a elementary technique parameter structure (see section 5. Structures and Constants)
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function must be used to populate TEccParam structure with an integer
Delphi example	See example of BL_LoadTechnique function

Function BL_UpdateParameters

Syntax	function BL_UpdateParameters(ID: int32; channel: uint8; TechIdx: int32; Params: TeccParams; EccFileName: PAnsiChar): int32;
Parameters	<i>ID</i> device identifier <i>channel</i> selected channel (0 .. 15) <i>TechIdx</i> index of the technique if several techniques have been started. The first have index 0, the second have index 1, ... <i>Params</i>

	array of parameters of selected technique (see section 5. Structures and Constants). See section 7. Techniques for a complete description of parameters available for each technique.
	<i>EccFileName</i> pointer to the buffer containing the name of the *.ecc file which defines the technique (C-string format)
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function updates a technique with new parameters on the selected channel. You should call this function only while an experiment is running.
Note	- For timing reasons, you cannot update more than 10 parameters at a time. Any call to this function with more than 10 parameters will fail. - There are parameters that cannot be changed while running. These are set before the technique begins and are unlocked only at the end, see Error: Reference source not found.
Delphi example	See example of BL_LoadTechnique function

Function	BL_UpdateParameters_LV
Syntax	function BL_UpdateParameters_LV(ID: int32; channel: uint8; TechIndx: int32; HdlParams: PPEccParams_LV; EccFileName: PAnsiChar): int32;
Parameters	<i>ID</i> device identifier <i>channel</i> selected channel (0 .. 15) <i>TechIndx</i> index of the technique if several techniques have been started. The first have index 0, the second have index 1, ... <i>HdlParams</i> array of parameters of selected technique (see section 5. Structures and Constants). See section 7. Techniques for a complete description of parameters available for each technique. <i>EccFileName</i> pointer to the buffer containing the name of the *.ecc file which

	defines the technique (C-string format)
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function updates a technique with new parameters on the selected channel. It has been developed for LabView compatibility.
Note	- For timing reasons, you cannot update more than 10 parameters at a time. Any call to this function with more than 10 parameters will fail. - There are parameters that cannot be changed while running. These are set before the technique begins and are unlocked only at the end, see Error: Reference source not found.
Delphi example	See OCV LabView examples (ocv.vi)

.7. Start/stop functions

Function	BL_StartChannel
Syntax	<pre>function BL_StartChannel(ID: int32; channel: uint8): int32;</pre>
Parameters	<i>ID</i> device identifier <i>channel</i> selected channel (0 .. 15)
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function starts technique(s) loaded on selected channel.
Delphi example	<pre>var ID: int32; {device identifier} procedure StartChannel; begin if BL_StartChannel(ID, 0) = 0 then ShowMessage('Channel started !'); end;</pre>

Function	BL_StartChannels
Syntax	<pre>function BL_StartChannels(ID: int32; pChannels: puint8; pResults: pint32; Length: uint8): int32;</pre>
Parameters	<i>ID</i> device identifier <i>pChannels</i> pointer to the array representing channels of the device. For each element of the array: = 0: channel not selected = 1: channel selected <i>pResults</i> pointer to the array that will receive the result of the function for each channels : = 0: the function succeeded < 0: see section 5. Structures and Constants <i>Length</i> length of the arrays pointed by pChannels and pResults.
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function starts technique(s) loaded on selected channels.
Delphi example	<pre>var ID: int32; {device identifier} procedure StartChannels; var Channels: array[1..16] of uint8; Results: array[1..16] of int32; begin {Initialize array} BL_GetChannelsPlugged(ID, @Channels, 16); zeromemory(@Results, sizeof(Results)); {Start channels} if BL_StartChannels(ID, @Channels, @Results, 16) = 0 then ShowMessage('Channels started !'); end;</pre>

Function	BL_StopChannel
Syntax	<pre>function BL_StopChannel(ID: int32; channel: uint8): int32;</pre>
Parameters	<i>ID</i> device identifier <i>channel</i> selected channel (0 .. 15)
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function stops the selected channel.
Delphi example	<pre>var ID: int32; {device identifier} procedure StopChannel; begin if BL_StopChannel(ID, 0) = 0 then ShowMessage('Channel stopped !'); end;</pre>

Function	BL_StopChannels
Syntax	<pre>function BL_StopChannels(ID: int32; pChannels: puint8; pResults: pint32; Length: uint8): int32;</pre>
Parameters	<i>ID</i> device identifier <i>pChannels</i> pointer to the array representing channels of the device. For each element of the array: = 0: channel not selected = 1: channel selected <i>pResults</i> pointer to the array that will receive the result of the function for each channels: = 0: the function succeeded < 0: see section 5. Structures and Constants <i>Length</i> length of the arrays pointed by pChannels and pResults.
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function stops the selected channels.
Delphi example	<pre>var ID: int32; {device identifier} procedure StopChannels; var Channels: array[1..16] of uint8; Results: array[1..16] of int32; begin {Initialize array} BL_GetChannelsPlugged(ID, @Channels, 16); zeromemory(@Results, sizeof(Results)); {Stop channels} if BL_StopChannels(ID, @Channels, @Results, 16) = 0 then ShowMessage('Channels stopped !'); end;</pre>

6.8. Data functions

Function	BL_GetCurrentValues
Syntax	<pre>function BL_GetCurrentValues(ID: int32; channel: uint8; pValues: PCurrentValues): int32;</pre>
Parameters	<i>ID</i> device identifier <i>channel</i> selected channel (0 .. 15) <i>pValues</i> pointer to a current values structure (see section 5. Structures and Constants)
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function copies current values (Ewe, Ece, I, t, ...) from the selected channel into the structure TCurrentValues.
Delphi example	<pre>var ID: int32; {device identifier} procedure DisplayCurrentValues; var curvalues: TCurrentValues; begin if BL_GetCurrentValues(ID, 0, @curvalues) = 0 then ShowMessage('Ewe(V)= ' + FloatToStr(curvalues.Ewe) + ' Ece(V)= ' + FloatToStr(curvalues.Ece) + ' I(A)= ' + FloatToStr(curvalues.I)); end;</pre>

Function	BL_GetData
Syntax	<pre>function BL_GetData (ID : int32; channel : uint8; pBuf : PDataBuffer; pInfos : PdataInfos; pValues : PCurrentValues): int32;</pre>
Parameters	<i>ID</i> device identifier <i>channel</i> selected channel (0 .. 15)

	<i>pBuf</i> pointer to the buffer that will receive data. <i>pInfos</i> pointer to a data information structure (see section 5. Structures and Constants) <i>pValues</i> pointer to TCurrentValues structure (see section 5. Structures and Constants)
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function copies data from the selected channel into the buffer and copies information into the TDataInfos structure. The field TECHNIQUEID of the structure TDataInfos contains the ID of the technique used to record data. Thanks to this technique ID one can identify the format of the data in the buffer (see section 7. Techniques for a complete description of data formats for each technique). Be aware that techniques can also be composed of several process (PEIS and GEIS for instance). In this case one can identify the process used to record data with the field PROCESSINDEX of the structure TDataInfos.
Delphi example	<pre> var ID: int32; {device identifier} procedure GetData; var buf: TPDataBuffer; infos: TDataInfos; values: TCurrentValues; idx: int32; row: int32; thigh: int32; tlow: int32; t: double; Ec: single; Ewe: single; Ece: single; I: single; Imoy: single; freq: single; cycle: int32; boardType: uint32; begin BL_GetChannelBoardType(ID, Channel, @boardType); repeat {Get data} if BL_GetData(ID, 0, @buf, @infos, @values) <> 0 then Exit; ShowMessage('Technique ID=' + inttostr(infos.TechniqueID) + ' Technique index=' + inttostr(infos.TechniqueIndex) + ' Number of points=' + inttostr(infos.NbRows)); {Display data} for row := 0 to infos.NbRows - 1 do begin idx := row*infos.NbCols; </pre>

```

        {OCV technique}
        if (infos.TechniqueID = TECHNIQUEID_OCV) then
        begin
            BL_ConvertTimeChannelNumericIntoSeconds(@buf[idx+1], &t,
values.TimeBase, boardType);
            t := t + infos.StartTime*0.001;
            BL_ConvertChannelNumericIntoSingle(buf[idx+3], @Ewe,
boardType);
            BL_ConvertChannelNumericIntoSingle(buf[idx+4], @Ece,
boardType);
            ShowMessage('t=' + format('%.3e', [t]) + 's ' +
'Ewe=' + format('%.3e', [Ewe]) + 'V ' +
'Ece=' + format('%.3e', [Ece]) + 'V ');
        end

        {CV technique}
        else if (infos.TechniqueID = TECHNIQUEID_CV) then
        begin
            BL_ConvertTimeChannelNumericIntoSeconds(@buf[idx+1], &t,
values.TimeBase, boardType);
            t := t + infos.StartTime*0.001;
            BL_ConvertChannelNumericIntoSingle(buf[idx+3], @Ec,
boardType);
            BL_ConvertChannelNumericIntoSingle(buf[idx+4], @Imoy,
boardType);
            BL_ConvertChannelNumericIntoSingle(buf[idx+5], @Ewe,
boardType);
            cycle := buf[idx+6];
            ShowMessage('t=' + format('%.3e', [t]) + 's ' +
'Ec=' + format('%.3e', [Ec]) + 'V ' +
'Imoy=' + format('%.3e', [Imoy]) + 'A ' +
'Ewe=' + format('%.3e', [Ewe]) + 'V ' +
'cycle=' + inttostr(cycle));
        end

        {PEIS technique}
        else if (infos.TechniqueID = TECHNIQUEID_PEIS) then
        begin
            if infos.ProcessIndex = 0 then {PEIS, 1th process}
            begin
                BL_ConvertTimeChannelNumericIntoSeconds(@buf[idx+1], &t,
values.TimeBase, boardType);
                t := t + infos.StartTime*0.001;
                BL_ConvertChannelNumericIntoSingle(buf[idx+3], @Ewe,
boardType);
                BL_ConvertChannelNumericIntoSingle(buf[idx+4], @I,
boardType);
                ShowMessage('t=' + format('%.3e', [t]) + 's ' +
'Ewe=' + format('%.3e', [Ewe]) + 'V ' +
'I=' + format('%.3e', [I]) + 'A ');
            end
            else if infos.ProcessIndex = 1 then {PEIS, 2th process}
            begin
                BL_ConvertChannelNumericIntoSingle(buf[idx+1], @freq,
boardType);
                {...}
                BL_ConvertChannelNumericIntoSingle(buf[idx+5], @Ewe,
boardType);
                BL_ConvertChannelNumericIntoSingle(buf[idx+6], @I,
boardType);
                {...}
                BL_ConvertChannelNumericIntoSingle(buf[idx+14], @t,
boardType);
                {...}
                ShowMessage('freq=' + format('%.3e', [freq]) + 'Hz ' +
't=' + format('%.3e', [t]) + 's ' +
'Ewe=' + format('%.3e', [Ewe]) + 'V ' +
'I=' + format('%.3e', [I]) + 'A ');
            end;
        end;

```

```

        end;

        end;    {for-loop}
        until (values.MemFilled = 0) and
            (values.State = STATE_STOP);
    end;

```

Function BL_GetData_LV

Syntax

```

function BL_GetData_LV (ID      : int32;
                        channel : uint8;
                        pBuf    : PDataBuffer;
                        pInfos   : PdataInfos;
                        pValues  : PCurrentValues): int32;

```

Parameters

ID
device identifier

channel
selected channel (0 .. 15)

pBuf
pointer to the buffer that will receive data.

pInfos
pointer to a data information structure (see section 5. Structures and Constants)

pValues
pointer to TCurrentValues structure (see section 5. Structures and Constants)

Return value

= 0: the function succeeded
< 0: see section 5. Structures and Constants

Description

This function copies data from the selected channel into the buffer and copies information into the TDataInfos structure. It has been developed for LabView compatibility.
It is based on the function BL_GetData, with an adjustment of the memory to avoid the problem of alignment.

The field TECHNIQUEID of the structure TDataInfos contains the ID of the technique used to record data. Thanks to this technique ID one can identify the format of the data in the buffer (see section 7. Techniques for a complete description of data format for each technique).

Be aware that techniques can also be composed of several process (PEIS and GEIS for instance). In this case one can identify the process used to record data with the field PROCESSINDEX of the structure TDataInfos.

Function	BL_ConvertNumericIntoSingle
Syntax	function BL_ConvertNumericIntoSingle(num: uint32; psgl: psingle): int32;
Parameters	<i>num</i> numerical value <i>psgl</i> pointer to the single that will receive the result of conversion
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	Deprecated. Use BL_ConvertChannelNumericIntoSingle instead. The function still remains in ECLib.dll and applications using it will still work as long as no Premium P boards are included in the instrument.
Delphi example	

Function	BL_ConvertChannelNumericIntoSingle
Syntax	function BL_ConvertChannelNumericIntoSingle(num: uint32; pRetFloat: psingle; ChannelType: uint32): int32;
Parameters	<i>num</i> numerical value <i>pRetFloat</i> pointer to the single that will receive the result of conversion <i>ChannelType</i> Channel type constant. Return from BL_GetChannelBoardType.
Return value	= 0: the function succeeded

	< 0: see section 5. Structures and Constants
Description	This function converts a numerical value coming from the data buffer of BL_GetData function received from the specified channel type into single float format.
Delphi example	See BL_GetData.

Function	BL_ConvertTimeChannelNumericIntoSeconds
Syntax	<pre>function BL_ConvertTimeChannelNumericIntoSeconds(pnum: uint32; pRetTime: pdouble; Timebase: single; ChannelType: uint32): int32;</pre>
Parameters	<i>pnum</i> Pointer to the numerical value in the data buffer <i>pRetTime</i> pointer to the double that will receive the result of conversion <i>Timebase</i> Timebase value for the technique that recorded the data. Can be read from the current <i>ChannelType</i> Channel type constant. Return from BL_GetChannelBoardType.
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function converts a numerical value coming from the data buffer of BL_GetData function received from the specified channel type into a time value in second from the technique start in a double float format.
Delphi example	See BL_GetData.

6.9. Miscellaneous functions

Function	BL_SetExperimentInfos
Syntax	<pre>function BL_SetExperimentInfos(ID: int32; channel: uint8; ExpInfos: TExperimentInfos): int32;</pre>
Parameters	<i>ID</i> device identifier <i>channel</i> selected channel (0 .. 15) <i>ExpInfos</i> Experiment information (see section 5. Structures and Constants)
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function saves experiment information on selected channel.
Delphi example	<pre>var ID: int32; {device identifier} procedure SaveExperimentInfos; var ExpInfos: TExperimentInfos; begin zeromemory(@ExpInfos, sizeof(TExperimentInfos)); ExpInfos.Group:= 0; ExpInfos.PCidentifier:= 1; ExpInfos.TimeHMS:= 2; ExpInfos.TimeYMD:= 3; StrCopy(@ExpInfos.FileName, 'test'); if BL_SetExperimentInfos(ID, 0, ExpInfos) = 0 then ShowMessage('Experiment information saved !'); end;</pre>

Function	BL_GetExperimentInfos
Syntax	<pre>function BL_GetExperimentInfos(ID: int32; channel: uint8; pExpInfos: PexperimentInfos): int32;</pre>
Parameters	<i>ID</i> device identifier <i>channel</i> selected channel (0 .. 15) <i>pExpInfos</i> pointer to an experiment information structure (see section 5. Structures and Constants)
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function copies experiment information from selected channel into the TEXPperimentINFOS structure.
Delphi example	<pre>var ID: int32; {device identifier} procedure ReadExperimentInfos; var ExpInfos: TExperimentInfos; begin zeromemory(@ExpInfos, sizeof(TExperimentInfos)); if BL_GetExperimentInfos(ID, 1, @ExpInfos) = 0 then ShowMessage('Experiment information read !'); end;</pre>

Function	BL_SendMsg
Syntax	<pre>function BL_SendMsg(ID: int32; ch: uint8; pbuf: pointer; plen: uint32): int32;</pre>
Parameters	<i>ID</i> device identifier <i>ch</i> selected channel (0 .. 15) <i>pbuf</i> pointer to the data buffer <i>plen</i> pointer to a uint32 specifying the length of data to transfer. It also returns the length of data copied into the buffer.
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function sends a message to the selected channel. Function for advanced users only.

Function	BL_LoadFlash
Syntax	<pre>function BL_LoadFlash(ID: int32; pfname: PChar; ShowGauge: boolean): int32;</pre>
Parameters	<i>ID</i> device identifier <i>pfname</i> file name (*.flash extension) of the new communication firmware (C-string format) <i>ShowGauge</i> show a gauge during transfer
Return value	= 0: the function succeeded < 0: see section 5. Structures and Constants
Description	This function updates the communication firmware of the instrument. Function for advanced users only.

7. Techniques

7.1. Notes

- See the description of the function `BL_LoadTechnique` for a complete example of how to load a technique and its parameters on a channel.
- See the description of the function `BL_GetData` for a complete example of data recovery and data conversion.
- PEIS, GEIS, SPEIS, SGEIS, PZIR and GZIR techniques can only be used with boards with impedance capabilities.
- Electrochemical techniques parameters must be carefully programmed according to the instrument hardware specifications. Be aware that wrong parameters can generate faulty operations of the technique.
- Please note that the name of the ECC file is *name.ecc* except to VMP-300 series. For VMP-300 series the name is *name4.ecc*. For VMP-300 series P boards, the name is *name5.ecc*.
- The counter-electrode value of the EIS techniques must not be taken into account with the VMP-300 series.
- Some technique parameters cannot be modified when an experiment is running. They are set at the start of the technique and are unlocked only when it is finished. The immutable parameters are presented in section 7.1.3.
- The parameter names are *case-sensitive*. The `BL_LoadTechnique` and `BL_UpdateParameters` will issue an error when a parameter is not recognized.

7.1.1 Recording additional values and XCTR changes from v5.34

From version 5.34 and onwards, the *xctr* parameter was enhanced to allow recording of some additional variables during an experiment.

As a consequence, some experiment variables are not recorded by default anymore in several techniques, and must be specified by the *xctr* parameter. On the other hand, it allowed several technique timebase to be lowered since the record step does not take as much time as before. For instance, the CV technique got its timebase lowered from 50 to 45 μ s.

If you want to record more than the default fields, you will have to specify the additional values to record by the XCTR parameter. Any value added to the record step will appear in the data array that is returned by the `BL_GetData` function in the order of the XCTR bit-field flag being set.

You will also need to manually add a small delay (depending on the value recorded, see 7.1.2) to the timebase, otherwise the technique might miss some measurements. These steps are explained in the [XCTR definition](#) and in the technique documentation.

7.1.2 Timebase calculation

The timebase that is presented in the next pages is the minimum amount of time that you should allow between two records for a technique. It provides sufficient time for the technique to record the basic set of data it has to measure.

When using XCTR to record additional experiment values, you will have to specify a timebase that is large enough for the machine to handle the acquisition of these extra values. The two following points will tell you how to calculate the optimal delay to add to the technique:

- Adding one extra measurement has a base cost of **5µs**. So anytime you start adding records to your technique, you should add 5µs to the original timebase.
- Each additional measurement after the first one has a cost of **0.5µs**, but you should round the final delay at the next integer value

Examples of calculation

<i>Additional value</i>	<i>Delay to add to timebase</i>
Ece	5µs
Ece, Analog IN1	$5 + 0.5 = 5.5\mu\text{s}$ rounded to 6µs
Ece, Analog IN1, IRange	$5+0.5+0.5 =$ 6µs

7.1.3 Hardware parameters

Some parameters configure the base acquisition and cannot be modified throughout the experiment. They are set at the beginning of the technique and unlocked at the end. You can set them regardless of the technique.

Hardware parameters

Label	Description	Data type	Data range
I_Range	I range	integer	see IRange constants allowed on section 5. Structures and Constants
E_Range	Ewe range	integer	see ERange constants allowed on section 5. Structures and Constants
Bandwidth	Bandwidth	integer	see bandwidth constants allowed on section 5. Structures and Constants
tb	Time base (s)	single	>0, often µs range

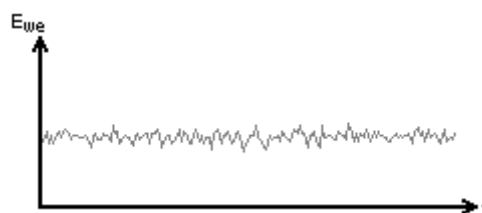
Hardware parameters

Label	Description	Data type	Data range
--------------	--------------------	------------------	-------------------

7.2. Open Circuit Voltage technique

Technique ID: **100**

Instrument Series	VMP3	VMP-300
File	ocv.ecc	ocv4.ecc ovc5.ecc
Timebase	20μs	20μs



7.2.1. Description

The Open Circuit Voltage (OCV) technique consists of a period during which no potential or current is applied to the working electrode. The cell is disconnected from the power amplifier. Only, the potential measurement is available. So the evolution of the rest potential can be recorded.

7.2.2. Technique parameters

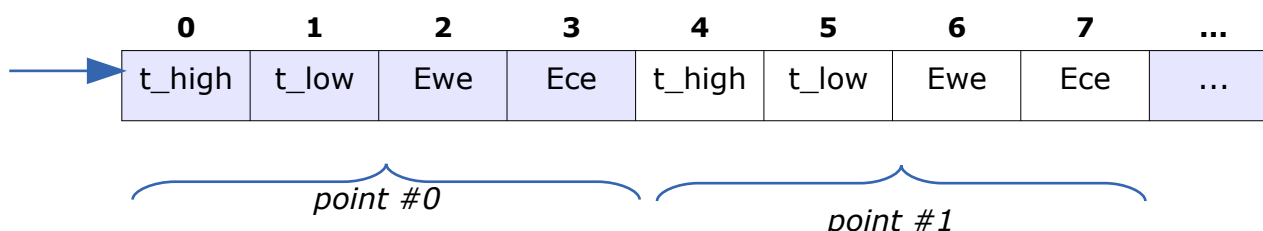
Technique parameters available for the function BL_LoadTechnique:

OCV parameters			
Label	Description	Data type	Data range
Rest_time_T	Rest duration (s)	single	[0..tb*2 ³¹]
Record_every_dE	Record every dE (V)	single	≥ 0
Record_every_dT	Record every dT (s)	single	≥ 0

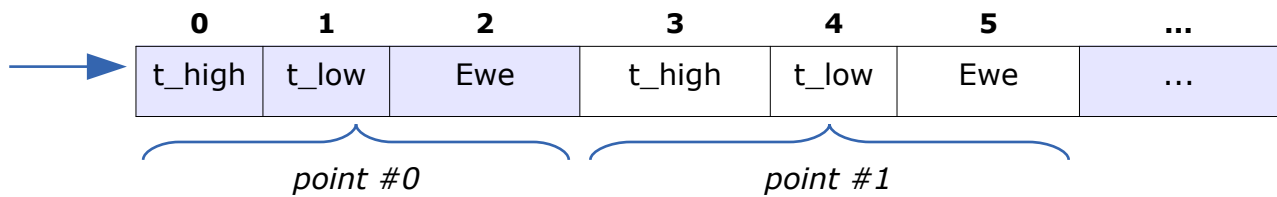
7.2.3. Data format

Data format returned by the function BL_GetData:

VMP3 series:



VMP-300 series:



The number of points saved in the buffer is returned in the field `TDataInfos.NbRows`.
The number of variables defining a point is returned in the field `TDataInfos.NbCols`.

7.2.4. Data conversion

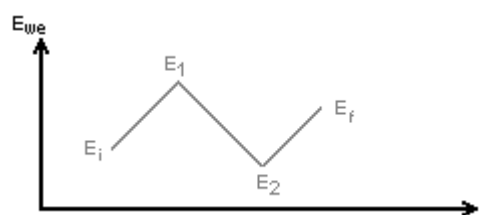
Data returned into the buffer are not usable as-is, one must convert the data before:

- time:
The time should be converted using the function `BL_ConvertTimeChannelNumericIntoSeconds`
- Float conversion:
`Ewe` and `Ece` must be converted with the function `BL_ConvertChannelNumericIntoSingle`

7.3. Cyclic Voltammetry technique

Technique ID: **103**

Instrument Series	VMP3	VMP-300
File	cv.ecc	cv4.ecc cv5.ecc
Timebase	40μs	45μs



7.3.1. Description

Cyclic voltammetry (CV) is the most widely used technique for acquiring qualitative information about electrochemical reactions. CV provides information on redox processes, heterogeneous electron-transfer reactions and adsorption processes. It offers a rapid location of redox potential of the electroactive species.

CV consists of scanning linearly the potential of a stationary working electrode using a triangular potential waveform. During the potential sweep, the potentiostat measures the current resulting from electrochemical reactions (consecutive to the applied potential). The cyclic voltammogram is a current response as a function of the applied potential.

7.3.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

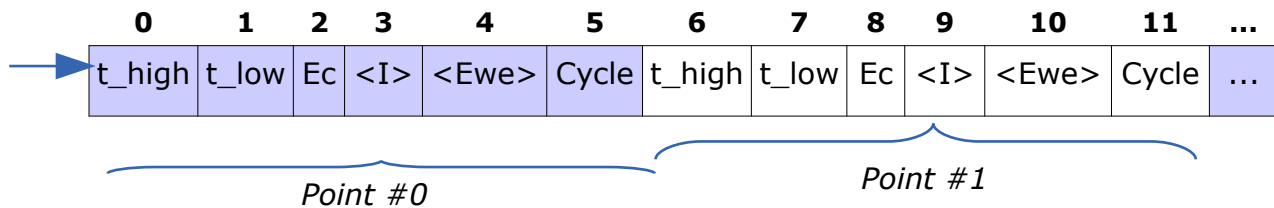
CV parameters			
Label	Description	Data type	Data range
vs_initial	Current step vs initial one	Array of 5 boolean	True/False
Voltage_step	Voltage step (V)	Array of 5 single	[E _i , E ₁ , E ₂ , E _i , E _f] see CV picture.
Scan_Rate	slew rate array (mV/s)	Array of 5 single	≥ 0
Scan_number	Scan number	integer	= 2
Record_every_dE	recording on dE (V)	single	≥ 0
Average_over_dE	average every dE	boolean	True/False
N_Cycles	Number of cycle	integer	≥ 0
Begin_measuring_I	Begin step accumulation. "1" means 100% of step	single	[0..1]

CV parameters

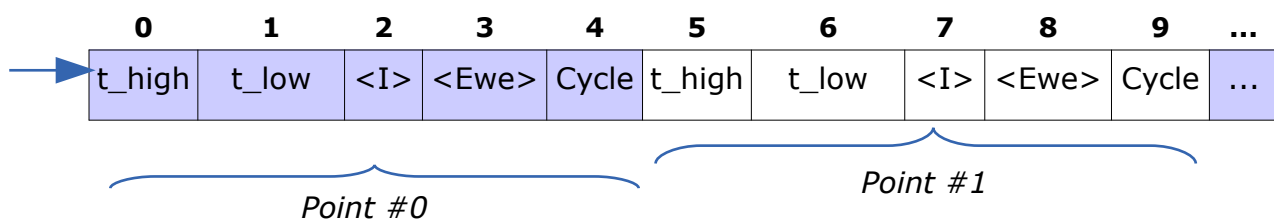
Label	Description	Data type	Data range
End_measuring_I	End step accumulation. "1" means 100% of step	single	[0..1]

7.3.3. Data format

Data format returned by the function BL_GetData on VMP3:



VMP-300 Series:



The number of points saved in the buffer is returned in the field TDataInfos.NbRows. The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.3.4. Data conversion

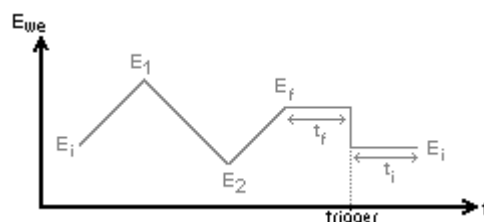
Data returned into the buffer are not usable as-is, one must convert the data before:

- time:
The time should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
Ewe and Ece must be converted with the function BL_ConvertChannelNumericIntoSingle
- cycle:
no conversion needed

7.4. Cyclic Voltammetry Advanced technique

Technique ID: **126**

Instrument Series	VMP3	VMP-300
File	biovscan.ecc	Not supported
Timebase	40μs	



7.4.1. Description

Cyclic voltammetry (CV) is the most widely used technique for acquiring qualitative information about electrochemical reactions. CV provides information on redox processes, heterogeneous electron-transfer reactions and adsorption processes. It offers a rapid location of redox potential of the electroactive species.

CV consists of scanning linearly the potential of a stationary working electrode using a triangular potential waveform. During the potential sweep, the potentiostat measures the current resulting from electrochemical reactions (consecutive to the applied potential). The cyclic voltammogram is a current response as a function of the applied potential.

7.4.1. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

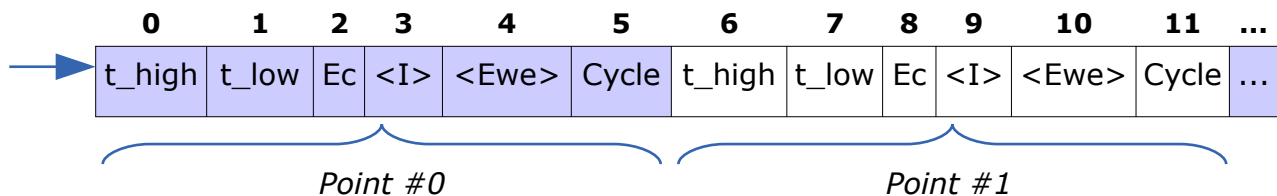
CVA parameters			
Label	Description	Data type	Data range
vs_initial_scan	Current scan vs initial one	Array of 4 boolean	True/False
Voltage_scan	Voltage scan (V)	Array of 4 single	[E _i , E ₁ , E ₂ , E _f] see CVA picture.
Scan_Rate	slew rate array (mV/s)	Array of 4 single	≥ 0
Scan_number	Scan number	integer	=2
Record_every_dE	recording on dE	single	≥ 0
Average_over_dE	average every dE	boolean	True/False
N_Cycles	Number of cycle	integer	≥ 0
Begin_measuring_I	Begin step accumulation.	single	[0..1]

CVA parameters

Label	Description	Data type	Data range
	"1" means 100% of step		
End_measuring_I	End step accumulation. "1" means 100% of step	single	[0..1]
vs_initial_step	Current step vs initial one	Array of 2 boolean	True/False
Voltage_step	Voltage step (V)	Array of 2 single	≥ 0
Duration_step	Duration step (s)	Array of 2 single	$[0..tb \cdot 2^{31}]$
Step_number	Number of steps minus 1	integer	$= 0$
Record_every_dT	Recording on dT	single	≥ 0
Record_every_dI	Recording on dI	single	≥ 0
Trig_on_off	trigger	boolean	True/False

7.4.2. Data format

Data format returned by the function BL_GetData:



The number of points saved in the buffer is returned in the field TDataInfos.NbRows. The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.4.3. Data conversion

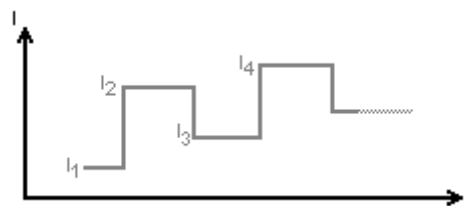
Data returned into the buffer are not usable as-is, one must convert the data before:

- time:
The time should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
Ewe and Ece must be converted with the function BL_ConvertChannelNumericIntoSingle
- cycle:
no conversion needed

7.5. Chrono-Potentiometry technique

Technique ID: **102**

Instrument Series	VMP3	VMP-300
File	cp.ecc	cp4.ecc cp5.ecc
Timebase	21μs	21μs



7.5.1. Description

The Chronopotentiometry (CP) is a controlled current technique. The current is controlled and the potential is the variable determined as a function of time. The chronopotentiometry technique is similar to the Chronoamperometry technique, potential steps being replaced by current steps. The current is applied between the working and the counter electrode.

This technique can be used for different kind of analysis or to investigate electrode kinetics. It is considered less sensitive than voltammetric techniques for analytical uses. Generally, the curves $E_{we} = f(t)$ contains plateaus that correspond to the redox potential of electroactive species.

7.5.1. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

CP parameters

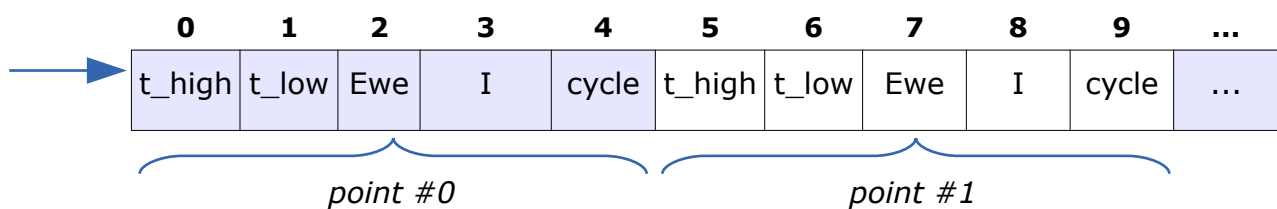
Label	Description	Data types	Data range
Current_step	Current step (A)	Array of 100 single	-
vs_initial	Current step vs initial one	Array of 100 boolean	True/False
Duration_step	Duration step (s)	Array of 100 single	$[0..tb \cdot 2^{31}]$
Step_number	Number of steps minus 1	integer	$[0..98]$
Record_every_dT	Record every dt (s)	single	≥ 0
Record_every_dE	Record every dE (V)	single	≥ 0
N_Cycles	Number of times the technique is repeated	integer	≥ 0

CP parameters

Label	Description	Data types	Data range
I_Range	I range	integer	see IRange constants allowed on section 5. Structures and Constants Warning: I Auto-range is not allowed

7.5.2. Data format

Data format returned by the function BL_GetData :



The number of points saved in the buffer is returned in the field TDataInfos.NbRows. The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.5.3. Data conversion

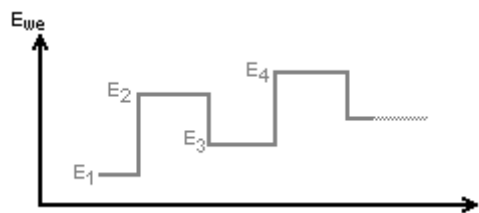
Data returned into the buffer are not usable as-is, one must convert the data before:

- time:
The time should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
Ewe and I must be converted with the function BL_ConvertChannelNumericIntoSingle
- cycle:
no conversion needed

7.6. Chrono-Amperometry technique

Technique ID: **101**

Instrument Series	VMP3	VMP-300
File	ca.ecc	ca4.ecc ca5.ecc
Timebase	24μs	21μs



7.6.1. Description

The basis of the controlled-potential techniques is the measurement of the current response to an applied potential step.

The Chronoamperometry (CA) technique involves stepping the potential of the working electrode from an initial potential, at which (generally) no faradic reaction occurs, to a potential E_i at which the faradic reaction occurs. The current-time response reflects the change in the concentration gradient in the vicinity of the surface.

Chronoamperometry is often used for measuring the diffusion coefficient of electroactive species or the surface area of the working electrode. This technique can also be applied to the study of electrode processes mechanisms.

An alternative and very useful mode for recording the electrochemical response is to integrate the current, so that one obtains the charge passed as a function of time. This is the chronocoulometric mode that is particularly used for measuring the quantity of adsorbed reactants.

7.6.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

CA parameters

Label	Description	Data types	Data range
Voltage_step	Voltage step (V)	Array of 100 single	-
vs_initial	Voltage step vs initial one	Array of 100 boolean	True/False
Duration_step	Duration step (s)	Array of 100 single	≥ 0
Step_number	Number of steps minus 1	integer	[0..98]
Record_every_dT	Record every dt (s)	single	≥ 0
Record_every_dI	Record every dI (A)	single	≥ 0

CA parameters

Label	Description	Data types	Data range
N_Cycles	Number of times the technique is repeated	integer	≥ 0

7.6.3. Data format

See CP technique data format.

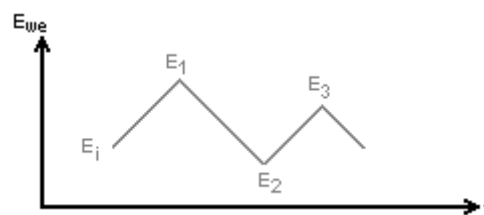
7.6.4. Data conversion

See CP technique data conversion.

7.7. Voltage Scan technique

Technique ID: **124**

Instrument Series	VMP3	VMP-300
File	vscan.ecc	vscan4.ecc vscan5.ecc
Timebase	40μs	50μs



7.7.1. Description

The Potentiodynamic (PDYN) technique allows the user to perform potentiodynamic periods with different scan rates.

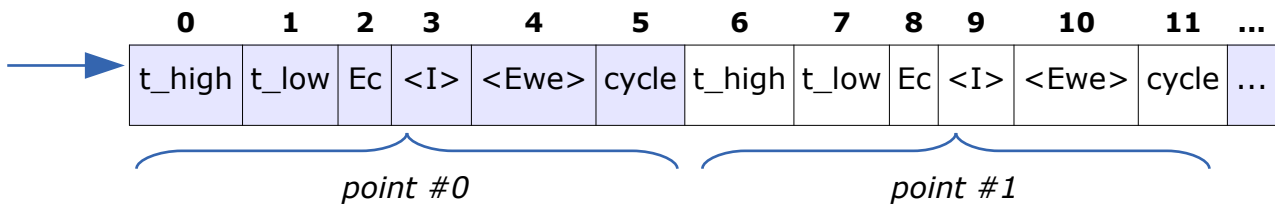
7.7.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

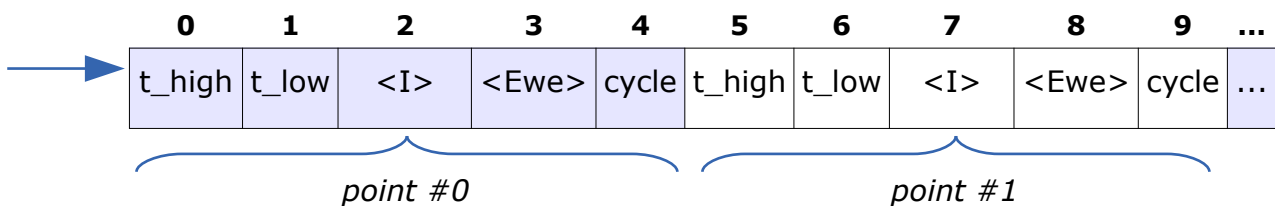
VSCAN parameters			
Parameters	Description	Data types	Data range
Voltage_step	Vertex potential (V)	Array of 100 single	-
vs_initial	Vertex potential vs initial one	Array of 100 boolean	True/False
Scan_Rate	Scan rate (V/s) from previous vertex potential	Array of 100 single	> 0, Value of the first scan-rate is ignored
Scan_number	Number of scans minus 1	integer	[0..98]
Record_every_dE	Record every dE (V)	single	≥ 0
N_Cycles	Number of times the technique is repeated	integer	≥ 0
Begin_measuring_I	Begin step accumulation. "1" means 100% of step	single	[0..1]
End_measuring_I	End step accumulation. "1" means 100% of step	single	[0..1]

7.7.3. Data format

Data format returned by the function BL_GetData for VMP3:



For VMP-300:



The number of points saved in the buffer is returned in the field TDataInfos.NbRows. The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.7.4. Data conversion

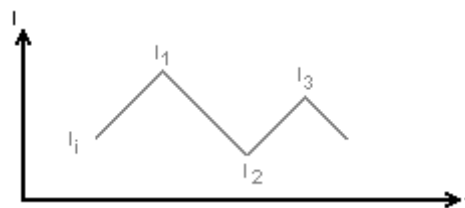
Data returned into the buffer are not usable as-is, one must convert the data before :

- time:
The time should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
Ec, <I> and Ewe must be converted with the function BL_ConvertChannelNumericIntoSingle
- cycle:
no conversion needed

7.8. Current Scan technique

Technique ID: **125**

Instrument Series	VMP3	VMP-300
File	iscan.ecc	iscan 4.ecc
Timebase	40μs	50μs



7.8.1. Description

The Galvanodynamic (GDYN) technique allows the user to perform galvanodynamic periods with different scan rates.

7.8.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

ISCAN parameters

Label	Description	Data types	Data range
Current_step	Vertex current (A)	Array of 100 single	-
vs_initial	Vertex current vs initial one	Array of 100 boolean	True/False
Scan_Rate	Scan rate (A/s) from previous vertex current	Array of 100 single	> 0
Scan_number	Number of scans minus 1	integer	[0..98]
Record_every_dI	Record every dI (A)	single	≥ 0
N_Cycles	Number of times the technique is repeated	integer	≥ 0
Begin_measuring_E	Select the part of the current step (1 = 100%) used for data averaging	single	[0..1]
End_measuring_E		single	[0..1]

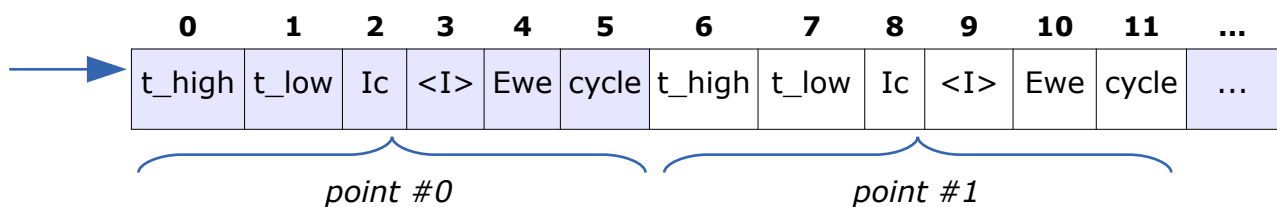
ISCAN parameters

Label	Description	Data types	Data range
I_Range	I range	integer	see IRange constants authorized on section 5. Structures and Constants

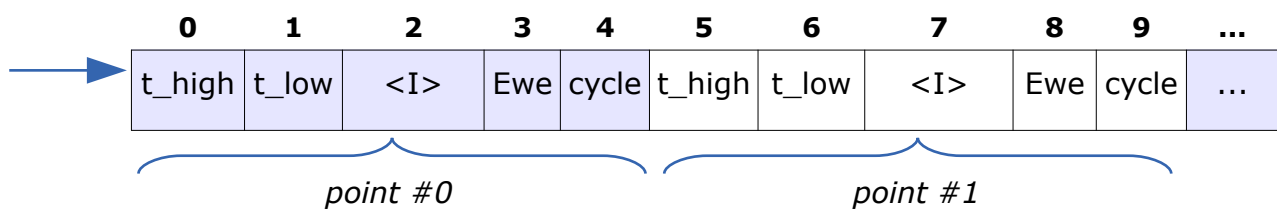
Warning : I Auto-range is not allowed

7.8.3. Data format

Data format returned by the function BL_GetData for VMP3:



For VMP-300:



The number of points saved in the buffer is returned in the field TDataInfos.NbRows.
The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.8.4. Data conversion

Data returned into the buffer are not usable as-is, one must convert the data before:

- time:
The time should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
Ic, <I> and Ewe must be converted with the function BL_ConvertChannelNumericIntoSingle
- cycle:
no conversion needed

7.9. Constant Power technique

Technique ID: **110**

Instrument Series	VMP3	VMP-300
File	pow.ecc	/
Timebase	100μs	

7.9.1. Description

The Constant Power technique is designed to study the discharge (or the charge) of a cell at constant power.

7.9.2. Technique parameters

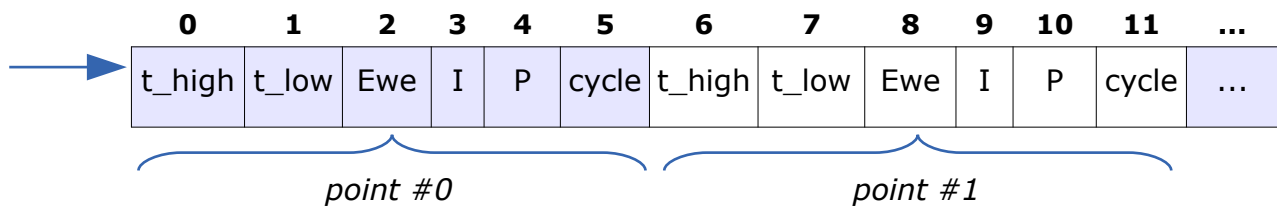
Technique parameters available for the function BL_LoadTechnique:

Constant Power parameters			
Label	Description	Data types	Data range
Power_step	Power step (W)	Array of 100 single	-
vs_initial	Power step vs initial one	Array of 100 boolean	True/False
Duration_step	Duration step (s)	Array of 100 single	$[0..tb \cdot 2^{31}]$
Step_number	Number of steps minus 1	integer	$[0..98]$
Record_every_dT	Record every dt (s)	single	≥ 0
Record_every_dE	Record every dE (V)	single	≥ 0
N_Cycles	Number of times the technique is repeated	integer	≥ 0
I_Range	I range	integer	see IRange constants authorized on section 5. Structures and Constants

Warning : I Auto-range not authorized

7.9.3. Data format

Data format returned by the function BL_GetData:



The number of points saved in the buffer is returned in the field TDataInfos.NbRows.
The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.9.4. Data conversion

Data returned into the buffer are not usable as-is, one must convert the data before:

- time:
The time should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
Ewe, I and P must be converted with the function BL_ConvertChannelNumericIntoSingle
- cycle:
no conversion needed

7.10. Constant Load technique

Technique ID: **111**

Instrument Series	VMP3	VMP-300
File	load.ecc	/
Timebase	100μs	

7.10.1. Description

The Constant Load technique is designed to discharge a battery at a constant resistance.

7.10.2. Technique parameters

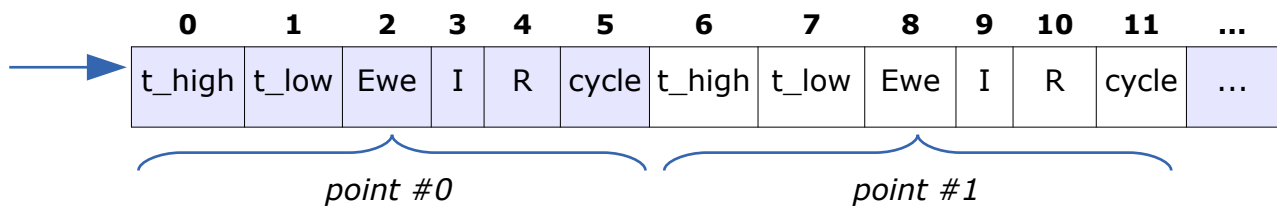
Technique parameters available for the function BL_LoadTechnique:

Constant Load parameters			
Label	Description	Data types	Data range
Load_step	Resistor step value (Ohm)	Array of 100 single	-
vs_initial	Resistor step value vs initial one	Array of 100 boolean	True/False
Duration_step	Duration step (s)	Array of 100 single	$[0..tb \cdot 2^{31}]$
Step_number	Number of steps minus 1	integer	$[0..98]$
Record_every_dT	Record every dt (s)	single	≥ 0
Record_every_dE	Record every dE (V)	single	≥ 0
N_Cycles	Number of times the technique is repeated	integer	≥ 0
I_Range	I range	integer	see IRange constants authorized on section 5. Structures and Constants

Warning : I Auto-range is not allowed

7.10.3. Data format

Data format returned by the function BL_GetData :



The number of points saved in the buffer is returned in the field TDataInfos.NbRows.
The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.10.4. Data conversion

Data returned into the buffer are not usable as-is, one must convert the data before:

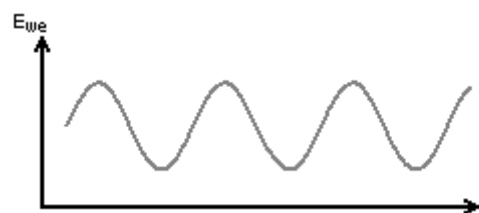
- time :
The time should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
Ewe, I and R must be converted with the function BL_ConvertChannelNumericIntoSingle
- cycle:
no conversion needed

7.11. Potentio Electrochemical Impedance Spectroscopy technique

Technique ID: **104**

Instrument Series	VMP3	VMP-300
File	peis.ecc	peis4.ecc peis5.ecc
Timebase	24μs*	24μs*

* Timebase is for first process only.



7.11.1. Description

The Potentio Electrochemical Impedance Spectroscopy (PEIS) technique performs impedance measurements into potentiostatic mode in applying a sine wave around a DC potential E that can be set to a fixed value or relatively to the cell equilibrium potential.

For very capacitive or low impedance electrochemical systems, the potential amplitude can lead to a current overflow that can stop the experiment in order to protect the unit from overheating. Using GEIS instead of PEIS can avoid this inconvenient situation.

Moreover, during corrosion experiment, a potential shift of the electrochemical system can occur. PEIS technique can lead to impedance measurements far from the corrosion potential while GEIS can be performed at a zero current.

7.11.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

PEIS parameters

Label	Description	Data types	Data range
vs_initial	Voltage step vs initial one	boolean	True/False
Initial_Voltage_step	Initial voltage step (V)	single	-
Duration_step	Step duration (s)	single	$[0..tb \cdot 2^{31}]$
Record_every_dT	Record every dt (s)	single	≥ 0
Record_every_dI	Record every dI (A)	single	≥ 0
Final_frequency	Final frequency (Hz)	single	Depend on instrument

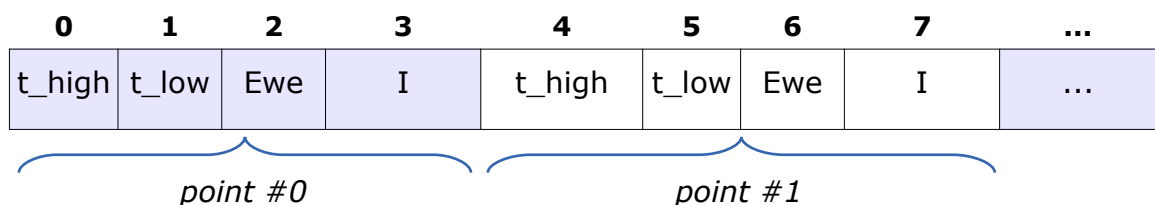
PEIS parameters

Label	Description	Data types	Data range
Initial_frequency	Initial frequency (Hz)	single	Depend on instrument
sweep	sweep linear/logarithmic (TRUE for linear points spacing)	boolean	True/False
Amplitude_Voltage	Sine amplitude (V)	single	Depend on instrument
Frequency_number	Number of frequencies	integer	≥ 1
Average_N_times	Number of repeat times (used for frequencies averaging)	integer	≥ 1
Correction	Non-stationary correction	boolean	True/False
Wait_for_steady	Number of period to wait before each frequency	single	≥ 0

7.11.3. Data format

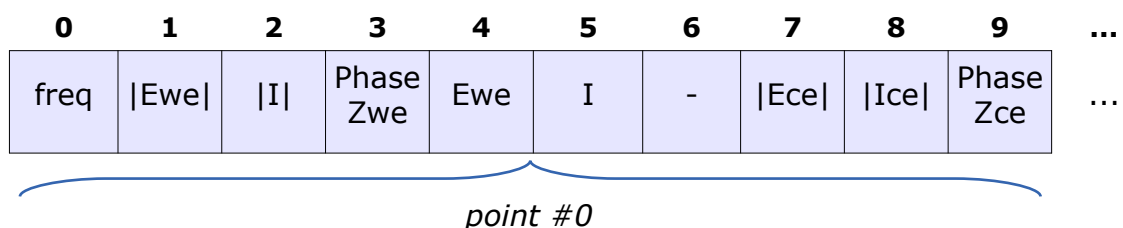
Data format depends of the technique process used to record data. The process index is returned in the field TDataInfos.ProcessIndex.

- Data format of process 0:



- Data format of process 1:

VMP3 Series:



...	10	11	12	13	14	15	16	17	18	...		
...	Ece	-	-	t	IRange	freq	Ewe	I	Phase Zwe	...		
<i>point #0</i>					<i>point #1</i>							
...	19	20	21	22	23	24	25	26	27	28	29	...
...	Ewe	I	-	Ece	Ice	Phase Zce	Ece	-	-	t	IRange	...
<i>point #1</i>												

VMP-300 Series:

	0	1	2	3	4	5	6	7	8	9	...
	freq	Ewe	I	Phase Zwe	Ewe	I	-	Ece	Ice	Phase Zce	...
<i>point #0</i>											
...	10	11	12	13	14	15	16	17			...
...	Ece	-	-	t	freq	Ewe	I	Phase Zwe			...
<i>point #0</i>					<i>point #1</i>						
...	18	19	20	21	22	23	24	25	26	27	...
...	Ewe	I	-	Ece	Ice	Phase Zce	Ece	-	-	t	...
<i>point #1</i>											

The number of points saved in the buffer is returned in the field TDataInfos.NbRows.
The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.11.4. Data conversion

Data returned into the buffer are not usable as-is, one must convert the data before:

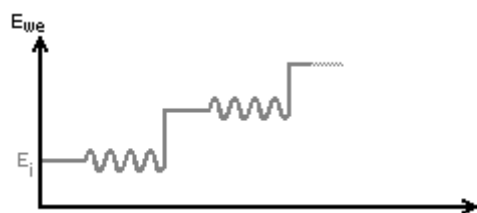
- time:
The time for process 0 should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
time (only time of process 1), IRange, freq, Ewe, |Ewe|, Ece, |Ece|, I, |I|, Phase Zwe, Phase Zce must be converted with the function BL_ConvertChannelNumericIntoSingle

- cycle:
no conversion needed

7.12. Staircase Potentio Electrochemical Impedance Spectroscopy technique

Technique ID: **113**

Instrument Series	VMP3	VMP-300
File	seisp.ecc	seisp4.ecc seisp5.ecc
Timebase	24μs	24μs



7.12.1. Description

The Staircase Potentio Electrochemical Impedance Spectroscopy (SPEIS) technique is designed to perform successive impedance measurements (on a whole frequency range) during a potential sweep. The main application of this technique is to study electrochemical reaction kinetics along voltamperometric (I(E)) curves in analytical electrochemistry. Thus this technique finds all its interest to study the complexity of non-stationary interfaces with faradic processes where the total AC response (whole frequency range) is required.

Another common application of such a technique is semi-conductor materials study. For these stationary systems only two or three frequencies for each potential step are required to determine the donor density and the flat band potential.

The SPEIS technique consists in a staircase potential sweep (potential limits and number of steps defined by the user). An impedance measurement (with an adjustable number of frequencies) is performed on each potential step. For all these applications a Mott-Schottky plot ($1/C^2$ vs. E_{we}) can be displayed and a special linear fit is applied to extract the semi-conductor parameters.

7.12.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

SPEIS parameters

Label	Description	Data types	Data range
vs_initial	Voltage step vs initial one	boolean	True/False
vs_final	Voltage step vs initial one	boolean	True/False
Initial_Voltage_step	Initial voltage step (V)	single	-
Final_Voltage_step	Final voltage step (V)	single	-

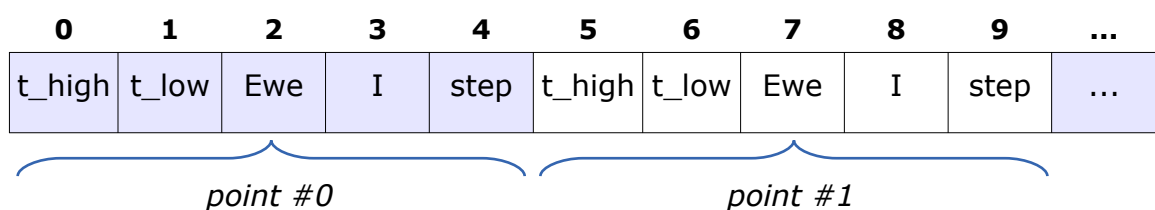
SPEIS parameters

Label	Description	Data types	Data range
Duration_step	Step duration (s)	single	[0..tb*2 ³¹]
Step_number	Number of voltage steps	integer	[0..98]
Record_every_dT	Record every dt (s)	single	≥ 0
Record_every_dI	Record every dI (A)	single	≥ 0
Final_frequency	Final frequency (Hz)	single	Depend on instrument
Initial_frequency	Initial frequency (Hz)	single	Depend on instrument
sweep	sweep linear/logarithmic (TRUE for linear points spacing)	boolean	True/False
Amplitude_Voltage	Sine amplitude (V)	single	Depend on instrument
Frequency_number	Number of frequencies	integer	≥ 1
Average_N_times	Number of repeat times (used for frequencies averaging)	integer	≥ 1
Correction	Non-stationary correction	boolean	True/False
Wait_for_steady	Number of period to wait before each frequency	single	≥ 0

7.12.3. Data format

Data format depends of the technique process used to record data. The process index is returned in the field TDataInfos.ProcessIndex.

- Data format of process 0:



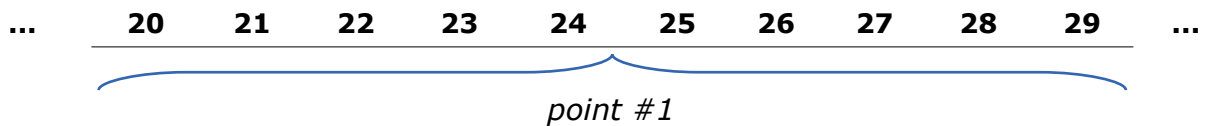
- Data format of process 1:

VMP3 Series:

	0	1	2	3	4	5	6	7	8	9	...
	freq	Ewe	I	Phase Zwe	Ewe	I	-	Ece	Ice	Phase Zce	...
	<i>point #0</i>										
...	10	11	12	13	14	15	16	17	18	19	...
...	Ece	-	-	t	Irangle	step	freq	Ewe	I	Phase Zwe	...
	<i>point #0</i>						<i>point #1</i>				
...	20	21	22	23	24	25	26	27	28	29	...
...	Ewe	I	-	Ece	Ice	Phase Zce	Ece	-	-	t	...
	<i>point #1</i>										
...	30	31	...								
...	Irangle	step	...								
	<i>point #1</i>										

VMP-300 Series:

	0	1	2	3	4	5	6	7	8	9	...
	freq	Ewe	I	Phase Zwe	Ewe	I	-	Ece	Ice	Phase Zce	...
	<i>point #0</i>										
...	10	11	12	13	14	15	16	17	18	19	...
...	Ece	-	-	t	step	freq	Ewe	I	Phase Zwe	Ewe	...
	<i>point #0</i>						<i>point #1</i>				
...	20	21	22	23	24	25	26	27	28	29	...
...	I	-	Ece	Ice	Phase Zce	Ece	-	-	t	step	...



The number of points saved in the buffer is returned in the field `TDataInfos.NbRows`.
The number of variables defining a point is returned in the field `TDataInfos.NbCols`.

7.12.4. Data conversion

Data returned into the buffer are not usable as-is, one must convert the data before:

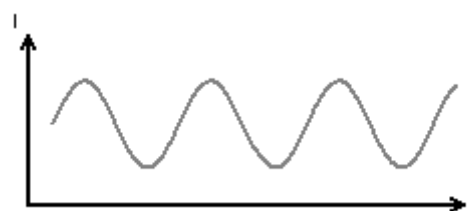
- time:
The time for process 0 should be converted using the function `BL_ConvertTimeChannelNumericIntoSeconds`
- Float conversion:
time (only time of process 1), `IRange`, `freq`, `Ewe`, `|Ewe|`, `Ece`, `|Ece|`, `I`, `|I|`, `Phase Zwe`, `Phase Zce` must be converted with the function `BL_ConvertChannelNumericIntoSingle`
- cycle:
no conversion needed

7.13. Galvano Electrochemical Impedance Spectroscopy technique

Technique ID: **107**

Instrument Series	VMP3	VMP-300
File	geis.ecc	geis4.ecc geis5.ecc
Timebase	24μs*	24μs*

* Timebase is for first process only.



7.13.1. Description

The Galvano Electrochemical Impedance Spectroscopy (GEIS) technique performs impedance measurements into galvanostatic mode in applying a sine wave around a DC current I that can be set to a fixed value or relatively to a previous controlled current. In the case of particular non-linear systems it can be necessary to use PEIS instead of GEIS.

The Electrochemical Impedance spectroscopy (EIS) finds many of applications in corrosion, battery, fuel cell development, sensors and physical electrochemistry. It can provide information on reaction parameters, corrosion rates, electrode surfaces porosity, coating, mass transport, interfacial capacitance measurements.

7.13.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

GEIS parameters			
Label	Description	Data types	Data range
vs_initial	Current step vs initial one	boolean	True/False
Initial_Current_step	Initial current step (A)	single	-
Duration_step	Step duration (s)	single	[0..tb*2 ³¹]
Record_every_dT	Record every dt (s)	single	≥ 0
Record_every_dE	Record every dE (V)	single	≥ 0
Final_frequency	Final frequency (Hz)	single	Depend on instrument
Initial_frequency	Initial frequency (Hz)	single	Depend on instrument

GEIS parameters

Label	Description	Data types	Data range
sweep	sweep linear/logarithmic (TRUE for linear points spacing)	boolean	True/False
Amplitude_Current	Sine amplitude (A)	single	[0..50% of the Irange]
Frequency_number	Number of frequencies	integer	≥ 1
Average_N_times	Number of repeat times (used for frequencies averaging)	integer	≥ 1
Correction	Non-stationary correction	boolean	True/False
Wait_for_steady	Number of period to wait before each frequency	single	≥ 0
I_Range	I range	integer	see IRange constants authorized on section 5. Structures and Constants Warning : I Auto-range is not allowed

7.13.3. Data format

See PEIS technique data format.

7.13.4. Data conversion

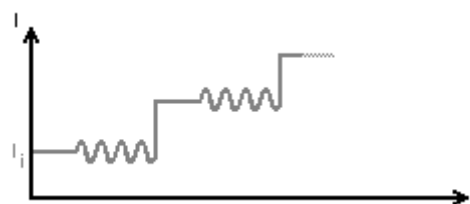
See PEIS technique data conversion.

7.14. Staircase Galvano Electrochemical Impedance Spectroscopy technique

Technique ID: **114**

Instrument Series	VMP3	VMP-300
File	seisg.ecc	seisg4.ecc seisg5.ecc
Timebase	24μs*	24μs*

* Timebase for first process only



7.14.1. Description

The Staircase Galvano Electrochemical Impedance Spectroscopy (SGEIS) technique is close to the SPEIS technique. The difference is that the potentiostat works as a galvanostat and applies a current sweep (staircase shape). In the same way an impedance measurement (whole frequency range) can be performed on each current step. The user can also select several frequencies.

7.14.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

SGEIS parameters

Label	Description	Data types	Data range
vs_initial	Current step vs initial one	boolean	True/False
vs_final	Current step vs initial one	boolean	True/False
Initial_Current_step	Initial current step (A)	single	-
Final_Current_step	Final current step (A)	single	-
Duration_step	Step duration (s)	single	[0..tb*2 ³¹]
Step_number	Number of voltage steps	integer	≥ 0
Record_every_dT	Record every dt (s)	single	≥ 0
Record_every_dE	Record every dE (V)	single	≥ 0
Final_frequency	Final frequency (Hz)	single	Depend on instrument
Initial_frequency	Initial frequency (Hz)	single	Depend on instrument

SGEIS parameters

Label	Description	Data types	Data range
sweep	sweep linear/logarithmic (TRUE for linear points spacing)	boolean	True/False
Amplitude_Current	Sine amplitude (A)	single	[0..50% of range]
Frequency_number	Number of frequencies	integer	≥ 1
Average_N_times	Number of repeat times (used for frequencies averaging)	integer	≥ 1
Correction	Non-stationary correction	boolean	True/False
Wait_for_steady	Number of period to wait before each frequency	single	≥ 0
I_Range	I range	integer	see IRange constants authorized on section 5. Structures and Constants Warning : I Auto-range is not allowed

7.14.3. Data format

See SPEIS technique data format.

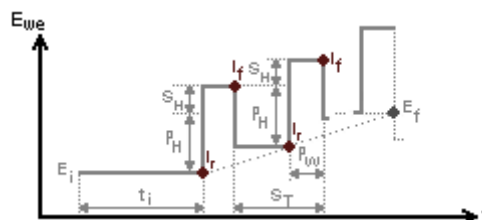
7.14.4. Data conversion

See SPEIS technique data conversion.

7.15. Differential Pulse Voltammetry technique

Technique ID: **127**

Instrument Series	VMP3	VMP-300
File	dpv.ecc	dpv4.ecc dpv5.ecc
Timebase	40μs	40μs



7.15.1. Description

DPV is very useful technique for analytical determination (for example metal ion quantification in a sample). The differential measurements discriminate faradic current from capacitive one.

In this technique, the applied waveform is the sum of a pulse train and a staircase from the initial potential (E_i) to a final potential (E_f). The current is sampled just before the pulse and near the end of the pulse. The resulting current is the difference between these two currents. It has a relatively flat baseline. The current peak height is directly related to the concentration of the electroactive species in the electrochemical cell.

7.15.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

DPV parameters			
Label	Description	Data types	Data range
E_i	Initial potential (V)	single	-
OCi	Initial potential vs initial one	boolean	True/False
Rest_time_Ti	E_i duration	single	[0..tb*2 ³¹]
E_f	Final potential (v)	single	-
OCf	Final potential vs initial one	boolean	True/False
PH	Pulse height (mV)	single	≥ 0
PW	Pulse width (ms)	single	≥ 0
SH	Step height (mV)	single	≥ 0
ST	Step width (ms)	single	≥ 0

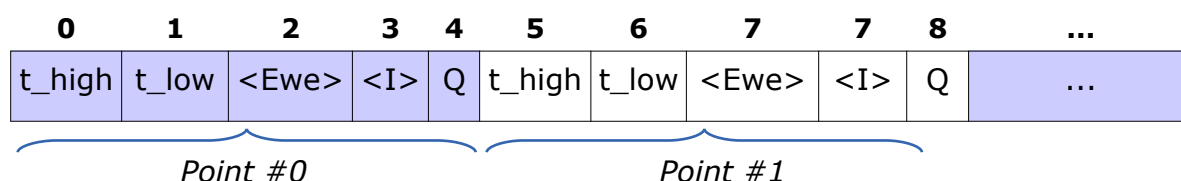
DPV parameters

Label	Description	Data types	Data range
Begin_measuring_I	Select the part of the current step. 1 = 100	single	[0..1]
End_measuring_I	% used for data averaging	single	[0..1]
I_Range	I range	integer	see IRange constants authorized on section 5. Structures and Constants

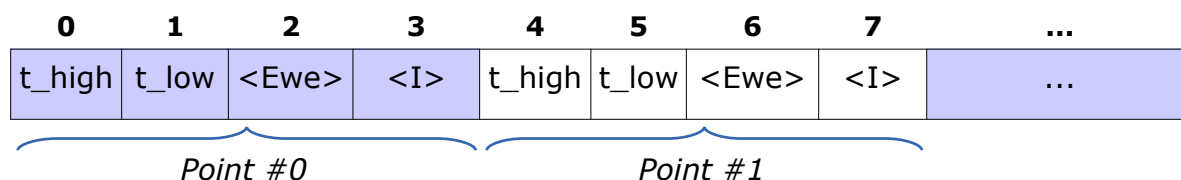
Warning: I Auto-range is not allowed

7.15.3. Data format

Data format returned by the function BL_GetData for VMP3:



For VMP-300:



The number of points saved in the buffer is returned in the field TDataInfos.NbRows.
The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.15.4. Data conversion

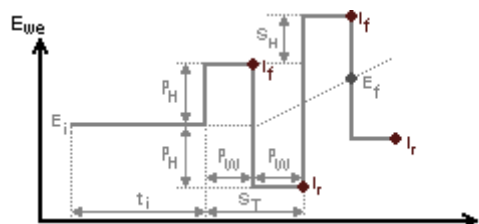
Data returned into the buffer are not usable as-is, one must convert the data before:

- time:
The time for process 0 should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
time (only time of process 1), IRange, freq, Ewe, |Ewe|, Ece, |Ece|, I, |I|, Phase Zwe, Phase Zce, Q-Qo must be converted with the function BL_ConvertChannelNumericIntoSingle

7.16. Square Wave Voltammetry technique

Technique ID: **128**

Instrument Series	VMP3	VMP-300
File	swv.ecc	swv4.ecc swv5.ecc
Timebase	40μs	40μs



7.16.1. Description

Among the electroanalytical techniques, the Square Wave Voltammetry (SWV) combines the background suppression, the sensitivity of DPV and the diagnostic value of NPV.

The SWV is a large amplitude differential technique characterized by a pulse height (PH) and a pulse width (PW). The pulse width can be expressed in term of square wave frequency $f=1/(2PW)$. The scan rate is $v = PH/(2PW)$. The current is sampled twice, once at the end of the forward pulse and once at the end of the reverse pulse. The difference between the two measurements is plotted versus the base staircase potential. The resulting peak-shaped voltammogram is symmetrical about the half-wave potential and the peak current is proportional to the concentration.

Excellent sensitivity accrues from the fact that the net current is larger than either the forward or reverse components (since it is the difference between them).

7.16.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

SWV parameters

Label	Description	Data types	Data range
Ei	Initial potential (V)	single	-
OCi	Initial potential vs initial one	boolean	True/False
Rest_time_Ti	Ei duration	single	[0..tb*2 ³¹]
Ef	Final potential (v)	single	-
OCf	Final potential vs initial one	boolean	True/False
PH	Pulse height (mV)	single	≥ 0

SWV parameters

Label	Description	Data types	Data range
PW	Pulse width (ms)	single	≥ 0
SH	Step height (mV)	single	≥ 0
Begin_measuring_I	Select the part of the current step (1 = 100%)	single	[0..1]
End_measuring_I	used for data averaging	single	[0..1]
I_Range	I range	integer	see IRange constants authorized on section 5. Structures and Constants

Warning : I Auto-range is not allowed

7.16.3. Data format

Data format returned by the function BL_GetData for VMP3:

0	1	2	3	4	5	6	7	8	9	...
t_high	t_low	<Ewe>	<I>	Q	t_high	t_low	<Ewe>	<I>	Q	...
Point #0					Point #1					

For VMP-300:

0	1	2	3	4	5	6	7	...
t_high	t_low	<Ewe>	<I>	t_high	t_low	<Ewe>	<I>	...
Point #0				Point #1				

The number of points saved in the buffer is returned in the field TDataInfos.NbRows.
The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.16.4. Data conversion

Data returned into the buffer are not usable as-is, one must convert the data before:

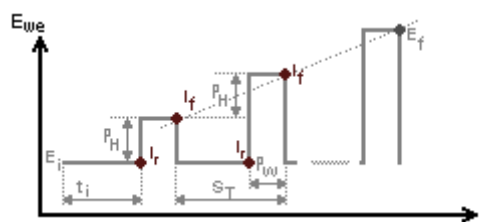
- time:
The time for process 0 should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
time (only time of process 1), IRange, freq, Ewe, |Ewe|, Ece, |Ece|, I, |I|,

Phase Zwe, Phase Zce, Q-Qo must be converted with the function
BL_ConvertChannelNumericIntoSingle

7.17. Normal Pulse Voltammetry technique

Technique ID: **129**

Instrument Series	VMP3	VMP-300
File	npv.ecc	npv4.ecc npv5.ecc
Timebase	40μs	40μs



7.17.1. Description

Pulsed techniques have been introduced to increase the ratio between the faradic and nonfaradic currents in order to permit a quantification of species to very low concentration levels.

The Normal Pulse Voltammetry is one of the first pulsed techniques elaborated for polarography needs. An essential idea behind the NPV is the cyclic renewal of the diffusion layer. With a DME, this is achieved by the stirring accompanying the fall of the mercury drop. But at other electrodes renewal may not be so easily accomplished.

NPV consists of a series of pulses of linear increasing amplitude (from E_i to E_f). The potential pulse is ended by a return to the base value E_i . The usual practice is to select E_i in a region where the electroactive species of interest does not react at the electrode. The current is sampled at a time t near the end of the pulse and at a time t' before the pulse. The plotted current is the difference of both currents measured at the end of the pulse (forward) and at the end of the period previous to the pulse (reverse).

7.17.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

NPV parameters

Label	Description	Data types	Data range
E_i	Initial potential (V)	single	-
OCi	Initial potential vs initial one	boolean	True/False
Rest_time_Ti	E_i duration	single	$[0..tb \cdot 2^{31}]$
E_f	Final potential (v)	single	-
OCf	Final potential vs initial one	boolean	True/False
PH	Pulse height (mV)	single	≥ 0

NPV parameters

Label	Description	Data types	Data range
PW	Pulse width (ms)	single	≥ 0
ST	Step width (ms)	single	≥ 0
Begin_measuring_I	Select the part of the current step (1 = 100%)	single	[0..1]
End_measuring_I	used for data averaging	single	[0..1]
I_Range	I range	integer	see IRange constants authorized on section 5. Structures and Constants

Warning: I Auto-range is not allowed

7.17.3. Data format

Data format returned by the function BL_GetData for VMP3:

0	1	2	3	4	5	6	7	8	9	...
t_high	t_low	<Ewe>	<I>	Q	t_high	t_low	<Ewe>	<I>	Q	...
Point #0					Point #1					

For VMP-300:

0	1	2	3	4	5	6	7	...
t_high	t_low	<Ewe>	<I>	t_high	t_low	<Ewe>	<I>	...
Point #0				Point #1				

The number of points saved in the buffer is returned in the field TDataInfos.NbRows.
The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.17.4. Data conversion

Data returned into the buffer are not usable as-is, one must convert the data before:

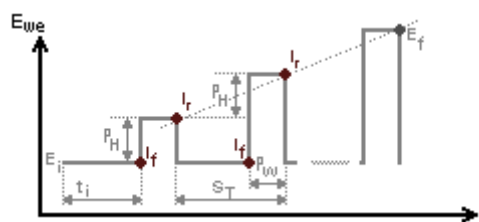
- time:
The time for process 0 should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
time (only time of process 1), IRange, freq, Ewe, |Ewe|, Ece, |Ece|, I, |I|, Phase Zwe, Phase Zce, Q-Qo must be converted with the function

BL_ConvertChannelNumericIntoSingle

7.18. Reverse Normal Pulse Voltammetry technique

Technique ID: **130**

Instrument Series	VMP3	VMP-300
File	rnpv.ecc	rnpv4.ecc rnpv5.ecc
Timebase	40μs	40μs



7.18.1. Description

The Reverse Normal Pulse Voltammetry is a derivative technique from the NPV. The main difference is that the initial (base) potential E_i is placed in the diffusion-limited region for electrolysis of the species present in the bulk solution. The pulses are made through the region where the species in solution is not electroactive.

The RNPV experiment involves a significant faradic current. This method is a reversal experiment because of the detection of the product from a prior electrolysis.

7.18.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

RNPV parameters

Label	Description	Data types	Data range
Ei	Initial potential (V)	single	-
OCi	Initial potential vs initial one	boolean	True/False
Rest_time_Ti	Ei duration	single	[0..tb*2 ³¹]
Ef	Final potential (v)	single	-
OCf	Final potential vs initial one	boolean	True/False
PH	Pulse height (mV)	single	≥ 0
PW	Pulse width (ms)	single	≥ 0
ST	Step width (ms)	single	≥ 0
Begin_measuring_I	Select the part of the current step (1 = 100%)	single	[0..1]
End_measuring_I	used for data averaging	single	[0..1]

RNPV parameters

Label	Description	Data types	Data range
I_Range	I range	integer	see IRange constants authorized on section 5. Structures and Constants Warning: I Auto-range is not allowed

7.18.3. Data format

Data format returned by the function BL_GetData for VMP3:

0	1	2	3	4	5	6	7	8	9	...
t_high	t_low	<Ewe>	<I>	Q	t_high	t_low	<Ewe>	<I>	Q	...
Point #0					Point #1					

For VMP-300:

0	1	2	3	4	5	6	7	...
t_high	t_low	<Ewe>	<I>	t_high	t_low	<Ewe>	<I>	...
Point #0				Point #1				

The number of points saved in the buffer is returned in the field TDataInfos.NbRows.
The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.18.4. Data conversion

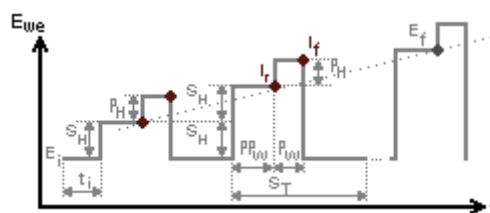
Data returned into the buffer are not usable as-is, one must convert the data before:

- time:
The time for process 0 should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
time (only time of process 1), IRange, freq, Ewe, |Ewe|, Ece, |Ece|, I, |I|, Phase Zwe, Phase Zce, Q-Qo must be converted with the function BL_ConvertChannelNumericIntoSingle

7.19. Differential Normal Pulse Voltammetry technique

Technique ID: **131**

Instrument Series	VMP3	VMP-300
File	dnpv.ecc	dnpv4.ecc dnpv5.ecc
Timebase	40μs	40μs



7.19.1. Description

Originally introduced as a polarographic technique (performed at a DME), the Differential Normal Pulse Voltammetry is a sensitive electroanalytical technique very close to the DPV technique with a pulsed potential sweep. The potential pulse is swept from an initial potential E_i to a final potential E_f . There are two main differences with the DPV technique: first the pulse waveform is made with a prepulse (S_H amplitude with PPW duration) before the pulse (S_H amplitude with PW duration) and second the potential always comes back to the initial potential (E_i) after the pulsed sequence. E_i is assumed to be the potential where no faradic reaction occurs. The plotted current is the difference of both currents measured at the end of the pulse (I forward) and at the end of the prepulse (I reverse).

This technique is often used in polarography and by biologists to define the most appropriate potential for the electrochemical detection to a fixed potential with the DPA technique.

7.19.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

DNPV parameters

Label	Description	Data types	Data range
E_i	Initial potential (V)	single	-
OCi	Initial potential vs initial one	boolean	True/False
Rest_time_Ti	E_i duration	single	$[0..tb \cdot 2^{31}]$
E_f	Final potential (v)	single	-
OCf	Final potential vs initial one	boolean	True/False
PH	Pulse height (mV)	single	≥ 0
PPW	PrePulse Width	single	≥ 0

DNPV parameters

Label	Description	Data types	Data range
PW	Pulse width (ms)	single	≥ 0
SH	Step height (mV)	single	≥ 0
ST	Step width (ms)	single	≥ 0
Begin_measuring_I	Select the part of the current step (1 = 100%)	single	[0..1]
End_measuring_I	used for data averaging	single	[0..1]
I_Range	I range	integer	see IRange constants authorized on section 5. Structures and Constants

Warning: I Auto-range is not allowed

7.19.3. Data format

Data format returned by the function BL_GetData for VMP3:

0	1	2	3	4	5	6	7	8	9	...
t_high	t_low	<Ewe>	<I>	Q	t_high	t_low	<Ewe>	<I>	Q	...
Point #0					Point #1					

For VMP-300:

0	1	2	3	4	5	6	7	...
t_high	t_low	<Ewe>	<I>	t_high	t_low	<Ewe>	<I>	...
Point #0				Point #1				

The charge can be recorded additionally using XCTR.

The number of points saved in the buffer is returned in the field TDataInfos.NbRows. The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.19.4. Data conversion

Data returned into the buffer are not usable as-is, one must convert the data before:

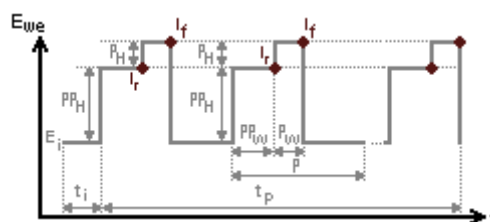
- time:
The time for process 0 should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds

- Float conversion:
time (only time of process 1), IRange, freq, Ewe, |Ewe|, Ece, |Ece|, I, |I|, Phase Zwe, Phase Zce, Q-Qo must be converted with the function BL_ConvertChannelNumericIntoSingle

7.20. Differential Pulse Amperometry technique

Technique ID: **132**

Instrument Series	VMP3	VMP-300
File	dpa.ecc	dpa4.ecc dpa5.ecc
Timebase	40μs	40μs



7.20.1. Description

The Differential Pulse Amperometry results from the DNPV technique without increasing pulse steps. The potential waveform and the current sampling are the same as for DNPV. A DPA experiment is often used as a sensitive method for the quantification of electrochemical species at a defined potential (E_s). This potential value is often determined with a DNPV experiment (using a potential sweep with the same waveform) previously performed. This technique is dedicated to the quantification of biological electroactive species.

7.20.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

DPA parameters

Label	Description	Data types	Data range
Ei	Initial potential (V)	single	-
OCi	Initial potential vs initial one	boolean	True/False
Rest_time_Ti	Ei duration	single	[0..tb*2 ³¹]
PPH	PrePulse height	single	≥ 0
PPW	PrePulse Width	single	≥ 0
PH	Pulse height (mV)	single	≥ 0
PW	Pulse width (ms)	single	≥ 0
P	Period (mV)	single	≥ 0
Tp	Duration (s)	single	≥ 0
Begin_measuring_I	Select the part of the current step (1 = 100%)	single	[0..1]
End_measuring_I	used for data averaging	single	[0..1]

DPA parameters

Label	Description	Data types	Data range
I_Range	I range	integer	see IRange constants authorized on section 5. Structures and Constants

Warning: I Auto-range is not allowed

7.20.3. Data format

Data format returned by the function BL_GetData for VMP3:

0	1	2	3	4	5	6	7	8	9	...
t_high	t_low	<Ewe>	<I>	Q	t_high	t_low	<Ewe>	<I>	Q	...
Point #0					Point #1					

For VMP-300:

0	1	2	3	4	5	6	7	...
t_high	t_low	<Ewe>	<I>	t_high	t_low	<Ewe>	<I>	...
Point #0				Point #1				

The number of points saved in the buffer is returned in the field TDataInfos.NbRows.
The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.20.4. Data conversion

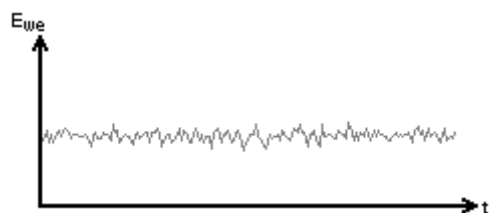
Data returned into the buffer are not usable as-is, one must convert the data before:

- time:
The time for process 0 should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
time (only time of process 1), IRange, freq, Ewe, |Ewe|, Ece, |Ece|, I, |I|, Phase Zwe, Phase Zce, Q-Qo must be converted with the function BL_ConvertChannelNumericIntoSingle

7.21. Ecorr. Vs Time technique

Technique ID: **133**

Instrument Series	VMP3	VMP-300
File	evt.ecc	evt4.ecc evt5.ecc
Timebase	20μs	20μs



7.21.1. Description

This technique corresponds to the follow up of the corrosion potential (when the circuit is open) versus time. During the measurement no potential or current is applied to the cell.

7.21.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

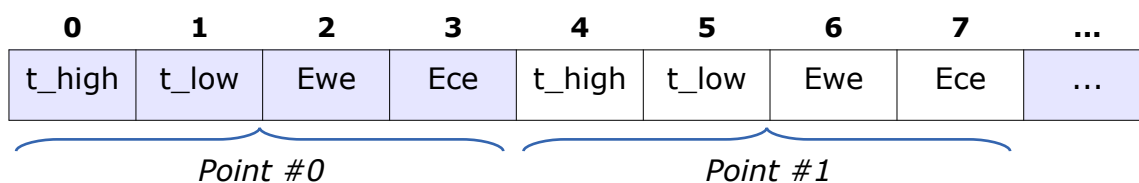
EVT parameters

Label	Description	Data type	Data range
Record_every_dEr	Record every dE (V)	single	≥ 0
Rest_time_T	Rest duration (s)	single	$[0..tb*2^{31}]$
Record_every_dTr	Record every dT (s)	single	≥ 0

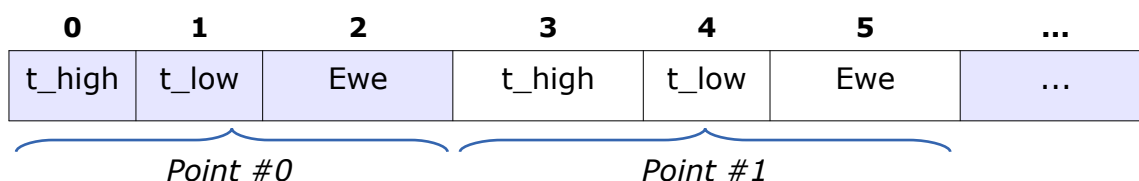
7.21.3. Data format

Data format returned by the function BL_GetData:

VMP3 series:



VMP-300 series:



The number of points saved in the buffer is returned in the field TDataInfos.NbRows.
The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.21.4. Data conversion

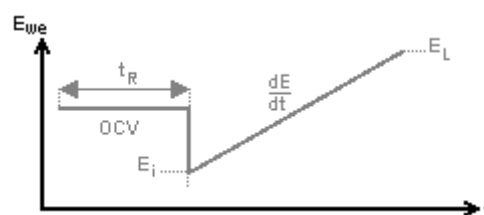
Data returned into the buffer are not usable as-is, one must convert the data before:

- time:
The time should be converted using the function
BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
Ewe and Ece must be converted with the function
BL_ConvertChannelNumericIntoSingle

7.22. Linear Polarization technique

Technique ID: **134**

Instrument Series	VMP3	VMP-300
File	lp.ecc	lp4.ecc lp5.ecc
Timebase	40μs	40μs



7.22.1. Description

The linear polarization technique is used in corrosion monitoring. This technique is especially designed for the determination of a polarization resistance R_p of a material and I_{corr} through potential steps around the corrosion potential.

R_p is defined as the slope of the potential-current density curve at the free corrosion potential: $R_p = (dE/dI)_{dE \rightarrow 0}$

R_p is determined using the "Rp fit" graphic tool. Contrary to the Potentiodynamic Pitting (PDP) technique, no current limitation is available with the linear polarization technique.

7.22.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

LP parameters

Label	Description	Data types	Data range
Record_every_dEr	Record every dE (V)	single	≥ 0
Rest_time_T	Rest duration (s)	single	$[0..tb \cdot 2^{31}]$
Record_every_dTr	Record every dT (s)	single	≥ 0
OC1	Step voltage vs initial one (not used)	boolean	True/False
E1	Step voltage (V) (not used)	single	≥ 0
T1	Step duration (s) (not used)	single	≥ 0
vs_initial_scan	Current scan vs initial one	Array of 2 boolean	True/False
Voltage_scan	Voltage scan (V)	Array of 2	-

LP parameters

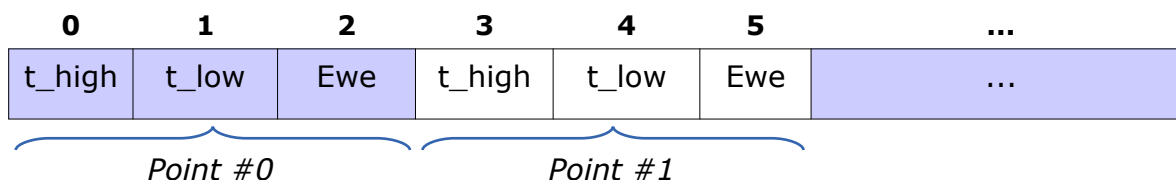
Label	Description	Data types	Data range
		single	
Scan_Rate	slew rate array (mV/s)	Array of 2 single	≥ 0
Scan_number	Scan number	integer	$= 0$
Record_every_dE	recording on dE	single	≥ 0
Average_over_dE	average every dE	boolean	True/False
Begin_measuring_I	Select the part of the current step (1 = 100%)	single	[0..1]
End_measuring_I	used for data averaging	single	[0..1]

7.22.3. Data format

Data format returned by the function BL_GetData

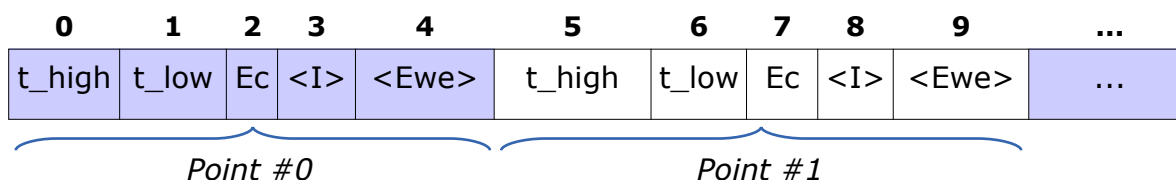
Data format depends of the technique process used to record data. The process index is returned in the field TDataInfos.ProcessIndex.

- Data format of process 0:

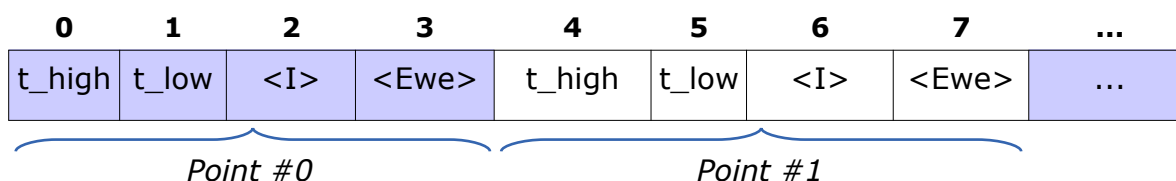


- Data format of process 1:

VMP3 Series:



VMP-300 Series:



The number of points saved in the buffer is returned in the field TDataInfos.NbRows.
The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.22.4. Data conversion

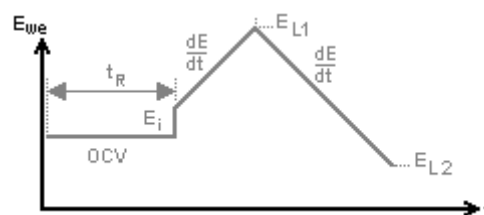
Data returned into the buffer are not usable as-is, one must convert the data before:

- time:
The time for process 0 should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
time (only time of process 1), IRange, freq, Ewe, |Ewe|, Ece, |Ece|, I, |I|, Phase Zwe, Phase Zce, Q-Qo must be converted with the function BL_ConvertChannelNumericIntoSingle

7.23. Generalized Corrosion technique

Technique ID: **135**

Instrument Series	VMP3	VMP-300
File	gc.ecc	gc4.ecc gc5.ecc
Timebase	40μs	40μs



7.23.1. Description

The generalized corrosion technique is applied for general corrosion (sometimes called uniform corrosion) study. For this corrosion, anodic dissolution is uniformly distributed over the entire metallic surface. The corrosion rate is nearly constant at all locations.

Microscopic anodes and cathodes are continuously changing their electrochemical behavior from anode to cathode cells for a uniform attack.

7.23.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

GC parameters			
Label	Description	Data types	Data range
Record_every_dEr	Record every dE (V)	single	≥ 0
Rest_time_T	Rest duration (s)	single	$[0..tb*2^{31}]$
Record_every_dTr	Record every dT (s)	single	≥ 0
OC1	Step voltage vs initial one (not used)	boolean	True/False
E1	Step voltage (V) (not used)	single	≥ 0
T1	Step duration (s) (not used)	single	≥ 0
vs_initial_scan	Current scan vs initial one	Array of 3 boolean	True/False
Voltage_scan	Voltage scan (V)	Array of 3 single	-
Scan_Rate	slew rate array (mV/s)	Array of 3 single	≥ 0
Scan_number	Scan number	integer	= 1

GC parameters

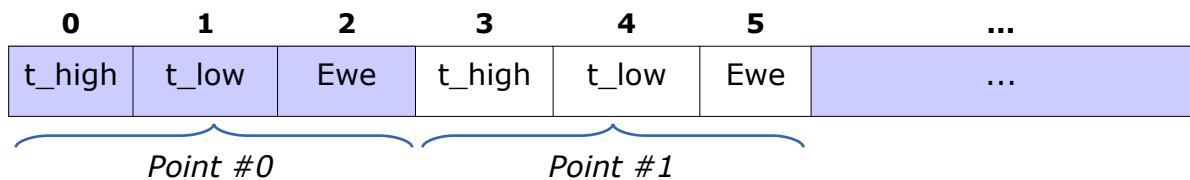
Label	Description	Data types	Data range
Record_every_dE	recording on dE	single	≥ 0
Average_over_dE	average every dE	boolean	True/False
Begin_measuring_I	Select the part of the current step (1 = 100%)	single	[0..1]
End_measuring_I	used for data averaging	single	[0..1]

7.23.3. Data format

Data format returned by the function BL_GetData

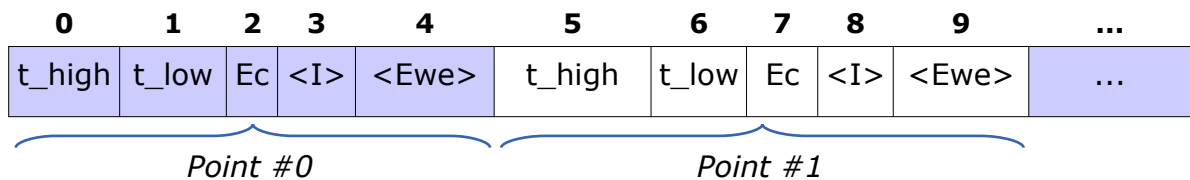
Data format depends of the technique process used to record data. The process index is returned in the field TDataInfos.ProcessIndex.

- Data format of process 0:

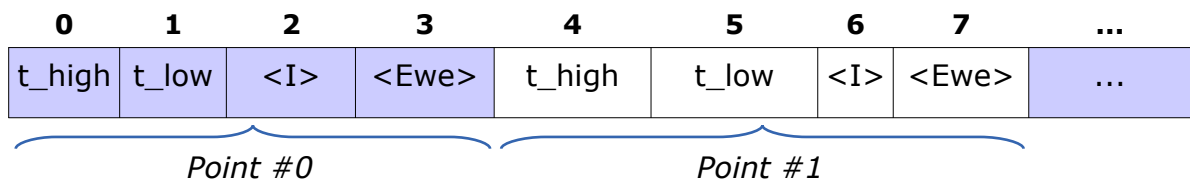


- Data format of process 1:

For VMP3:



For VMP-300:



The number of points saved in the buffer is returned in the field TDataInfos.NbRows. The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.23.4. Data conversion

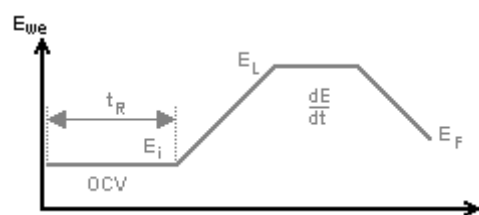
Data returned into the buffer are not usable as-is, one must convert the data before:

- time:
The time for process 0 should be converted using the function `BL_ConvertTimeChannelNumericIntoSeconds`
- Float conversion:
time (only time of process 1), IRange, freq, Ewe, |Ewe|, Ece, |Ece|, I, |I|, Phase Zwe, Phase Zce, Q-Qo must be converted with the function `BL_ConvertChannelNumericIntoSingle`

7.24. Cyclic Potentiodynamic Polarization technique

Technique ID: **136**

Instrument Series	VMP3	VMP-300
File	cpp.ecc	cpp4.ecc cpp5.ecc
Timebase	40μs	44μs



7.24.1. Description

The Cyclic Potentiodynamic Polarization is often used to evaluate pitting susceptibility. It is the most common electrochemical test for localized corrosion resistance. The potential is swept in a single cycle or slightly less than one cycle. The size of the hysteresis is examined along with the difference between the values of the starting Open circuit corrosion potential and the return passivation potential. The existence of hysteresis is usually indicative of pitting, while the size of the loop is often related to the amount of pitting. This technique can be used to determine the pitting potential and the repassivation potential.

This technique is based both on the PDP and PSP techniques. It begins with a potentiodynamic phase where the potential increases. This phase is limited either with a limit potential (E_L) or with a pitting current (I_p) defined by the user. If the pitting current is not reached during the potentiodynamic phase, then a potentiostatic phase is applied until pitting (I_p is reached). I_p can be used as a safety parameter in order to avoid damages on the working electrode. Then an additional potentiodynamic phase is done as a reverse scan.

7.24.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

CPP parameters

Label	Description	Data types	Data range
Record_every_dEr	Record every dE (V)	single	≥ 0
Rest_time_T	Rest duration (s)	single	$[0..tb*2^{31}]$
Record_every_dTr	Record every dT (s)	single	≥ 0
vs_initial_scan	Current scan vs initial one	Array of 3 boolean	True/False
Voltage_scan	Voltage scan (V)	Array of 3 single	-

CPP parameters

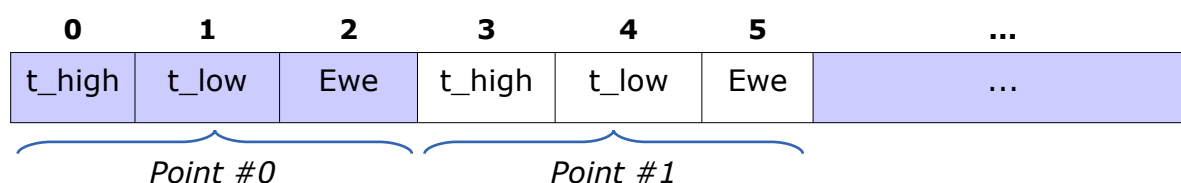
Label	Description	Data types	Data range
Scan_Rate	slew rate array (mV/s)	Array of 3 single	≥ 0
Scan_number	Scan number	integer	= 1
I_pitting	Pitting current	single	≥ 0
t_b	Check condition $ I > I_p$ after time t_b	single	≥ 0
Record_every_dE	recording on dE	single	≥ 0
Average_over_dE	average every dE	boolean	True/False
Begin_measuring_I	Select the part of the current step (1 = 100%) used for data averaging	single	[0..1]
End_measuring_I		single	[0..1]
Record_every_dT	recording on dt	single	≥ 0

7.24.3. Data format

Data format returned by the function BL_GetData

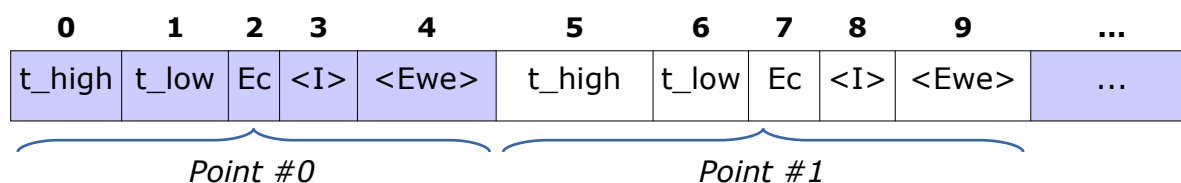
Data format depends of the technique process used to record data. The process index is returned in the field TDataInfos.ProcessIndex.

- Data format of process 0:

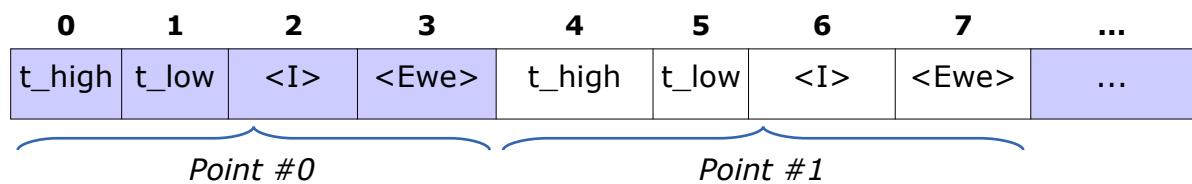


- Data format of process 1:

VMP3 Series:



VMP-300 Series:



The number of points saved in the buffer is returned in the field `TDataInfos.NbRows`. The number of variables defining a point is returned in the field `TDataInfos.NbCols`.

7.24.4. Data conversion

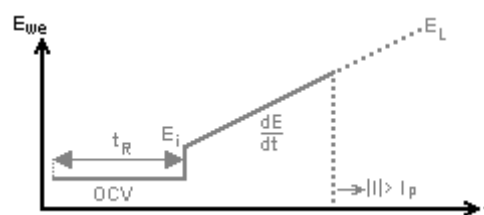
Data returned into the buffer are not usable as-is, one must convert the data before:

- time:
The time for process 0 should be converted using the function `BL_ConvertTimeChannelNumericIntoSeconds`
- Float conversion:
time (only time of process 1), `IRange`, `freq`, `Ewe`, `|Ewe|`, `Ece`, `|Ece|`, `I`, `|I|`, `Phase Zwe`, `Phase Zce`, `Q-Qo` must be converted with the function `BL_ConvertChannelNumericIntoSingle`

7.25. Potentiodynamic Pitting technique

Technique ID: **137**

Instrument Series	VMP3	VMP-300
File	pdp.ecc	pdp4.ecc pdp5.ecc
Timebase	40μs	42μs



7.25.1. Description

Pitting corrosion occurs when discrete areas of a material undergo rapid attack while the vast majority of the surface remains virtually unaffected. The basic requirement for pitting is the existence of a passive state for the material in the environment of interest. Pitting of a given material depends strongly upon the presence of an aggressive species in the environment and a sufficiently oxidizing potential.

This technique corresponds to the pitting potential determination of a material, using a potential sweep. The experiment stops when a pitting current (I_p) defined by the user is reached. I_p can be used as a safety parameter in order to avoid damages on the working electrode.

7.25.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

PDP parameters

Label	Description	Data types	Data range
Record_every_dEr	Record every dE (V)	single	≥ 0
Rest_time_T	Rest duration (s)	single	$[0..tb*2^{31}]$
Record_every_dTr	Record every dT (s)	single	≥ 0
vs_initial_scan	Current scan vs initial one	Array of 2 boolean	True/False
Voltage_scan	Voltage scan (V)	Array of 2 single	-
Scan_Rate	slew rate array (mV/s)	Array of 2 single	≥ 0
Scan_number	Scan number	integer	= 1

PDP parameters

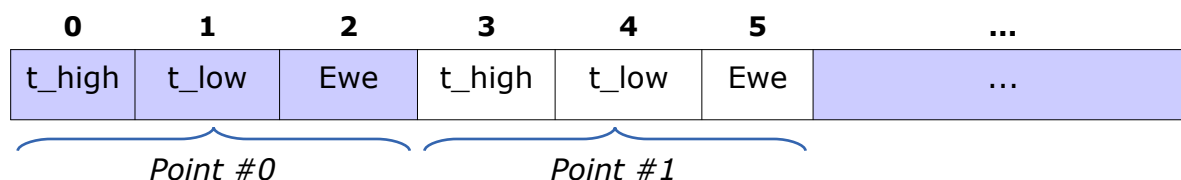
Label	Description	Data types	Data range
I_pitting	Pitting current	single	≥ 0
t_b	Check condition $ I > I_p$ after time t_b	single	≥ 0
Record_every_dE	recording on dE	single	≥ 0
Average_over_dE	average every dE	boolean	True/False
Begin_measuring_I	elect the part of the current step (1 = 100%)	single	[0..1]
End_measuring_I	used for data averaging	single	[0..1]
Record_every_dT	recording on dt	single	≥ 0
Hold	Hold potential	boolean	True/False

7.25.3. Data format

Data format returned by the function BL_GetData

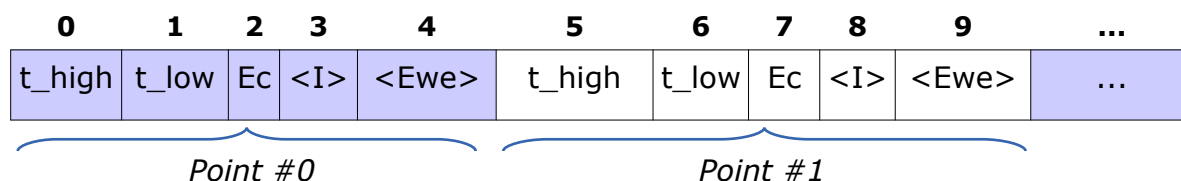
Data format depends of the technique process used to record data. The process index is returned in the field TDataInfos.ProcessIndex.

- Data format of process 0:

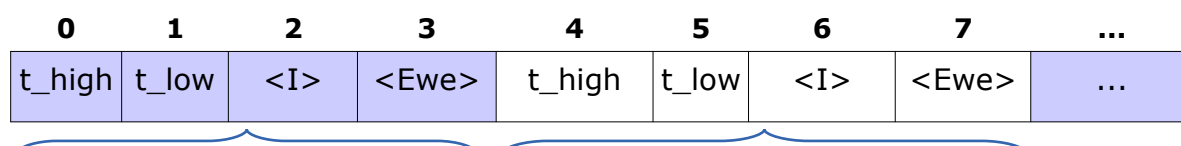


- Data format of process 1:

VMP3 Series:



VMP-300 Series:



0	1	2	3	4	5	6	7	...
<i>Point #0</i>					<i>Point #1</i>			

The number of points saved in the buffer is returned in the field TDataInfos.NbRows.
The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.25.4. Data conversion

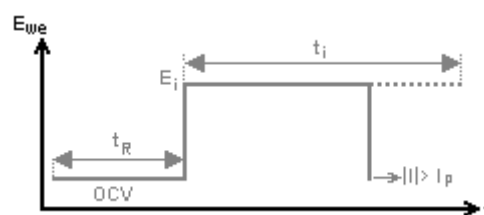
Data returned into the buffer are not usable as-is, one must convert the data before:

- time:
The time for process 0 should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
time (only time of process 1), IRange, freq, Ewe, |Ewe|, Ece, |Ece|, I, |I|, Phase Zwe, Phase Zce, Q-Qo must be converted with the function BL_ConvertChannelNumericIntoSingle

7.26. PotentioStatic Pitting technique

Technique ID: **138**

Instrument Series	VMP3	VMP-300
File	psp.ecc	psp4.ecc psp5.ecc
Timebase	24μs	24μs



7.26.1. Description

The PSP technique corresponds to studying pitting occurrence under applied constant potential.

The experiment stops when a pitting current (I_p) defined by the user is reached. I_p can be used as a safety parameter in order to avoid damages on the working electrode.

7.26.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

PSP parameters

Label	Description	Data types	Data range
Record_every_dEr	Record every dE (V)	single	≥ 0
Rest_time_T	Rest duration (s)	single	$[0..tb*2^{31}]$
Record_every_dTr	Record every dT (s)	single	≥ 0
Ei	Initial Potential (V)	single	-
OCi	Initial Potential vs initial one	boolean	True/False
Rest_time_Ti	Ei duration (s)	single	≥ 0
Record_every_dT	recording on dt	single	≥ 0
Record_every_dI	recording on dI (A)	single	≥ 0
I_pitting	Pitting current	single	≥ 0
t_b	Check condition $ I > I_p$ after time t_b	single	≥ 0
I_Range	I range	integer	see IRange

PSP parameters

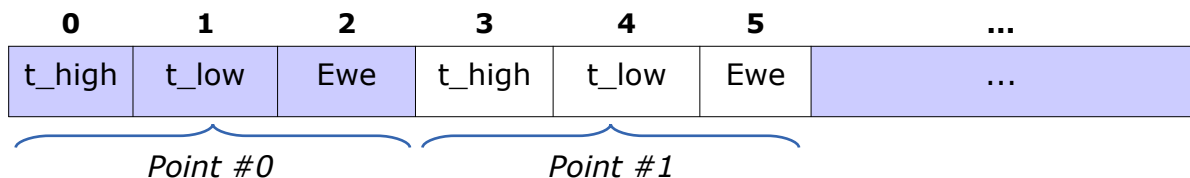
Label	Description	Data types	Data range
			constants authorized on section 5. Structures and Constants Warning : I Auto-range is not allowed

7.26.3. Data format

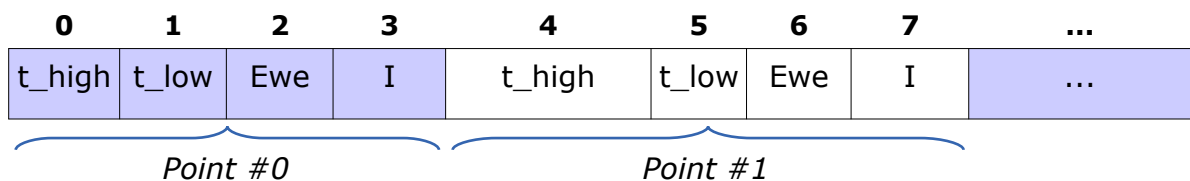
Data format returned by the function BL_GetData

Data format depends of the technique process used to record data. The process index is returned in the field TDataInfos.ProcessIndex.

- Data format of process 0:



- Data format of process 1:



The number of points saved in the buffer is returned in the field TDataInfos.NbRows.
The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.26.4. Data conversion

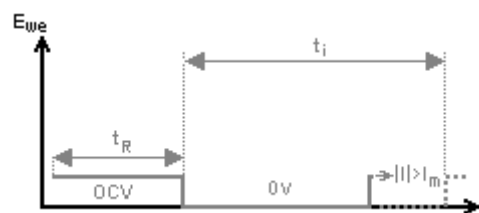
Data returned into the buffer are not usable as-is, one must convert the data before:

- time:
The time for process 0 should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
time (only time of process 1), IRange, freq, Ewe, |Ewe|, Ece, |Ece|, I, |I|, Phase Zwe, Phase Zce, Q-Qo must be converted with the function BL_ConvertChannelNumericIntoSingle

7.27. Zero Resistance Ammeter technique

Technique ID: **139**

Instrument Series	VMP3	VMP-300
File	zra.ecc	zra4.ecc zra5.ecc
Timebase	40μs	40μs



7.27.1. Description

The Zero Resistance Ammeter technique is used to examine the effects of coupling dissimilar metals and to perform some types of electrochemical noise measurement.

This technique consists into applying zero volts between the working electrode (WE) and the counter electrode (CE) and then measures the current and the potentials (E_{we} , E_{ce}) versus the reference electrode (REF).

7.27.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

ZRA parameters

Label	Description	Data types	Data range
Record_every_dEr	Record every dE (V)	single	≥ 0
Rest_time_T	Rest duration (s)	single	$[0..tb*2^{31}]$
Record_every_dTr	Record every dT (s)	single	≥ 0
Ei	Initial Potential (V)	single	-
OCi	Initial Potential vs initial one	boolean	True/False
Rest_time_Ti	Ei duration (s)	single	≥ 0
Record_every_dT	recording on dt	single	≥ 0
Record_every_dI	recording on dI (A)	single	≥ 0
I_max	Pitting current	single	≥ 0
t_b	Check condition $ I > I_p$ after time t_b	single	≥ 0

7.27.3. Data format

Data format returned by the function BL_GetData

Data format depends of the technique process used to record data. The process index is returned in the field TDataInfos.ProcessIndex.

- Data format of process 0:

VMP3 series:

0	1	2	3	4	5	6	7	...
t_high	t_low	Ewe	Ece	t_high	t_low	Ewe	Ece	...
Point #0				Point #1				

VMP-300 series:

0	1	2	3	4	5	...
t_high	t_low	Ewe	t_high	t_low	Ewe	...
Point #0			Point #1			

- Data format of process 1:

VMP3 series:

0	1	2	3	4	5	6	7	8	9	10	11	...
t_high	t_low	<Ewe>	<I>	Ece	Q	t_high	t_low	<Ewe>	<I>	Ece	Q	...
Point #0						Point #1						

VMP-300 series:

0	1	2	3	4	5	6	7	...
t_high	t_low	<Ewe>	<I>	t_high	t_low	<Ewe>	<I>	...
Point #0				Point #1				

The number of points saved in the buffer is returned in the field TDataInfos.NbRows.
The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.27.4. Data conversion

Data returned into the buffer are not usable as-is, one must convert the data before:

- time:
The time for process 0 should be converted using the function `BL_ConvertTimeChannelNumericIntoSeconds`
- Float conversion:
time (only time of process 1), IRange, freq, Ewe, |Ewe|, Ece, |Ece|, I, |I|, Phase Zwe, Phase Zce, Q-Qo must be converted with the function `BL_ConvertChannelNumericIntoSingle`

7.28. Manual IR technique

Technique ID: **140**

Instrument Series	VMP3	VMP-300
File	IRcmp.ecc	IRcmp4.ecc IRcmp5.ecc
Timebase	20μs	20μs

7.28.1. Description

The ohmic drop iR_u is the voltage drop developed across the solution resistance R_u between the reference electrode and the working electrode, when current is flowing through. When the product iR_u gets significant it introduces an important error in the control of the working electrode potential and should be compensated.

In controlled potential techniques, The Manual IR (MIR) can be used to compensate the ohmic drop when the uncompensated solution resistance value (R_u) is known or measured before the experiment start. This technique will not measure R_u .

When used with linked techniques and loops, this technique allow the user to keep the same R_u value for each loop. The user can select the percentage of compensation. It is highly recommended to not exceed 85% of the R_u measured value in order to avoid oscillations of the instrument.

7.28.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

MIR parameters			
Label	Description	Data type	Data range
Rcmp_Value	R value to compensate	single	≥ 0
Rcmp_Mode	Ohmic compensation mode: 0 = software 1 = hardware (VMP-300 only)	integer	0 or 1

To deactivate the compensation, you can simply set the Rcmp_value to 0 and Rcmp_Mode to software.

7.28.3. Data format

No data recorded by this technique, it has been designed for linked experiments.

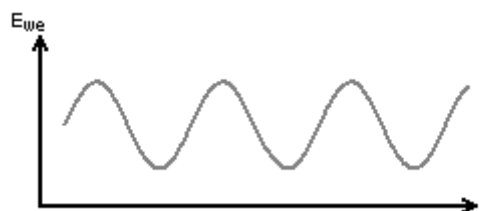
7.28.4. Data conversion

No data conversion.

7.29. IR Determination with Potentiostatic Impedance technique

Technique ID: **141**

Instrument Series	VMP3	VMP-300
File	pzir.ecc	pzir4.ecc pzir5.ecc
Timebase	24μs	24μs



7.29.1. Description

The ohmic drop iR_u is the voltage drop developed across the solution resistance R_u between the reference electrode and the working electrode, when current is flowing through. When the product iR_u gets significant it introduces an important error in the control of the working electrode potential and should be compensated.

The IR Determination with Potentiostatic Impedance (PZIR) technique utilizes Impedance measurements to determine the R_u Value. This technique applies a sinusoidal excitation around the DC potential measured at the beginning of the technique. PZIR technique determines the solution resistance R_u , for one high frequency value, as the real part of the measured impedance. A percentage of the R_u value will be used to compensate next potentiostatic techniques. It is highly recommended to not exceed 85% of the R_u measured value in order to avoid oscillations of the instrument. The `Rcmp_Mode` parameter will allow to specify the compensation mode for the next potentiostatic techniques (only for VMP-300 series).

When used in linked techniques including loops, R_u value can change during the experiment. PZIR can be an ideal tool to do a dynamic ohmic drop compensation between repeated techniques.

For low impedance electrochemical systems it is recommended to use GZIR instead of PZIR.

7.29.2. Technique parameters

Technique parameters available for the function `BL_LoadTechnique`:

PZIR parameters

Label	Description	Data types	Data range
Final_frequency	Final frequency (Hz)	single	= Initial_frequency
Initial_frequency	Initial frequency (Hz)	single	Depend on instrument

Amplitude_Voltage	Sinus amplitude (V)	single	Depend on instrument
Average_N_times	Number of repeat times (used for frequencies averaging)	integer	≥ 1
Wait_for_steady	Number of period to wait before each frequency	single	≥ 0
sweep	sweep linear/logarithmic (TRUE for linear points spacing)	boolean	= True
Rcomp_Level	% IR compensation	single	≥ 0
Rcmp_Mode	Ohmic drop compensation mode. 0 = software 1 = hardware (VMP-300 only)	integer	0 or 1

To deactivate the compensation, you can simply set the Rcomp_Level to 0 and Rcmp_Mode to software.

7.29.3. Data format

Data format returned by the function BL_GetData for VMP3:

	0	1	2	3	4	5	6	7	8	9	...
	freq	Ewe	I	Phase Zwe	Ewe	I	-	Ece	Ice	Phase Zce	...
	<i>Point #0</i>										
...	10	11	12	13	14	15	16	17	18		...
...	Ece	-	-	t	Irangle	freq	Ewe	I	Phase Zwe		...
	<i>Point #0</i>					<i>Point #1</i>					
...	19	20	21	22	23	24	25	26	27	28	...
...	Ewe	I	-	Ece	Ice	Phase Zce	Ece	-	-	t	...
	<i>Point #1</i>										
...	29	...									
...	Irangle	...									

... **29** ...

Point #1

For VMP-300:

	0	1	2	3	4	5	6	7	8	9	...
	freq	Ewe	I	Phase Zwe	Ewe	I	-	Ece	Ice	Phase Zce	...
	<i>Point #0</i>										...
...	10	11	12	13	14	15	16	...			...
...	Ece	-	t	freq	Ewe	I	Phase Zwe	...			...
	<i>Point #0</i>			<i>Point #1</i>				...			...
...	17	18	19	20	21	22	23	24	25	...	...
...	Ewe	I	-	Ece	Ice	Phase Zce	Ece	-	t	...	...
	<i>Point #1</i>										...

The number of points saved in the buffer is returned in the field TDataInfos.NbRows.
The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.29.4. Data conversion

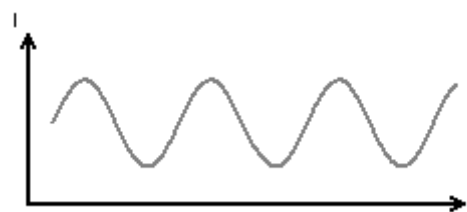
Data returned into the buffer are not usable as-is, one must convert the data before:

- time:
The time for process 0 should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
time (only time of process 1), IRange, freq, Ewe, |Ewe|, Ece, |Ece|, I, |I|, Phase Zwe, Phase Zce must be converted with the function BL_ConvertChannelNumericIntoSingle

7.30. IR Determination with GalvanoStatic Impedance technique

Technique ID: **142**

Instrument Series	VMP3	VMP-300
File	gzir.ecc	gzir4.ecc gzir5.ecc
Timebase	24μs	24μs



7.30.1. Description

The ohmic drop iR_u is the voltage drop developed across the solution resistance R_u between the reference electrode and the working electrode, when current is flowing through. When the product iR_u gets significant it introduces an important error in the control of the working electrode potential and should be compensated.

The IR Determination with Galvanostatic Impedance (GZIR) technique utilizes Impedance measurements to determine the R_u Value. This technique applies a sinusoidal excitation around the DC current measured at the beginning of the technique. GZIR technique determines the solution resistance R_u , for one high frequency value, as the real part of the measured impedance. A percentage of the R_u value will be used to compensate next potentiostatic techniques. It is highly recommended to not exceed 85% of the R_u measured value in order to avoid oscillations of the instrument.

When used in linked techniques including loops, R_u value can change during the experiment. GZIR can be an ideal tool to do a dynamic ohmic drop compensation between repeated techniques.

In the case of particular non-linear systems it can be necessary to use PZIR instead of GZIR.

7.30.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

GZIR parameters

Label	Description	Data types	Data range
Final_frequency	Final frequency (Hz)	single	= Initial_frequency
Initial_frequency	Initial frequency (Hz)	single	Depend on instrument

Amplitude_Current	Sinus amplitude (A)	singleDepen d on instrument	Depend on instrument
Average_N_times	Number of repeat times (used for frequencies averaging)	integer	≥ 1
Rcomp_Level	% IR compensation	single	≥ 0
Wait_for_steady	Number of period to wait before each frequency	single	≥ 0
sweep	sweep linear/logarithmic (TRUE for linear points spacing)	boolean	= True
I_Range	I range	integer	see IRange constants authorized on section 5. Structures and Constants Warning : I Auto- range is not allowed
Rcmp_Mode	Ohmic drop compensation mode. 0 = software 1 = hardware	integer	0 or 1

To deactivate the compensation, you can simply set the Rcomp_Level to 0 and Rcmp_Mode to software.

7.30.3. Data format

Data format returned by the function BL_GetData for VMP3:

	0	1	2	3	4	5	6	7	8	9	...
	freq	Ewe	I	Phase Zwe	Ewe	I	-	Ece	Ice	Phase Zce	...
	<i>Point #0</i>										
...	10	11	12	13	14	15	16	17	18	...	...
...	Ece	-	-	t	Irange	freq	Ewe	I	Phase Zwe	...	...
	<i>Point #0</i>						<i>Point #1</i>				

...	20	21	22	23	24	25	26	27	28	29	...
...	Ewe	I	-	Ece	Ice	Phase Zce	Ece	-	-	t	...
<i>Point #1</i>											
...	29	...									
...	Irangle	...									
<i>Point #1</i>											

For VMP-300:

	0	1	2	3	4	5	6	7	8	9	...	
	freq	Ewe	I	Phase Zwe	Ewe	I	-	Ece	Ice	Phase Zce	...	
												
	<i>Point #0</i>											
...	10	11	12	13	14	15	16					...
...	Ece	-	t	freq	Ewe	I	Phase Zwe					...
												
	<i>Point #0</i>			<i>Point #1</i>								
...	17	18	19	20	21	22	23	24	25	...		
...	Ewe	I	-	Ece	Ice	Phase Zce	Ece	-	t	...		
												
	<i>Point #1</i>											

The number of points saved in the buffer is returned in the field TDataInfos.NbRows.
The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.30.4. Data conversion

Data returned into the buffer are not usable as-is, one must convert the data before:

- time:
The time for process 0 should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
time (only time of process 1), IRange, freq, Ewe, |Ewe|, Ece, |Ece|, I, |I|, Phase Zwe, Phase Zce must be converted with the function BL_ConvertChannelNumericIntoSingle

7.31. Loop technique

Technique ID: **150**

Instrument Series	VMP3	VMP-300
File	loop.ecc	loop4.ecc loop5.ecc
Timebase	20μs	20μs

7.31.1. Description

The loop technique has been built to repeat all or a part of an experiment made with several linked techniques. The user has to define the technique number N_e where he wants to go back to ($N_e = 0$ for the first technique of the experiment). Then all the techniques linked after the selected one will be repeated. The user has to choose the number of time nt that the experiment will be looped. An experiment with $nt = 2$ will have three loops. The loop technique can be also used as a mandatory goto technique when the experiment will be looped for unlimited number of times. To activate this mode the user has to put $nt = -1$.

7.31.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

Loop parameters			
Parameters	Description	Data type	Data range
loop_N_times	Loop N times	integer	≥ 1 for conditional goto -1 for mandatory goto
protocol_number	Index of the technique to be linked (index 0-based)	integer	≥ 0

7.31.3. Data format

No data recorded by this technique, it has been designed for linked experiments.

7.31.4. Data conversion

No data conversion.

7.32. Trigger Out technique

Technique ID: **151**

Instrument Series	VMP3	VMP-300
File	TO.ecc	TO4.ecc TO5.ecc
Timebase	20μs	20μs

7.32.1. Description

The 'Trigger Out' technique can be used to synchronize a potentiostat channel with an external instrument. A trigger out pulse is generated by the potentiostat during the technique placed after the 'Trigger Out' technique. The pulse duration and level can be programmed by the user. The pulse cannot last more than the next technique duration. Before and after the pulse the potentiostat drives the trigger out signal to the default level set by the 'Trigger Out Set' technique.

7.32.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

TO parameters			
Parameters	Description	Data type	Data range
Trigger_Logic	Trigger out level	integer	0 or 1
Trigger_Duration	Trigger out duration (s)	single	≥ 0

7.32.3. Data format

No data recorded by this technique, it has been designed for linked experiments.

7.32.4. Data conversion

No data conversion.

7.33. Trigger In technique

Technique ID: **152**

Instrument Series	VMP3	VMP-300
File	TI.ecc	TI4.ecc TI5.ecc
Timebase	20μs	20μs

7.33.1. Description

The 'Trigger In' technique can be used to synchronize a potentiostat channel with an external instrument. The potentiostat waits an external trigger to continue the experiment with the technique set after the 'Trigger In' technique. Before receiving the trigger the potentiostat goes to the next technique control mode. The trigger in signal is level sensitive and can be set to be either logic low or high. For the potentiostat to recognize the trigger a pulse must be set and held for a minimum of 100 μs.

7.33.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

TI parameters			
Parameters	Description	Data type	Data range
Trigger_Logic	Trigger in level	integer	0 or 1

7.33.3. Data format

No data recorded by this technique, it has been designed for linked experiments.

7.33.4. Data conversion

No data conversion.

7.34. Trigger Out Set technique

Technique ID: **153**

Instrument Series	VMP3	VMP-300
File	TOS.ecc	TOS4.ecc TOS5.ecc
Timebase	20μs	20μs

7.34.1. Description

The 'Trigger Out Set' technique can be used together with the 'Trigger Out' technique to synchronize the potentiostat with an external instrument. 'Trigger Out Set' technique sets the default level of the trigger out signal to be either at a logic low or high level. Before and after a pulse generated by the 'Trigger Out' technique the potentiostat drives the trigger out signal to the default level. The trigger out default level can be changed only by another execution of a 'Trigger Out Set' technique or by a power-up or reset of the potentiostat.

7.34.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

TOS parameters

Parameters	Description	Data type	Data range
Trigger_Value	Trigger out value	integer	0 or 1

7.34.3. Data format

No data recorded by this technique, it has been designed for linked experiments.

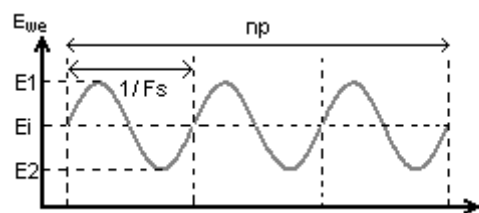
7.34.4. Data conversion

No data conversion.

7.35. Large Amplitude Sinusoidal Voltammetry technique

Technique ID: **159**

Instrument Series	VMP3	VMP-300
File	lasv.ecc	lasv4.ecc lasv5.ecc
Timebase	50μs	50μs



7.35.1. Description

Large Amplitude Sinusoidal Voltammetry (LASV) is an electrochemical technique where the potential excitation of the working electrode is a large amplitude sinusoidal waveform. Similar to the cyclic voltammetry (CV) technique, it gives qualitative and quantitative information on redox processes. In contrast to the CV, the double layer capacitive current is not subject to sharp transitions at reverse potentials. Since the electrochemical systems are non-linear the current response exhibits higher order harmonics at large sinusoidal amplitudes. Valuable information can be found from data analysis in the frequency domain.

7.35.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

LASV parameters			
Label	Description	Data types	Data range
Ei	Initial potential	single	-
Ei_vs_initial	Initial potential vs initial one	boolean	True/False
Fs	Frequency of applied sinusoidal	Array of 20 single	≥ 0
E1	High Potential of sinusoidal (V)	Array of 20 single	-
E1_vs_initial	Voltage E1 vs initial one	Array of 20 boolean	True/False
E2	Low Potential of sinusoidal (V)	Array of 20 single	-
vs_initial	Voltage E2 vs initial one	Array of 20 boolean	True/False
Period_number	Number of periods	Array of 20 integer	≥ 0

LASV parameters

Label	Description	Data types	Data range
Record_every_dT	Record every dt (s)	Array of 20 single	≥ 0
Record_every_dI	Record every dI (A)	Array of 20 single	≥ 0
Step_number	Number of steps minus 1	integer	[0..19]
N_Cycles	Number of times the technique is repeated	integer	≥ 0

This parameters cannot be updated with BL_UpdateParameters function.

7.35.3. Data format

See CP technique data format.

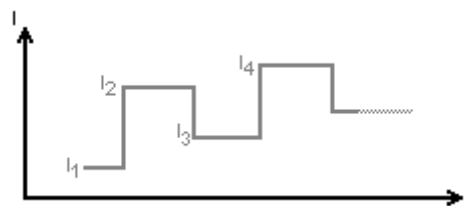
7.35.4. Data conversion

See CP technique data conversion.

7.36. Chrono-Potentiometry technique with limits

Technique ID: **155**

Instrument Series	VMP3	VMP-300
File	cplimit.ecc	cplimit4.ecc cplimit5.ecc
Timebase	50μs	34μs



7.36.1. Description

The Chronopotentiometry (CP) is a controlled current technique. The current is controlled and the potential is the variable determined as a function of time. The chronopotentiometry technique is similar to the Chronoamperometry technique, potential steps being replaced by current steps. The current is applied between the working and the counter electrode.

This technique can be used for different kind of analysis or to investigate electrode kinetics. It is considered less sensitive than voltammetric techniques for analytical uses. Generally, the curves $E_{we} = f(t)$ contains plateaus that correspond to the redox potential of electroactive species.

Three limits (tests) are available. These three limits can be combined with logical operator (AND, OR). A limit is defined with a variable, a compare operator, and a value.

The variables are potential (E), current (I), potential on Auxiliary 1 (AUX1) or potential on auxiliary 2 (AUX2).

The logical operators are "<" or ">".

The value is a single data type value.

7.36.1. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

CPLIMIT parameters			
Label	Description	Data types	Data range
Current_step	Current step (A)	Array of 20 single	-
vs_initial	Current step vs initial one	Array of 20 boolean	True/False
Duration_step	Duration step (s)	Array of 20 single	[0 ... $tb \cdot 2^{31}$]
Step_number	Number of steps	integer	[0 ... 19]

**CPLIMIT
parameters**

Label	Description	Data types	Data range
	minus 1		
Record_every_dT	Record every dt (s)	single	≥ 0.0
Record_every_dE	Record every dE (V)	single	≥ 0.0
N_Cycles	Number of times the technique is repeated	integer	≥ 0
Test1_Config	Configuration of Test1 by step	Array of 20 integer	See format below
Test1_Value	Value of Test1 by step	Array of 20 single	-
Test2_Config	Configuration of Test1 by step	Array of 20 integer	See format below
Test2_Value	Value of Test2 by step	Array of 20 single	-
Test3_Config	Configuration of Test3 by step	Array of 20 integer	See format below
Test3_Value	Value of Test3 by step	Array of 20 single	-
Exit_Cond	Exit condition	Array of 20 integer	0: Next Step 1: Next Technique 2: STOP Experiment
I_Range	I range	integer	see IRange constants allowed on section 5. Structures and Constants Warning: I Auto-range is not allowed

Test configuration format:

The test configuration is a 32-bits integer.

31	...	5	4	3	2	1	0
Variable			Sign			Logic	Active

E (Voltage) = 0			
AUX1 (Auxiliary Input 1) = 1	"<" = 0	OR = 0	active = 1
AUX2 (Auxiliary Input 2) = 2	">" = 1	AND = 1	not active = 0
I (Current) = 3			

If Test3 is active, Test1 and Test2 must be active. The Test3 is ignored if Test2 or Test1 is inactive. The comparison formula is: (Test1 AND/OR Test2) AND/OR Test3.

If Test2 is active, Test1 must be active. The Test2 is ignored if Test1 is inactive. The comparison formula is: Test1 AND/OR Test2.

Test value format:

The test value is a single-precision floating point value. It should be set regarding the test configuration:

- the limit in V for the configuration with E;
- the limit in V for the configuration with AUX1;
- the limit in V for the configuration with AUX2;
- the limit in A for the configuration with I.

7.36.2. Data format

See CP technique data format.

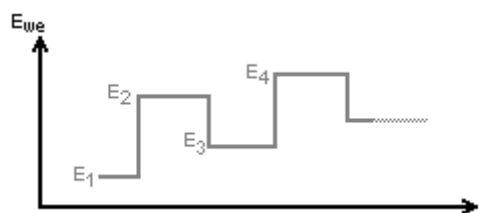
7.36.3. Data conversion

See CP technique data format.

7.37. Chrono-Amperometry technique with limits

Technique ID: **157**

Instrument Series	VMP3	VMP-300
File	calimit.ecc	calimit4.ecc calimit5.ecc
Timebase	50μs	34μs



7.37.1. Description

The basis of the controlled-potential techniques is the measurement of the current response to an applied potential step.

The Chronoamperometry (CA) technique involves stepping the potential of the working electrode from an initial potential, at which (generally) no faradic reaction occurs, to a potential E_i at which the faradic reaction occurs. The current-time response reflects the change in the concentration gradient in the vicinity of the surface.

Chronoamperometry is often used for measuring the diffusion coefficient of electroactive species or the surface area of the working electrode. This technique can also be applied to the study of electrode processes mechanisms.

An alternative and very useful mode for recording the electrochemical response is to integrate the current, so that one obtains the charge passed as a function of time. This is the chronocoulometric mode that is particularly used for measuring the quantity of adsorbed reactants.

Three limits (tests) are available. These three limits can be combined with logical operator (AND, OR). A limit is defined with a variable, a compare operator, and a value.

The variables are potential (E), current (I), potential on Auxiliary 1 (AUX1) or potential on auxiliary 2 (AUX2).

The logical operators are "<" or ">".

The value is a single data type value.

7.37.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

CALIMIT parameters			
Label	Description	Data types	Data range
Voltage_step	Voltage step (V)	Array of 20 single	-

**CALIMIT
parameters**

Label	Description	Data types	Data range
vs_initial	Voltage step vs initial one	Array of 20 boolean	True/False
Duration_step	Duration step (s)	Array of 20 single	[0 ... $tb \cdot 2^{31}$]
Step_number	Number of steps minus 1	integer	[0 ... 19]
Record_every_dT	Record every dt (s)	single	≥ 0.0
Record_every_dI	Record every dI (A)	single	≥ 0.0
Test1_Config	Configuration of Test1 by step	Array of 20 integer	See format below
Test1_Value	Value of Test1 by step	Array of 20 single	-
Test2_Config	Configuration of Test1 by step	Array of 20 integer	See format below
Test2_Value	Value of Test2 by step	Array of 20 single	-
Test3_Config	Configuration of Test3 by step	Array of 20 integer	See format below
Test3_Value	Value of Test3 by step	Array of 20 single	-
Exit_Cond	Exit condition	Array of 20 integer	0: Next Step 1: Next Technique 2: STOP Experiment
N_Cycles	Number of times the technique is repeated	integer	≥ 0

Test configuration format:

The test configuration is a 32-bits integer.

31	...	5	4	3	2	1	0
Variable			Sign		Logic		Active

E (Voltage) = 0			
AUX1 (Auxiliary Input 1) = 1	"<" = 0	OR = 0	active = 1
AUX2 (Auxiliary Input 2) = 2	">" = 1	AND = 1	not active = 0
I (Current) = 3			

If Test3 is active, Test1 and Test2 must be active. The Test3 is ignored if Test2 or Test1 is inactive. The comparison formula is: (Test1 AND/OR Test2) AND/OR Test3.

If Test2 is active, Test1 must be active. The Test2 is ignored if Test1 is inactive. The comparison formula is: Test1 AND/OR Test2.

Test value format:

The test value is a single-precision floating point value. It should be set regarding the test configuration:

- the limit in V for the configuration with E;
- the limit in V for the configuration with AUX1;
- the limit in V for the configuration with AUX2;
- the limit in A for the configuration with I.

7.37.3. Data format

See CP technique data format.

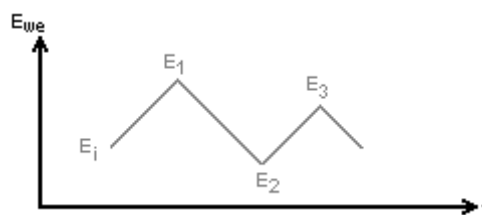
7.37.4. Data conversion

See CP technique data conversion.

7.38. Voltage Scan technique with limits

Technique ID: **158**

Instrument Series	VMP3	VMP-300
File	vscanlimit.ecc	vscanlimit4.ecc vscanlimit5.ecc
Timebase	60μs	60μs



7.38.1. Description

The Potentiodynamic (PDYN) technique allows the user to perform potentiodynamic periods with different scan rates.

Three limits (tests) are available. These three limits can be combined with logical operator (AND, OR). A limit is defined with a variable, a compare operator, and a value.

The variables are potential (E), current (I), potential on Auxiliary 1 (AUX1) or potential on auxiliary 2 (AUX2).

The logical operators are "<" or ">".

The value is a single data type value.

7.38.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

PDYNLIMIT parameters

Parameters	Description	Data types	Data range
Voltage_step	Vertex potential (V)	Array of 20 single	-
vs_initial	Vertex potential vs initial one	Array of 20 boolean	True/False
Scan_Rate	Scan rate (V/s) from previous vertex potential	Array of 20 single	> 0, Value of the first scan-rate is ignored
Scan_number	Number of scans minus 1	integer	[0 ... 19]
Record_every_dE	Record every dE (V)	single	≥ 0.0
N_Cycles	Number of times the	integer	≥ 0

PDYNLIMIT parameters

Parameters	Description	Data types	Data range
	technique is repeated		
Begin_measuring_I	Select the part of the potential step.	signed integer [0 .. 1.0]	
End_measuring_I	1 = 100% used for data averaging	signed integer [0 .. 1.0]	
Test1_Config	Configuration of Test1 by step	Array of 20 integer	See format below
Test1_Value	Value of Test1 by step	Array of 20 single	-
Test2_Config	Configuration of Test1 by step	Array of 20 integer	See format below
Test2_Value	Value of Test2 by step	Array of 20 single	-
Test3_Config	Configuration of Test3 by step	Array of 20 integer	See format below
Test3_Value	Value of Test3 by step	Array of 20 single	-
Exit_Cond	Exit condition	Array of 20 integer	0: Next Step 1: Next Technique 2: STOP Experiment

Test configuration format:

The test configuration is a 32-bits integer.

31	...	5	4	3	2	1	0
Variable			Sign			Logic	Active
E (Voltage) = 0							
AUX1 (Auxiliary Input 1) = 1						OR = 0	active = 1
AUX2 (Auxiliary Input 2) = 2						AND = 1	not active = 0
I (Current) = 3							

If Test3 is active, Test1 and Test2 must be active. The Test3 is ignored if Test2 or Test1 is inactive. The comparison formula is: (Test1 AND/OR Test2) AND/OR Test3.

If Test2 is active, Test1 must be active. The Test2 is ignored if Test1 is inactive. The comparison formula is: Test1 AND/OR Test2.

Test value format:

The test value is a single-precision floating point value. It should be set regarding the test configuration:

- the limit in V for the configuration with E;
- the limit in V for the configuration with AUX1;
- the limit in V for the configuration with AUX2;
- the limit in A for the configuration with I.

7.38.3. Data format

See PDYN technique data format.

7.38.4. Data conversion

See PDYN technique data conversion.

7.38.5. Delphi Example

```
var ID: int32; {device identifier}
function LoadTechniques: boolean;
var
  {VSCANLIMIT}
  EccParamArray_VSCAN: array of TEccParam;
  EccParams_VSCAN: TEccParams;
  fname_VSCAN: Pchar;

begin
  Result:= FALSE;
  fname_VSCAN:= nil;
  SetLength(EccParamArray_VSCAN, 28);

  try
    {Define VSCANLIMIT parameters}
    fname_VSCAN:= StrNew('vscanlimit4.ecc');
    {Vertex #0}
    BL_DefineSglParameter ('Voltage_step', 0.0, 0, @EccParamArray_VSCAN[0]);
    BL_DefineBoolParameter('vs_initial', FALSE, 0, @EccParamArray_VSCAN[1]);
    BL_DefineSglParameter ('Scan_Rate', 0.0, 0, @EccParamArray_VSCAN[2]);
    {Limits #0: ((E < 1.0V) OR (AUX2 > 3.0V)) AND (I < 0.05A)}
    BL_DefineIntParameter ('Test1_Config', 1, 0, @EccParamArray_VSCAN[3]);
    BL_DefineSglParameter ('Test1_Value', 1.0, 0, @EccParamArray_VSCAN[4]);
    BL_DefineIntParameter ('Test2_Config', 71, 0, @EccParamArray_VSCAN[5]);
    BL_DefineSglParameter ('Test2_Value', 3.0, 0, @EccParamArray_VSCAN[6]);
    BL_DefineIntParameter ('Test3_Config', 97, 0, @EccParamArray_VSCAN[7]);
    BL_DefineSglParameter ('Test3_Value', 0.05, 0, @EccParamArray_VSCAN[8]);
    BL_DefineIntParameter ('Exit_Cond', 0, 0, @EccParamArray_VSCAN[9]);
    {Vertex #1}
    BL_DefineSglParameter ('Voltage_step', 1.0, 1, @EccParamArray_VSCAN[10]);
    BL_DefineBoolParameter('vs_initial', FALSE, 1, @EccParamArray_VSCAN[11]);
```

```

BL_DefineSglParameter ('Scan_Rate', 10.0, 1, @EccParamArray_VSCAN[12]);
{Limits #1: (I > 0.1A)}
BL_DefineIntParameter ('Test1_Config', 101, 1, @EccParamArray_VSCAN[13]);
BL_DefineSglParameter ('Test1_Value', 0.1, 1, @EccParamArray_VSCAN[14]);
BL_DefineIntParameter ('Test2_Config', 0, 1, @EccParamArray_VSCAN[15]);
BL_DefineSglParameter ('Test2_Value', 0.0, 1, @EccParamArray_VSCAN[16]);
BL_DefineIntParameter ('Test3_Config', 0, 1, @EccParamArray_VSCAN[17]);
BL_DefineSglParameter ('Test3_Value', 0.0, 1, @EccParamArray_VSCAN[18]);
BL_DefineIntParameter ('Exit_Cond', 2, 1, @EccParamArray_VSCAN[19]);
{Misc.}
BL_DefineIntParameter ('Scan_number', 2, 0,
    @EccParamArray_VSCAN[20]);
BL_DefineIntParameter ('N_Cycles', 0, 0,
    @EccParamArray_VSCAN[21]);
BL_DefineSglParameter ('Record_every_dE', 0.01, 0,
    @EccParamArray_VSCAN[22]);
BL_DefineSglParameter ('Begin_measuring_I', 0.4, 0,
    @EccParamArray_VSCAN[23]);
BL_DefineSglParameter ('End_measuring_I', 0.8, 0,
    @EccParamArray_VSCAN[24]);
BL_DefineIntParameter ('I_Range', IRANGE_10MA, 0,
    @EccParamArray_VSCAN[25]);
BL_DefineIntParameter ('E_Range', ERANGE_AUTO, 0,
    @EccParamArray_VSCAN[26]);
BL_DefineIntParameter ('Bandwidth', BANDWIDTH_5, 0,
    @EccParamArray_VSCAN[27]);
{Load VSCANLIMIT on selected channel}
EccParams_VSCAN.len:= length(EccParamArray_VSCAN);
EccParams_VSCAN.pParams:= @EccParamArray_VSCAN[0];
if BL_LoadTechnique(ID, {device identifier}
    0, {selected channel}
    fname_VSCAN, {*.ecc filename(c-string)}
    EccParams_VSCAN, {parameters}
    TRUE, {first technique}
    TRUE, {last technique}
    FALSE) <> 0 then Exit; {display params}

Result:= TRUE;

finally
    if fname_VSCAN <> nil then StrDispose(fname_VSCAN);
    SetLength(EccParamArray_VSCAN, 0);

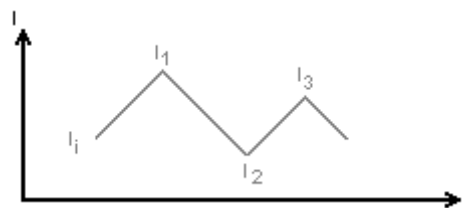
end;
end;

```

7.39. Current Scan technique with limits

Technique ID: **156**

Instrument Series	VMP3	VMP-300
File	iscanlimit.ecc	iscanlimit4.ecc iscanlimit5.ecc
Timebase	60μs	60μs



7.39.1. Description

The Galvanodynamic (GDYN) technique allows the user to perform galvanodynamic periods with different scan rates.

Three limits (tests) are available. These three limits can be combined with logical operator (AND, OR). A limit is defined with a variable, a compare operator, and a value.

The variables are potential (E), current (I), potential on Auxiliary 1 (AUX1) or potential on auxiliary 2 (AUX2).

The logical operators are "<" or ">".

The value is a single data type value.

7.39.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

GDYNLIMIT parameters

Label	Description	Data types	Data range
Current_step	Vertex current (A)	Array of 20 single	-
vs_initial	Vertex current vs initial one	Array of 20 boolean	True/False
Scan_Rate	Scan rate (A/s) from previous vertex current	Array of 20 single	> 0.0
Scan_number	Number of scans minus 1	integer	[0 ... 19]
Record_every_dI	Record every dI (A)	single	≥ 0.0
N_Cycles	Number of times the technique is repeated	integer	≥ 0

GDYNLIMIT parameters

Label	Description	Data types	Data range
Begin_measuring_	Select the part of the current step (1 = 100%) used for data averaging	single	[0.0 ... 1.0
End_measuring_E		single	[0.0 ... 1.0]
Test1_Config	Configuration of Test1 by step	Array of 20 integer	See format below
Test1_Value	Value of Test1 by step	Array of 20 single	-
Test2_Config	Configuration of Test1 by step	Array of 20 integer	See format below
Test2_Value	Value of Test2 by step	Array of 20 single	-
Test3_Config	Configuration of Test3 by step	Array of 20 integer	See format below
Test3_Value	Value of Test3 by step	Array of 20 single	-
Exit_Cond	Exit condition	Array of 20 integer	0: Next Step 1: Next Technique 2: STOP Experiment
I_Range	I range	integer	see IRange constants authorized on section 5. Structures and Constants Warning : I Auto-range is not allowed

Test configuration format:

The test configuration is a 32-bits integer.

31	...	5	4	3	2	1	0
Variable			Sign			Logic	Active
E (Voltage) = 0			" $<$ " = 0 " $>$ " = 1			OR = 0	active = 1
AUX1 (Auxiliary Input 1) = 1						AND = 1	not active = 0
AUX2 (Auxiliary Input 2) = 2							
I (Current) = 3							

If Test3 is active, Test1 and Test2 must be active. The Test3 is ignored if Test2 or Test1 is inactive. The comparison formula is: (Test1 AND/OR Test2) AND/OR Test3.

If Test2 is active, Test1 must be active. The Test2 is ignored if Test1 is inactive. The comparison formula is: Test1 AND/OR Test2.

Test value format:

The test value is a single-precision floating point value. It should be set regarding the test configuration:

- the limit in V for the configuration with E;
- the limit in V for the configuration with AUX1;
- the limit in V for the configuration with AUX2;
- the limit in A for the configuration with I.

7.39.3. Data format

See GDYN technique data format.

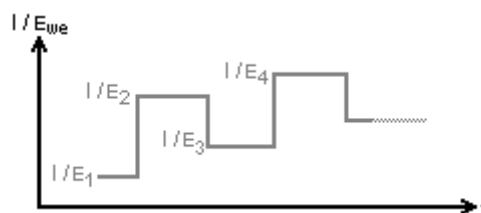
7.39.4. Data conversion

See GDYN technique data format.

7.40. Modular Pulse technique

Technique ID: **167**

Instrument Series	VMP3	VMP-300
File	mp.ecc	mp4.ecc mp5.ecc
Timebase	100μs	100μs



7.40.1. Description

The Modular pulse technique (MOD) allows the user to control successively in different sequences the current and/or the voltage of the cell. With this technique including galvanostatic and potentiostatic sequences, the switch from one mode to the other is very fast. The recording conditions included in the sequence (r_c) offer the possibility to record only few sequences in a long time experiment. This technique is particularly useful for electrochemical coating.

7.40.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

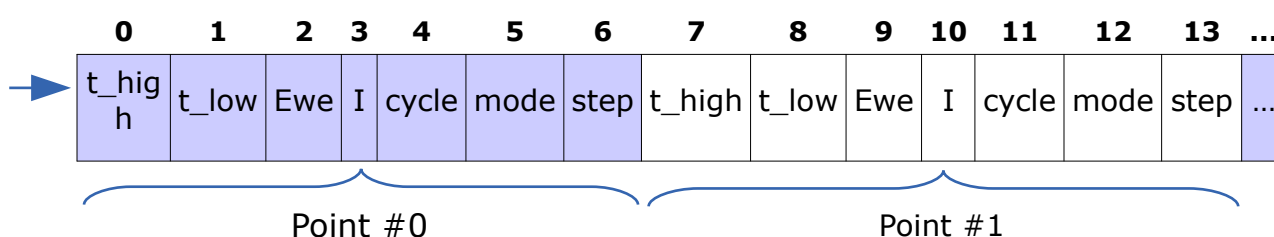
MOD parameters			
Label	Description	Data types	Data range
Value_step	Voltage step (V) in Potentiostatic mode or Current step (A) in Galvanostatic mode	Array of 20 single	-
vs_initial	Voltage / Current step vs initial one	Array of 20 boolean	True/False
Duration_step	Duration step (s)	Array of 20 single	$\geq 2*tb$
Record_every_dT	Record every dt (s)	Array of 20 single	≥ 0
Record_every_dM	Record every dI (A) in Potentiostatic Record every dE (V) in Galvanostatic	Array of 20 single	≥ 0
Mode_step	Potentiostatic or Galvanostatic	Array of 20 integer	0 : Potentiostatic 1 : Galvanostatic

MOD parameters

Label	Description	Data types	Data range
Step_number	Number of steps minus 1	integer	[0..19]
Record_every_rc	Record every cycle	integer	≥ 0
N_Cycles	Number of times the technique is repeated	integer	≥ 0

7.40.3. Data format

Data format returned by the function BL_GetData :



The number of points saved in the buffer is returned in the field TDataInfos.NbRows.
The number of variables defining a point is returned in the field TDataInfos.NbCols.

7.40.4. Data conversion

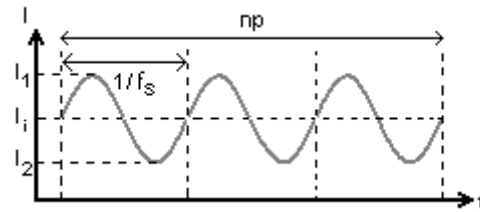
Data returned into the buffer are not usable as-is, one must convert the data before:

- time:
The time should be converted using the function BL_ConvertTimeChannelNumericIntoSeconds
- Float conversion:
Ewe and I must be converted with the function BL_ConvertChannelNumericIntoSingle
- cycle:
no conversion needed
- mode:
no conversion needed. '0' is the potentio mode and '1' is the galvano mode.
- step:
no conversion needed

7.41. Constant Amplitude Sinusoidal micro Galvano polarization technique

Technique ID: **169**

Instrument Series	VMP3	VMP-300
File	casg.ecc	casg4.ecc casg5.ecc
Timebase	50μs	50μs



7.41.1. Description

Constant Amplitude Sinusoidal micro Galvano polarization (CASG) is a technique similar than CASP. But in that case, the perturbation is performed around an initial current (I_i) with a small amplitude (I_a) and a constant low frequency (f_s). Thanks to a Direct Fourier Transform the amplitudes of the fundamental frequency (f_s), 1st ($2 f_s$) and 2nd ($3 f_s$) harmonics are determined. This technique can also be used for other applications such as battery, fuel cell, ...

7.41.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

CASG parameters			
Label	Description	Data types	Data range
I_i	Initial current	single	-
$I_i_vs_initial$	Initial current vs initial one	boolean	True/False
F_s	Frequency of applied sinusoidal	Array of 20 single	≥ 0
I_1	High current of sinusoidal (A)	Array of 20 single	-
$I_1_vs_initial$	Current I_1 vs initial one	Array of 20 boolean	True/False
I_2	Low Current of sinusoidal (A)	Array of 20 single	-
$I_2_vs_initial$	Current I_2 vs initial one	Array of 20 boolean	True/False
Period_number	Number of periods	Array of 20 integer	≥ 0

CASG parameters

Label	Description	Data types	Data range
Record_every_dT	Record every dt (s)	Array of 20 single	≥ 0
Record_every_dE	Record every dE (V)	Array of 20 single	≥ 0
Step_number	Number of steps minus 1	integer	[0..19]
N_Cycles	Number of times the technique is repeated	integer	≥ 0

7.41.3. Data format

See CP technique data format.

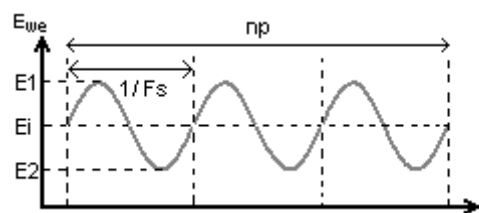
7.41.4. Data conversion

See CP technique data conversion.

7.42. Constant Amplitude Sinusoidal micro Potentio polarization technique

Technique ID: **170**

Instrument Series	VMP3	VMP-300
File	casp.ecc	casp4.ecc casp5.ecc
Timebase	50μs	50μs



7.42.1. Description

Constant Amplitude Sinusoidal micro Potentio polarization (CASP) is a technique used to determine the corrosion current and the *Tafel* coefficients. In this technique, a sinusoidal voltage is applied around a potential (E_i) with a small amplitude (V_a) and a constant low frequency (f_s). Thanks to a Direct Fourier Transform, the amplitudes of the fundamental frequency (f_s), 1st ($2 f_s$) and 2nd ($3 f_s$) harmonics are determined and used to calculate the corrosion current and the *Tafel* coefficients. This technique was designed to be faster than the usual linear polarization around the corrosion potential and, *compared to the Tafel fit, does not require an adjustment of the Tafel parameters to have access to I_{corr}* .

7.42.2. Technique parameters

Technique parameters available for the function BL_LoadTechnique:

CASP parameters			
Label	Description	Data types	Data range
E_i	Initial potential	single	-
$E_{i_vs_initial}$	Initial potential vs initial one	boolean	True/False
F_s	Frequency of applied sinusoidal	Array of 20 single	≥ 0
E_1	High Potential of sinusoidal (V)	Array of 20 single	-
$E_{1_vs_initial}$	Voltage E_1 vs initial one	Array of 20 boolean	True/False
E_2	Low Potential of sinusoidal (V)	Array of 20 single	-
$vs_initial$	Voltage E_2 vs initial	Array of 20 boolean	True/False

CASP parameters

Label	Description	Data types	Data range
	one		
Period_number	Number of periods	Array of 20 integer	≥ 0
Record_every_dT	Record every dt (s)	Array of 20 single	≥ 0
Record_every_dI	Record every dI (A)	Array of 20 single	≥ 0
Step_number	Number of steps minus 1	integer	[0..19]
N_Cycles	Number of times the technique is repeated	integer	≥ 0

This parameters cannot be updated with BL_UpdateParameters function.

7.42.3. Data format

See CP technique data format.

7.42.4. Data conversion

See CP technique data conversion.

8. Global parameters for hardware configuration

8.1 Electrode connection

The configuration parameters can be used with all techniques. The configuration parameters modification follows the same method as the technique parameters one.

CE to ground Configuration :

Electrode connection parameters (only for VMP3 series)

Parameters	Description	Data type	Data range
ce	CE to ground	integer	CE to ground mode: ce = 1 and em = 3
em	Controlled potential mode	integer	Standard mode: ce = 0 and em = 0

Note:

The functions BL_SetHardConf and BL_GetHardConf must be only used with VMP-300 series in order to change electrode connection mode. See [5. Structures and Constants](#) and [6. Functions reference](#) for more details.

8.2 Instrument ground

This parameters is only used with VMP-300 series. The functions BL_SetHardConf and BL_GetHardConf must be used to modify or get the value of the parameter. See [6. Functions reference](#).

8.3 Record and external control options

These options can be used only with VMP-300 series.

Record and external control options parameters (only for VMP-300 series)

Parameters	Description	Data type	Data range
Rcmp_Mod	Solution ohmic drop mode of	integer	0 = software based (via

Record and external control options parameters (only for VMP-300 series)

e	compensation. The VMP-300 has a hardware control of this compensation that can be enabled with this flag.		the <i>Rcmp_Value</i> or <i>Rcomp_Level</i> parameter) 1 = Hardware based (faster)																		
xctr	Bitfield controlling some extra options: Activation of External control, or record of additional values: Ece, Analog IN1 & 2, Ramp, Charge, Control and IRange values Conversion to binary value to activate each options (see data range). 1 : activated 0 : not activated	Integer / bitfield	<table><tr><th>Bit</th><th>Option</th></tr><tr><td>1</td><td>Record Ece</td></tr><tr><td>2</td><td>Record Analog IN1</td></tr><tr><td>3</td><td>Record Analog IN2</td></tr><tr><td>4</td><td>Enable External ctrl</td></tr><tr><td>5</td><td>Reserved</td></tr><tr><td>6</td><td>Record Control</td></tr><tr><td>7</td><td>Record Charge</td></tr><tr><td>8</td><td>Record IRange</td></tr></table> All remaining bits are reserved. For example, activate External and record Charge. Value in: binary = 0b01001000 integer = 72 hex = 0x0048	Bit	Option	1	Record Ece	2	Record Analog IN1	3	Record Analog IN2	4	Enable External ctrl	5	Reserved	6	Record Control	7	Record Charge	8	Record IRange
Bit	Option																				
1	Record Ece																				
2	Record Analog IN1																				
3	Record Analog IN2																				
4	Enable External ctrl																				
5	Reserved																				
6	Record Control																				
7	Record Charge																				
8	Record IRange																				
R32	Ece ERange	integer	see ERange constants authorized on section 5. Structures and Constants																		
raux1	Analog IN 1 ERange	integer	see ERange constants authorized on section 5. Structures and Constants																		
raux2	Analog IN 2 ERange	integer	see ERange constants authorized on section 5. Structures and Constants																		

Note:

- **The external control option allows to control the potentiostat / galvanostat from an external signal source through the Analog IN**

2 input. For the potentio techniques the input voltage simply adds on the technique waveform. For the galvano techniques the input voltage is converted into a current. A simply rule of thumb is to consider 1V as the full scale of the selected current range. For more precise control one should divide the input voltage, (which should not exceed $\pm 1V$) by the value of the shunt resistor. The input voltage has no effect if a technique is not running.

APPENDIX A. Find instruments

The **EC-Lab[®] Development Package** includes a library (**blfind.dll**) to find available instruments (Ethernet and USB).

Calling conventions

The library uses the **stdcall** calling conventions for all exported functions.

Multi-thread applications

All exported functions are protected by a synchronization object, they can be called in a multi-thread application.

Data types

See section 3.4. Data types

Functions reference

Function	BL_FINDECHEMDEV
Syntax	<pre>function BL_FindEChemDev(pLstdev: PChar; psize: uint32; pNbrDev : uint32): int32;</pre>
Parameters	<i>pLstdev</i> pointer to the buffer that will receive the serialization of instruments description (C-string format) (see section 5 Serializations format) <i>psize</i> pointer to a uint32 specifying the maximum number of characters of the buffer. It also returns the number of characters of the copied serialization. <i>pNbrDev</i> pointer to a uint32 receiving the number of detected Ethernet and USB instruments.
Return value	= 0 : the function succeeded < 0 : see section 6 Error codes
Description	This function finds Ethernet and USB electrochemistry instruments and copies into the buffer a serialization of descriptions of detected instruments.
Delphi example	<pre>procedure FindEchemDevice; var</pre>

```

pSerialization: PChar;
len, nbDev: uint32;
Err: int32;
begin
  len := 8192;
  pSerialization := StrAlloc(len);
  zeromemory(pSerialization, len);
  Err := BL_FindEChemDev( pSerialization, @len, @nbDev);
  ShowMessage('Instruments detected : ' + IntToStr(nbDev));
  StrDispose(pSerialization);
end;

```

Function BL_FINDECHEMETHDEV

Syntax	<pre> function BL_FindEChemEthDev(pLstdev: PChar; psize: puint32; pNbrDev : puint32): int32; </pre>
Parameters	<i>pLstdev</i> pointer to the buffer that will receive the serialization of instruments description (C-string format) (see section 5 Serializations format) <i>psize</i> pointer to a uint32 specifying the maximum number of characters of the buffer. It also returns the number of characters of the copied serialization. <i>pNbrDev</i> pointer to a uint32 receiving the number of detected Ethernet instruments.
Return value	= 0 : the function succeeded < 0 : see section 6 Error codes
Description	This function finds Ethernet electrochemistry instruments and copies into the buffer a serialization of descriptions of detected instruments.
Delphi example	<pre> procedure FindEchemEthDevice; var pSerialization: PChar; len, nbDev: uint32; Err: int32; begin len := 4096; pSerialization := StrAlloc(len); zeromemory(pSerialization, len); Err := BL_FindEChemEthDev(pSerialization, @len, @nbDev); ShowMessage('Instruments detected : ' + IntToStr(nbDev)); StrDispose(pSerialization); end; </pre>

Function	BL_FINDECHEMUSBDEV
Syntax	<pre>function BL_FindEChemUsbDev(pLstdev: PChar; psize: uint32; pNbrDev : uint32): int32;</pre>
Parameters	<i>pLstdev</i> pointer to the buffer that will receive the serialization of instruments description (C-string format) (see section 5 Serializations format) <i>psize</i> pointer to a uint32 specifying the maximum number of characters of the buffer. It also returns the number of characters of the copied serialization. <i>pNbrDev</i> pointer to a uint32 receiving the number of detected USB instruments.
Return value	= 0 : the function succeeded < 0 : see section 6 Error codes
Description	This function finds USB electrochemistry instruments and copies into the buffer a serialization of descriptions of the detected instruments.
Delphi example	<pre>procedure FindEchemUsbDevice; var pSerialization: PChar; len, nbDev: uint32; Err: int32; begin len := 4096; pSerialization := StrAlloc(len); zeromemory(pSerialization, len); Err := BL_FindEChemUsbDev(pSerialization, @len, @nbDev); ShowMessage('Instruments detected : ' + IntToStr(nbDev)); StrDispose(pSerialization); end;</pre>

Function	BL_SETCONFIG
Syntax	<pre>function BL_SetConfig(pIP: PChar; pCfg: PChar): int32;</pre>
Parameters	<i>pIP</i> pointer to the buffer specifying the IP address of the instrument to configure. <i>pCfg</i>

	pointer to the buffer specifying the new TCP/IP parameters of the instrument (see section 5 Serializations format)
Return value	= 0 : the function succeeded < 0 : see section 6 Error codes
Description	This function sets new TCP/IP parameters of selected instrument. IP address, netmask and gateway may be modified.
Delphi example	<pre> procedure SetTCPIP; var IPaddress: array[0..15] of char; newCfg: array[0..57] of char; vErr : int32; begin IPaddress := '192.109.209.220' + #0; newCfg := 'IP%192.109.209.22\$' + 'NM%255.255.255.0\$' + 'GW%192.109.209.170\$' + #0; vErr := BL_SetConfig(IPaddress, newCfg); end; </pre>

Procedure BL_GETERRORMSG

Syntax	<pre> procedure BL_GetErrorMsg(errorcode: int32; pmsg: PChar; psize: uint32); </pre>
Parameters	<i>errorcode</i> error code selected <i>pmsg</i> pointer to the buffer that will receive the text (C-string format) <i>psize</i> pointer to a uint32 specifying the maximum number of characters of the buffer. It also returns the number of characters of the copied string.
Return value	None
Description	This function copies into the buffer the corresponding message of the selected error code.
Delphi	<pre> procedure DisplayErrorMessage; </pre>

example

```

var
  msg: PChar;
  len: uint32;
begin
  len := 255;
  msg := StrAlloc(len);
  zeromemory(msg, len);
  BL_GetErrorMsg(-20, msg, @len);
  ShowMessage(msg);
  StrDispose(msg);
end;

```

Serializations format

5.1. Instruments descriptions

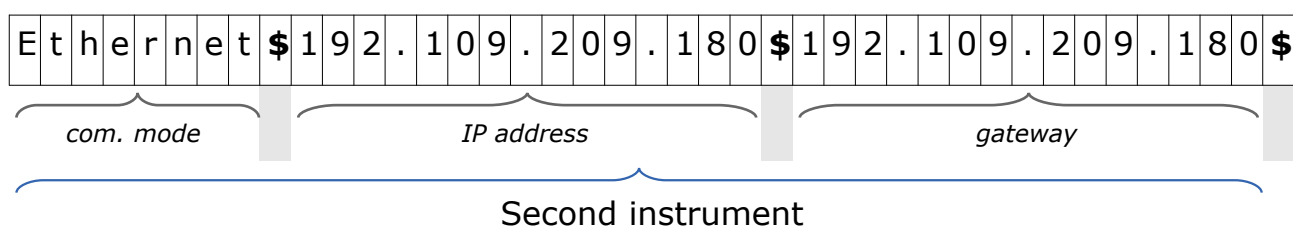
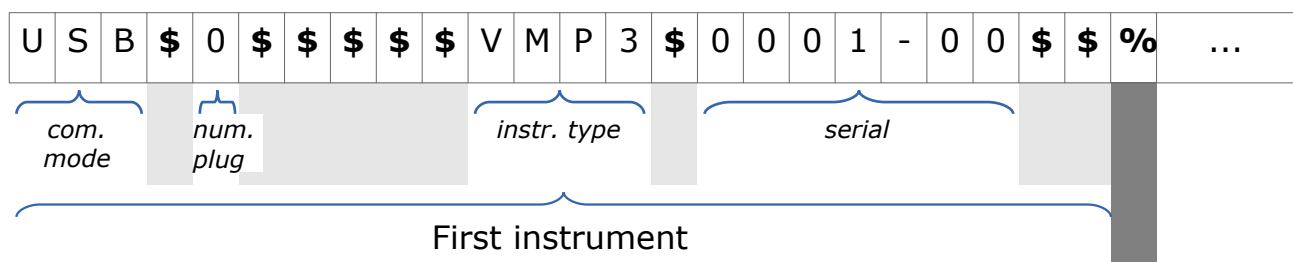
This serialization consists of an array of characters containing the descriptions of detected instruments. If more than one instrument is detected, serialized descriptions are separated by the character '% '.

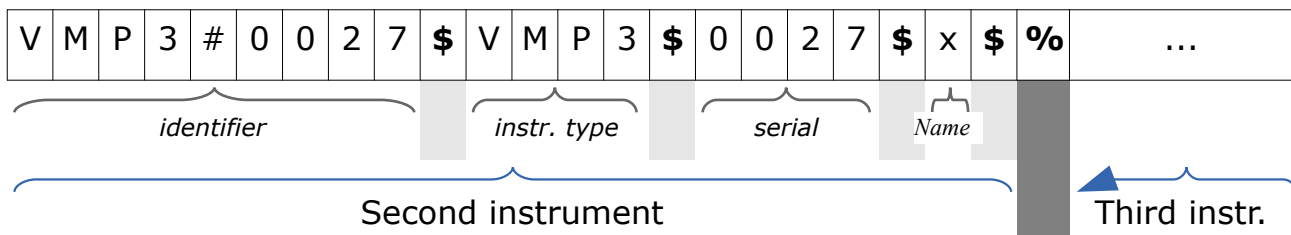
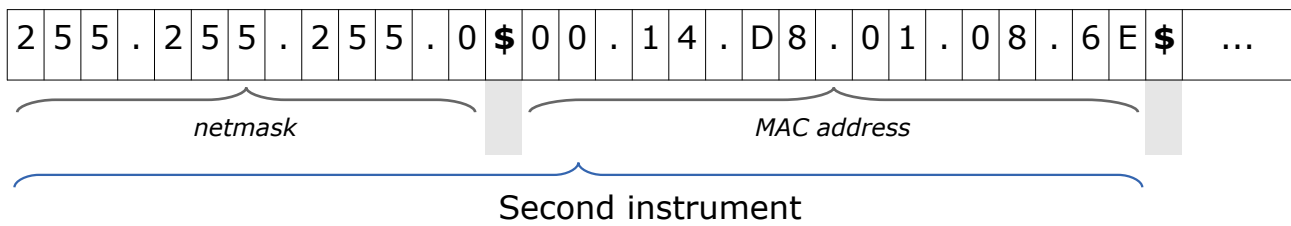
An instrument description is defined by a set of 9 string descriptors.

In the serialization, descriptors are separated by the character '\$' and are always serialized in the same order :

- (1) **Connection mode**
- (2) **IP address or USB plug index**
- (3) **Gateway** (always empty with USB)
- (4) **Netmask** (always empty with USB)
- (5) **MAC address** (always empty with USB)
- (6) **Identifier** (always empty with USB)
- (7) **Instrument type**
- (8) **Serial number**
- (9) **Name** (always empty with USB)

Example of instruments description serialization :





5.2. TCP/IP parameters

This serialization consists of an array of characters containing the definitions of TCP/IP parameters. If more than one parameter is defined, they must be separated by the character '\$'.

Each parameter must be defined by an identifier and a value, separated by the character '%'.

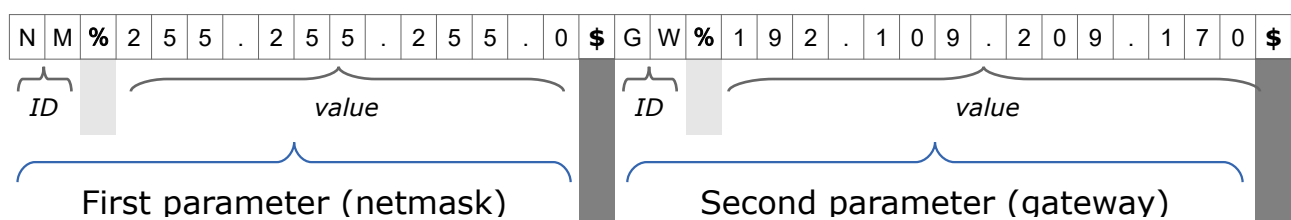
Identifier must be one of the followings key words :

- **IP** : for IP address
- **NM** : for netmask
- **GW** : for gateway

There is no order in the definition of TCP/IP parameters.

Key words must be in upper case.

Example of TCP/IP parameters serialization :



Error codes

General error codes

Constant	Value	Description
BLFIND_ERR_UNKNOWN	-1	unknown error
BLFIND_ERR_INVALID_PARAMETER	-2	invalid function parameters

Instrument error codes

Constant	Value	Description
BLFIND_ERR_ACK_TIMEOUT	-10	instrument response timeout
BLFIND_ERR_EXP_RUNNING	-11	experiment is running on instrument
BLFIND_ERR_CMD_FAILED	-12	instrument do not execute command

Find error codes

Constant	Value	Description
BLFIND_ERR_FIND_FAILED	-20	find failed
BLFIND_ERR_SOCKET_WRITE	-21	cannot write the request of the descriptions of Ethernet instruments
BLFIND_ERR_SOCKET_READ	-22	cannot read descriptions of Ethernet instrument

Modify error codes

Constant	Value	Description
BLFIND_ERR_CFG_MODIFY_FAILED	-30	set TCP/IP parameters failed
BLFIND_ERR_READ_PARAM_FAILED	-31	deserialization of TCP/IP parameters failed
BLFIND_ERR_EMPTY_PARAM	-32	not any TCP/IP parameters in serialization
BLFIND_ERR_IP_FORMAT	-33	invalid format of IP address
BLFIND_ERR_NM_FORMAT	-34	invalid format of netmask address
BLFIND_ERR_GW_FORMAT	-35	invalid format of gateway address
BLFIND_ERR_IP_NOT_FOUND	-38	instrument to modify not found
BLFIND_ERR_IP_ALREADYEXIST	-39	new IP address in TCP/IP parameters already exists

APPENDIX B. Version Compatibility

By default, the user should use technique files, firmware binaries from the same EC-Lab® Development Package version. To verify if it is the case, the version compatibility is listed below.

Version Compatibility List for VMP-300 Series

EC-Lab® Development Package	DSP Channel Firmware	FPGA Channel Firmware	ECLib.dll	BLFind.dll
V6.08	540 (V5.40)	43551 (0xAA1F)	V6.8.0.0	V1.18.0
V6.06	537 (V5.35)	43548 (0xAA1C)	V6.5.4.1	V1.18.0
V6.05	536 (V5.35)	43548 (0xAA1C)	V6.5.2.1	V1.16.0
V6.04	535 (V5.35)	43548 (0xAA1C)	V6.4.0.0	V1.16.0
V6.03	534 (V5.34)	43548 (0xAA1C)	V6.3.0.0	V1.15.0
V6.02	533 (V5.33)	43548 (0xAA1C)	V6.2.0.0	V1.14.0
V6.00	533 (V5.33)	43548 (0xAA1C)	V6.0.0.0	V1.7.0
V5.39	532 (V5.32)	43546 (0xAA1A)	V5.36.0.0	V1.7.0
V5.38	531 (V5.31)	43546 (0xAA1A)	V5.35.0.0	V1.7.0
V5.37	531 (V5.31)	43546 (0xAA1A)	V5.35.0.0	V1.7.0
...	...	...	...	...
V5.28	527 (V5.27)	43544 (0xAA18)	V5.28.0.0	V1.1.0
V5.27	527 (V5.27)	43544 (0xAA18)	V5.27.0.0	V1.1.0

Version Compatibility List for VMP3 Series

EC-Lab® Development Package	DSP Channel Firmware	FPGA Channel Firmware	ECLib.dll	BLFind.dll
V6.08	5282 (V5.28.2)	42511 (0xA60F)	V6.8.0.0	V1.18.0
V6.06	5282 (V5.28.2)	42511 (0xA60F)	V6.5.4.1	V1.18.0
V6.05	5282 (V5.28.2)	42511 (0xA60F)	V6.5.2.1	V1.16.0
V6.04	5282 (V5.28.2)	42511 (0xA60F)	V6.4.0.0	V1.16.0
V6.03	5282 (V5.28.2)	42511 (0xA60F)	V6.3.0.0	V1.15.0
V6.02			V6.2.0.0	V1.14.0
V6.00			V6.0.0.0	V1.7.0

Version Compatibility List for VMP3 Series

EC-Lab® Development Package	DSP Channel Firmware	FPGA Channel Firmware	ECLib.dll	BLFind.dll
V5.39			V5.36.0.0	V1.7.0
V5.38			V5.35.0.0	V1.7.0
V5.37			V5.35.0.0	V1.7.0
...	...	...	...	...
V5.28			V5.28.0.0	V1.1.0
V5.27			V5.27.0.0	V1.1.0